

# Contents

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Plantillas y utilidades</b>           | <b>2</b>  |
| 1.1      | Plantilla main + fast I/O                | 2         |
| 1.2      | Random / mt19937                         | 2         |
| 1.3      | BigInt (add + multiply, base 1e9)        | 3         |
| 1.4      | Coordinate compression                   | 4         |
| <b>2</b> | <b>Matematicas</b>                       | <b>4</b>  |
| 2.1      | Teoria de numeros                        | 4         |
| 2.1      | Extended Euclidean Algorithm             | 4         |
| 2.2      | Formulas utiles                          | 5         |
| 2.3      | Criba de Eratostenes                     | 5         |
| 2.4      | Criba lineal                             | 5         |
| 2.5      | Miller-Rabin primality                   | 6         |
| 2.6      | Pollard's Rho + factorizacion            | 6         |
| 2.7      | Modular exponentiation                   | 7         |
| 2.8      | Modular inverse                          | 7         |
| 2.9      | Chinese Remainder Theorem                | 7         |
| 2.10     | Euler totient precompute                 | 8         |
| 2.11     | Mobius precompute                        | 8         |
| 2.12     | Stern-Brocot (best fraction in interval) | 8         |
| 2.2      | Combinatoria                             | 9         |
| 2.1      | Catalan (formula)                        | 9         |
| 2.2      | Factoriales + nCr mod p                  | 9         |
| 2.3      | Lucas theorem                            | 9         |
| 2.4      | Stirling S2 (particiones)                | 10        |
| 2.5      | Stirling S1 (permutaciones por ciclos)   | 10        |
| 2.6      | Inclusion-exclusion over subsets         | 10        |
| 2.3      | Polinomios y transformadas               | 11        |
| 2.1      | FFT (iterativo, complex)                 | 11        |
| 2.2      | NTT (mod 998244353)                      | 11        |
| 2.3      | Polynomial utilities (mod p)             | 12        |
| 2.4      | Algebra lineal                           | 12        |
| 2.1      | Gauss-Jordan (mod p)                     | 12        |
| 2.2      | Matrix exponentiation                    | 13        |
| 2.5      | Juegos y teoria de juegos                | 14        |
| 2.1      | Sprague-Grundy template                  | 14        |
| 2.2      | Small game tables (Kayles SG values)     | 14        |
| <b>3</b> | <b>Estructuras de datos</b>              | <b>14</b> |
| 3.1      | Ordenado y contenedores                  | 14        |
| 3.1      | Ordered Set (pbds)                       | 14        |
| 3.2      | Basis (XOR basis)                        | 14        |
| 3.3      | Trie                                     | 15        |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 3.2      | Union-Find                             | 15        |
| 3.1      | DSU classico                           | 15        |
| 3.2      | DSU con rollback                       | 16        |
| 3.3      | Arboles de segmentos                   | 16        |
| 3.1      | Segment Tree Add + Sum                 | 16        |
| 3.2      | Segment Tree Lazy Assign + Sum         | 17        |
| 3.3      | Segment Tree Add + lower bound index   | 17        |
| 3.4      | Segment Tree Lazy Add + Set + Sum      | 18        |
| 3.5      | MO                                     | 19        |
| 3.6      | Segment Tree 2D                        | 19        |
| 3.7      | Segment Tree persistente               | 20        |
| 3.8      | Segment Tree iterativo                 | 20        |
| 3.4      | Fenwick / BIT                          | 21        |
| 3.1      | BIT point update + prefix sum          | 21        |
| 3.2      | BIT range add + point query            | 21        |
| 3.3      | BIT range add + range sum              | 21        |
| 3.5      | RMQ y sparse table                     | 22        |
| 3.1      | Sparse Table                           | 22        |
| 3.2      | Disjoint Sparse Table                  | 22        |
| 3.6      | Trees y DP en arbol                    | 23        |
| 3.1      | Binary Lifting / LCA                   | 23        |
| 3.2      | Heavy-Light Decomposition (HLD)        | 23        |
| 3.3      | Centroid Decomposition                 | 24        |
| 3.4      | Euler Tour + range queries             | 24        |
| 3.7      | BST balanceado                         | 25        |
| 3.1      | Treap implicito                        | 25        |
| 3.8      | Otras                                  | 25        |
| 3.1      | Mo con updates                         | 25        |
| 3.2      | sqrt-decomposition generico            | 26        |
| <b>4</b> | <b>Strings</b>                         | <b>26</b> |
| 4.1      | Hashing                                | 26        |
| 4.1      | Rolling Hash (doble modulo)            | 26        |
| 4.2      | Patrones                               | 27        |
| 4.1      | KMP + LPS                              | 27        |
| 4.2      | Z-algorithm                            | 27        |
| 4.3      | Rabin-Karp (single hash)               | 28        |
| 4.4      | Aho-Corasick (basic)                   | 28        |
| 4.5      | Aho-Corasick DP for forbidden patterns | 29        |
| 4.3      | Estructuras avanzadas                  | 29        |
| 4.1      | Suffix Array                           | 29        |
| 4.2      | LCP Array (Kasai)                      | 30        |
| 4.3      | Suffix Automaton                       | 31        |
| 4.4      | Palindromos                            | 32        |
| 4.1      | Manacher                               | 32        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.5      | Otros . . . . .                                                | 33        |
| 4.1      | Lyndon factorization (Duval) . . . . .                         | 33        |
| <b>5</b> | <b>Grafos</b> . . . . .                                        | <b>33</b> |
| 5.1      | Caminos minimos . . . . .                                      | 33        |
| 5.1      | Dijkstra . . . . .                                             | 33        |
| 5.2      | Bellman-Ford . . . . .                                         | 34        |
| 5.3      | Floyd-Warshall . . . . .                                       | 34        |
| 5.4      | 0-1 BFS . . . . .                                              | 35        |
| 5.2      | Componentes y cortes . . . . .                                 | 35        |
| 5.1      | Kosaraju SCC . . . . .                                         | 35        |
| 5.2      | Tarjan (puentes) . . . . .                                     | 36        |
| 5.3      | Tarjan (puntos de articulacion) . . . . .                      | 36        |
| 5.4      | Tarjan SCC . . . . .                                           | 36        |
| 5.3      | Matching . . . . .                                             | 37        |
| 5.1      | Kuhn (bipartite matching) . . . . .                            | 37        |
| 5.2      | Hopcroft-Karp . . . . .                                        | 37        |
| 5.4      | MST . . . . .                                                  | 38        |
| 5.1      | Kruskal . . . . .                                              | 38        |
| 5.2      | Prim (priority queue) . . . . .                                | 39        |
| 5.5      | Basicos . . . . .                                              | 39        |
| 5.1      | DFS / BFS / componentes . . . . .                              | 39        |
| 5.2      | Topological sort (Kahn + DFS) . . . . .                        | 39        |
| 5.3      | DP en DAG (camino mas largo / corto con relajacion) . . . . .  | 40        |
| 5.4      | Bipartite check + 2-coloring . . . . .                         | 40        |
| 5.6      | Flujos . . . . .                                               | 41        |
| 5.1      | Ford-Fulkerson (DFS max flow) . . . . .                        | 41        |
| 5.2      | Edmonds-Karp (max flow) . . . . .                              | 41        |
| 5.3      | Dinic (max flow) . . . . .                                     | 42        |
| 5.4      | Min-Cost Max-Flow . . . . .                                    | 42        |
| 5.7      | Especiales . . . . .                                           | 43        |
| 5.1      | 2-SAT (implication graph + SCC) . . . . .                      | 43        |
| 5.2      | Euler tour / Hierholzer (camino y circuito) . . . . .          | 44        |
| 5.3      | Hamiltonian path / cycle (bitmask DP + backtracking) . . . . . | 46        |
| <b>6</b> | <b>DP</b> . . . . .                                            | <b>48</b> |
| 6.1      | Knapsack 0/1 . . . . .                                         | 48        |
| 6.2      | Knapsack unbounded . . . . .                                   | 48        |
| 6.3      | Knapsack meet-in-the-middle . . . . .                          | 48        |
| 6.4      | LIS $O(n \log n)$ . . . . .                                    | 49        |
| 6.5      | LCS / Edit distance . . . . .                                  | 50        |
| 6.6      | Bitmask DP over subsets . . . . .                              | 50        |
| 6.7      | SOS DP . . . . .                                               | 50        |
| 6.8      | Digit DP template . . . . .                                    | 52        |
| 6.9      | Tree DP (reroot) . . . . .                                     | 53        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>7</b> | <b>Geometria</b> . . . . .                     | <b>53</b> |
| 7.1      | Puntos y segmentos . . . . .                   | 53        |
| 7.1      | Conceptos y formulas de geometria 2D . . . . . | 53        |
| 7.2      | Point + cross + dot . . . . .                  | 55        |
| 7.3      | Interseccion de segmentos . . . . .            | 55        |
| 7.4      | Contains colinear . . . . .                    | 55        |
| 7.5      | Distance point-segment / point-line . . . . .  | 56        |
| 7.6      | Projection of point onto line . . . . .        | 56        |
| 7.2      | Poligonos . . . . .                            | 56        |
| 7.1      | Convex Hull (monotono) . . . . .               | 56        |
| 7.2      | Polygon Area . . . . .                         | 56        |
| 7.3      | Point in polygon (ray casting) . . . . .       | 57        |
| 7.4      | Minkowski sum (convex polygons) . . . . .      | 57        |
| 7.3      | Herramientas . . . . .                         | 57        |
| 7.1      | Li Chao Tree (long long) . . . . .             | 57        |
| 7.2      | Sweep line template . . . . .                  | 58        |
| 7.3      | Closest pair of points (sweepline) . . . . .   | 58        |
| 7.4      | Rotating calipers (polygon diameter) . . . . . | 58        |

# 1 Plantillas y utilidades

## 1.1 Plantilla main + fast I/O

```
1 // TC: N/A (boilerplate)
2 // usa: punto de partida para cualquier problema
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int main(){
7     ios_base::sync_with_stdio(false);
8     cin.tie(0);
9
10    // freopen("input.txt", "r", stdin);
11    // freopen("output.txt", "w", stdout);
12
13    int n;
14    if(!(cin >> n)) return 0;
15    vector<int> a(n);
16    for(int i=0;i<n;i++) cin >> a[i];
17
18    // minimal example: print sum of n integers
19    long long sum = 0;
20    for(int x : a) sum += x;
21    cout << sum << "\n";
22    return 0;
23
24    // === Usage examples of other modules (uncomment as needed) ===
25    // See each module's listing for its own minimal main.
26
27    // --- Utilities ---
28    // shuffleVec(a); // random / mt19937
29    // BigInt b(123), c(456); c += b; cout << c.toString();
30    // DBGV(a); // debug pretty-printer
31    // vector<int> cmp = compress(a); // coordinate compression
32
33    // --- Math ---
34    // sieve(100); cout << primes.size();
35    // linearSieve(100);
36    // cout << millerRabin(17ULL);
37    // vector<u64> f = factorize(123456ULL);
38    // cout << modpow(2, 10);
39    // cout << modInv(3, MOD);
40    // cout << crt({2, 3}, {5, 7});
41    // phiSieve(100); cout << phi[10];
42    // mobiusSieve(100);
43    // buildFact(20); cout << nCr(5, 2);
44    // cout << nCrLucas(1000, 100, MOD);
45    // buildStirlingS2(10);
46    // multiplyFFT({1,2,3}, {4,5});
47    // multiplyNTT({1,2,3}, {4,5});
48
49    // --- Data Structures ---
50    // DSUrb d(5); d.unite(0,1); d.unite(1,2); d.rollback();
51    // SegTree2D st(10, 10); st.update(3, 4, 7); cout << st.query(0,0,3,4);
52    // BIT bit(10); bit.add(3, 5); cout << bit.sum(5);
```

```
53 // SparseTable sp(a); cout << sp.query(1, 3);
54 // LCA lca(n); lca.addEdge(u,v); lca.build(0); cout << lca.lca(u,v);
55 // HLD hld(n); hld.build(); auto segs = hld.pathSegments(u,v);
56
57 // --- Strings ---
58 // cout << zFunction("abacaba")[3];
59 // auto occ = rabinKarp("hello world", "world");
60 // AhoCorasick ac; ac.addString("he",0); ac.addString("she",1); ac.
61 // build();
62 // auto cnt = ac.countOccurrences("ushers", 2);
63 // auto [odd, even] = manacher("abacaba");
64 // SuffArr sa(s); auto lcp = buildLCP(s, sa.p);
65 // SAM sam(s.size()); sam.build(s); cout << sam.countDistinct();
66
67 // --- Graphs ---
68 // Graph g(n); g.addEdge(0,1); cout << g.bfs(0)[1];
69 // auto order = topoKahn(n, adj);
70 // auto [ok, col] = isBipartite(n, adj);
71 // cout << zeroOneBFS(n, adj, 0)[n-1];
72 // cout << kruskal(n, edges);
73 // TarjanSCC scc(n, adj); cout << scc.numComps;
74 // Dinic dinic(n, s, t); dinic.addEdge(u,v,c); cout << dinic.maxflow();
75 // MCMF mcmf(n); mcmf.addEdge(u,v,c,k); auto [flow,cost] = mcmf.
76 // minCostMaxFlow(s,t);
77 // TwoSAT ts(n); ts.addOr(0, true, 1, false); auto val = ts.solve();
78
79 // --- DP ---
80 // cout << knapsack01({2,3,4}, {3,4,5}, 5);
81 // cout << lisLength({1,3,2,4});
82 // cout << lcs("abcde", "ace");
83 // cout << editDistance("kitten", "sitting");
84 // cout << sosDP({1,2,3,4})[3];
85 // cout << digitDP(1000);
86 // TreeDP tdp(n); tdp.addEdge(u,v); auto ans = tdp.solve();
87
88 // --- Geometry ---
89 // cout << dist2PointSegment(p, a, b);
90 // auto [px, py] = projectPointLine(p, a, b);
91 // cout << pointInPolygon(p, poly);
92 // auto msum = minkowskiSum(polyA, polyB);
93 // LiChao lc(0, 1e9); lc.addLine(Line(1, 0)); cout << lc.query(5);
94 // cout << closestPair(pts);
95 // cout << rotatingCalipers(convexPoly);
96 }
```

## 1.2 Random / mt19937

```
1 // TC: O(1) per call
2 // usa: shuffling deterministico, generacion de casos, hash randomization
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // Seed with fixed value for reproducible runs; change chrono for varied.
7 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
8
```

```

9 // integer in [lo, hi]
10 int randInt(int lo, int hi){
11     return uniform_int_distribution<int>(lo, hi)(rng);
12 }
13
14 // real in [lo, hi]
15 double randReal(double lo, double hi){
16     return uniform_real_distribution<double>(lo, hi)(rng);
17 }
18
19 // shuffle a vector in O(n)
20 template<class T>
21 void shuffleVec(vector<T>& v){
22     shuffle(v.begin(), v.end(), rng);
23 }
24
25 // pick a random element from a vector (non-empty)
26 template<class T>
27 T& randPick(vector<T>& v){
28     return v[uniform_int_distribution<int>(0, (int)v.size() - 1)(rng)];
29 }
30
31 int main(){
32     // shuffle a vector deterministically
33     vector<int> a = {1,2,3,4,5};
34     shuffleVec(a);
35     for(int x : a) cout << x << " ";
36     cout << "\n";
37
38     // random int in [lo, hi]
39     int r = randInt(1, 100);
40     cout << "randInt = " << r << "\n";
41
42     // shuffle with a fixed seed for reproducibility (replace rng
43     // temporarily)
44     mt19937_64 fixedRng(42);
45     shuffle(a.begin(), a.end(), fixedRng);
46     for(int x : a) cout << x << " ";
47     cout << "\n";
48     return 0;
49 }

```

### 1.3 BigInt (add + multiply, base 1e9)

```

1 // TC: O(N^2) multiply, O(N) add where N is the number of base-1e9 digits
2 // usa: problemas donde el resultado cabe en algunos miles de digitos (
3 // factoriales grandes, etc.)
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 struct BigInt{
8     // store digits in base 1e9 for compactness
9     static const int BASE = 1000000000;
10    vector<int> d; // least significant first
11    bool neg;

```

```

11
12 BigInt(): neg(false){}
13 BigInt(long long x){ *this = x; }
14
15 BigInt& operator=(long long x){
16     neg = x < 0;
17     x = abs(x);
18     d.clear();
19     do{
20         d.push_back((int)(x % BASE));
21         x /= BASE;
22     }while(x > 0);
23     return *this;
24 }
25
26 BigInt& operator+=(const BigInt& other){
27     // assumes same sign for simplicity; extend if needed
28     int carry = 0;
29     size_t n = max(d.size(), other.d.size());
30     if(d.size() < n) d.resize(n, 0);
31     for(size_t i=0;i<n;i++){
32         long long cur = d[i] + (i < other.d.size() ? other.d[i] : 0) +
33         carry;
34         d[i] = (int)(cur % BASE);
35         carry = (int)(cur / BASE);
36     }
37     if(carry) d.push_back(carry);
38     return *this;
39 }
40
41 BigInt& operator*=(int m){
42     // m fits in int (typical contest case)
43     long long carry = 0;
44     for(size_t i=0;i<d.size();i++){
45         long long cur = 1LL * d[i] * m + carry;
46         d[i] = (int)(cur % BASE);
47         carry = cur / BASE;
48     }
49     while(carry){
50         d.push_back((int)(carry % BASE));
51         carry /= BASE;
52     }
53     return *this;
54 }
55
56 string toString() const{
57     if(d.empty()) return "0";
58     stringstream ss;
59     if(neg) ss << "-";
60     ss << d.back();
61     char buf[16];
62     for(int i=(int)d.size()-2;i>=0;i--){
63         snprintf(buf, sizeof(buf), "%09d", d[i]);
64         ss << buf;
65     }
66     return ss.str();

```

```

66     }
67 };
68
69 int main(){
70     // compute 100! and print it
71     BigInt fact(1);
72     for(int i=2;i<=100;i++) fact *= i;
73     cout << fact.toString() << "\n";
74
75     // add two large numbers
76     BigInt a(123456789012345LL), b(987654321098765LL);
77     a += b;
78     cout << a.toString() << "\n";
79
80     // negative number
81     BigInt neg(-42);
82     cout << neg.toString() << "\n";
83     return 0;
84 }

```

## 1.4 Coordinate compression

```

1 // TC: O(N log N)
2 // usa: cuando los valores son hasta 1e9 pero solo hay N de ellos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // returns a vector of the same size as v, where each value is replaced
7 // by its rank in the sorted distinct set (0-indexed).
8 vector<int> compress(const vector<int>& v){
9     vector<int> sorted = v;
10    sort(sorted.begin(), sorted.end());
11    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
12    vector<int> res(v.size());
13    for(size_t i=0;i<v.size();i++){
14        res[i] = (int)(lower_bound(sorted.begin(), sorted.end(), v[i]) -
15        sorted.begin());
16    }
17    return res;
18 }
19
20 // if you also need the sorted distinct array itself:
21 pair<vector<int>, vector<int>> compressWithMap(const vector<int>& v){
22     vector<int> sorted = v;
23     sort(sorted.begin(), sorted.end());
24     sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());
25     vector<int> res(v.size());
26     for(size_t i=0;i<v.size();i++){
27         res[i] = (int)(lower_bound(sorted.begin(), sorted.end(), v[i]) -
28         sorted.begin());
29     }
30     return {res, sorted};
31 }
32
33 // 2D compression: compress both coordinates of each point independently

```

```

32 // usa: grids, point sets, rectangle problems, etc.
33 // T puede ser int o long long (u otro tipo ordenable)
34 // TC: O(N log N)
35 template<typename T>
36 vector<pair<int,int>> compress2D(const vector<pair<T,T>>& points){
37     vector<T> xs, ys;
38     xs.reserve(points.size());
39     ys.reserve(points.size());
40     for(int i=0; i<points.size(); i++){
41         xs.push_back(points[i].first);
42         ys.push_back(points[i].second);
43     }
44     sort(xs.begin(), xs.end());
45     xs.erase(unique(xs.begin(), xs.end()), xs.end());
46     sort(ys.begin(), ys.end());
47     ys.erase(unique(ys.begin(), ys.end()), ys.end());
48
49     vector<pair<int,int>> res(points.size());
50     for(int i=0; i<points.size(); i++){
51         int cx = (int)(lower_bound(xs.begin(), xs.end(), points[i].first) -
52         xs.begin());
53         int cy = (int)(lower_bound(ys.begin(), ys.end(), points[i].second) -
54         ys.begin());
55         res[i] = make_pair(cx, cy);
56     }
57     return res;
58 }
59
60 int main(){
61     // typical usage: compress values up to 1e9 to [0, k)
62     vector<int> v = {1e9, 5, 100, 5, 1e9, 42}; // considerar long long
63     vector<int> c = compress(v);
64     for(int x : c) cout << x << " "; // e.g. 3 0 2 0 3 1
65     cout << "\n";
66
67     // 2D example: compress points with coordinates up to 1e9
68     // xs unicos: [5, 100, 1e9] -> [0, 1, 2]
69     // ys unicos: [5, 42, 1e9] -> [0, 1, 2]
70     vector<pair<long long,long long>> pts = {{1e9, 5}, {5, 1e9}, {100, 42},
71     {5, 1e9}};
72     auto c2d = compress2D(pts);
73     for(auto& p : c2d) cout << "(" << p.first << "," << p.second << ") ";
74     cout << "\n"; // (2,0) (0,2) (1,1) (0,2)
75     return 0;
76 }

```

## 2 Matematicas

### 2.1 Teoria de numeros

#### 2.1 Extended Euclidean Algorithm

```

1 // TC: O(log(min(a, b)))

```

```

2 // SC: O(log(min(a, b))) por la pila de recursion
3 int extgcd(int a, int b, int &x, int &y){
4     if(b==0){
5         x=1; y=0;
6         return a;
7     }
8     int x1,y1;
9     int g=extgcd(b, a%b, x1, y1);
10    x = y1;
11    y = x1 - y1 * (a / b);
12    return g;
13 }
14
15 int main(){
16     // ejemplo 1: encontrar x, y tales que a*x + b*y = gcd(a, b)
17     int x, y;
18     int g = extgcd(12, 8, x, y);
19     cout << "gcd = " << g << "\n"; // 4
20     cout << "x = " << x << ", y = " << y << "\n"; // 1, -1
21     // verifica: 12*1 + 8*(-1) = 4
22
23     // ejemplo 2: inverso modular (cuando gcd(a, m) = 1)
24     // encuentra x tal que a*x congruente 1 (mod m)
25     int xi, yi;
26     int a = 3, m = 7;
27     int g2 = extgcd(a, m, xi, yi);
28     if(g2 == 1){
29         int inv = (xi % m + m) % m; // ajustar a [0, m)
30         cout << a << " * " << inv << " mod " << m << " = "
31             << (a * inv) % m << "\n"; // 3 * 5 mod 7 = 1
32     }
33     return 0;
34 }

```

## 2.2 Formulas utiles

Numero de divisores

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdot \dots \cdot (e_k + 1)$$

Suma de divisores

$$\sigma(n) = \frac{p_1^{e_1+1} - 1}{p_1 - 1} \cdot \frac{p_2^{e_2+1} - 1}{p_2 - 1} \cdot \dots \cdot \frac{p_k^{e_k+1} - 1}{p_k - 1}$$

Producto de divisores

$$P(n) = n^{\frac{d(n)}{2}}$$

Euler

$$a^b \equiv a^{b \bmod \varphi(n)} \pmod{n}$$

Inclusion – Exclusion

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{\emptyset \neq I \subseteq \{1,2,\dots,n\}} (-1)^{|I|+1} \left| \bigcap_{i \in I} A_i \right|$$

## 2.3 Criba de Eratostenes

```

1 // TC: O(N log log N) build, O(1) isPrime query
2 // usa: enumerar primos hasta N, comprobar primalidad
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> primes;
7 vector<bool> is_composite;
8
9 void sieve(int n){
10     is_composite.assign(n + 1, false);
11     is_composite[0] = is_composite[1] = true;
12     for(int i=2 ; i*i<=n ; i++){
13         if(!is_composite[i]){
14             for(int j=i*i; j<=n; j+=i) is_composite[j] = true;
15         }
16     }
17     primes.clear();
18     for(int i=2; i<=n; i++)
19         if(!is_composite[i]) primes.push_back(i);
20 }
21
22 int main(){
23     sieve(100);
24     for(int p : primes) cout << p << " ";
25     cout << "\n";
26     cout << "first 10 primes count: " << primes.size() << "\n";
27     return 0;
28 }

```

## 2.4 Criba lineal

```

1 // TC: O(N) build
2 // usa: cuando se necesita lp[i] (lowest prime factor) para cada i
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> lp; // lowest prime factor
7 vector<int> primes;
8
9 void linearSieve(int n){
10     lp.assign(n + 1, 0);
11     primes.clear();
12     for(int i=2 ; i<=n ; i++){
13         if(lp[i] == 0){
14             lp[i] = i;
15             primes.push_back(i);
16         }
17         for(int p : primes){
18             long long v = 1LL * i * p;
19             if(v > n) break;
20             lp[v] = p;
21             if(p == lp[i]) break;

```

```

22     }
23 }
24 }
25
26 int main(){
27     linearSieve(100);
28     // lp[12] = smallest prime factor of 12 (= 2)
29     cout << "lp[12] = " << lp[12] << "\n";
30     cout << "#primes <= 100: " << primes.size() << "\n";
31     return 0;
32 }

```

## 2.5 Miller-Rabin primality

```

1 // TC:  $O(\log^3 n)$  but with tiny constant
2 // usa: primalidad de numeros hasta  $2^{64}$ 
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 using u64 = unsigned long long;
7 using u128 = __uint128_t;
8
9 static u64 modpow_mr(u64 a, u64 e, u64 mod){
10     u64 r = 1;
11     a %= mod;
12     while(e){
13         if(e & 1) r = (u128)r * a % mod;
14         a = (u128)a * a % mod;
15         e >>= 1;
16     }
17     return r;
18 }
19
20 static bool millerRabin(u64 n){
21     if(n < 2) return false;
22     for(u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL,
23         29ULL, 31ULL, 37ULL}){
24         if(n % p == 0) return n == p;
25     }
26     u64 d = n - 1, s = 0;
27     while((d & 1) == 0){ d >>= 1; s++; }
28     // bases sufficient for deterministic test below  $2^{64}$ 
29     for(u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL,
30         1795265022ULL}){
31         if(a % n == 0) continue;
32         u64 x = modpow_mr(a, d, n);
33         if(x == 1 || x == n - 1) continue;
34         bool comp = true;
35         for(u64 r=1 ; r<s ; r++){
36             x = (u128)x * x % n;
37             if(x == n - 1){ comp = false; break; }
38         }
39         if(comp) return false;
40     }
41     return true;

```

```

40 }
41
42 int main(){
43     // test primality of several numbers up to  $2^{64}$ 
44     for(u64 n : {2ULL, 17ULL, 561ULL, 1000000007ULL, (u64)1e18}){
45         cout << n << ": " << (millerRabin(n) ? "prime" : "composite") << "\n"
46     };
47     return 0;
48 }

```

## 2.6 Pollard's Rho + factorizacion

```

1 // TC: expected  $O(n^{\frac{1}{4}})$  per factor
2 // usa: descomponer numeros hasta  $2^{64}$  en factores primos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 using u64 = unsigned long long;
7 using u128 = __uint128_t;
8
9 // assumes millerRabin and modpow_mr are defined above
10
11 static u64 modmul(u64 a, u64 b, u64 mod){
12     return (u128)a * b % mod;
13 }
14
15 static u64 pollard(u64 n){
16     if(n % 2 == 0) return 2;
17     static std::mt19937_64 rng((u64)chrono::steady_clock::now().
18         time_since_epoch().count());
19     while(true){
20         u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
21         u64 x = uniform_int_distribution<u64>(0, n - 1)(rng);
22         u64 y = x;
23         u64 d = 1;
24         auto f = [&](u64 v){ return (modmul(v, v, n) + c) % n; };
25         while(d == 1){
26             x = f(x);
27             y = f(f(y));
28             d = std::gcd<u64>(x > y ? x - y : y - x, n);
29         }
30         if(d != n) return d;
31     }
32 }
33
34 // returns sorted list of prime factors (with multiplicity)
35 void factorRec(u64 n, vector<u64>& out){
36     if(n == 1) return;
37     if(millerRabin(n)){ out.push_back(n); return; }
38     u64 d = pollard(n);
39     factorRec(d, out);
40     factorRec(n / d, out);
41 }

```

```

42 vector<u64> factorize(u64 n){
43     vector<u64> out;
44     factorRec(n, out);
45     sort(out.begin(), out.end());
46     return out;
47 }
48
49 int main(){
50     // factor 123456789012345 = 3^2 * 5 * 7 * 13 * 29 * 41 * 271 * 9091
51     auto f = factorize(123456789012345ULL);
52     for(u64 p : f) cout << p << " ";
53     cout << "\n";
54     return 0;
55 }

```

## 2.7 Modular exponentiation

```

1 // TC: O(log e)
2 // usa: base^exp % mod
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 static const long long MOD = 998244353LL;
7
8 long long modpow(long long a, long long e, long long mod = MOD){
9     long long r = 1;
10    a %= mod;
11    if(a < 0) a += mod;
12    while(e){
13        if(e & 1) r = r * a % mod;
14        a = a * a % mod;
15        e >>= 1;
16    }
17    return r;
18 }
19
20 int main(){
21     // 2^10 mod (1e9+7)
22     cout << modpow(2, 10) << "\n";
23     // custom mod
24     cout << modpow(2, 10, 1000000007LL) << "\n";
25     return 0;
26 }

```

## 2.8 Modular inverse

```

1 // TC: O(log mod)
2 // usa: 1/a mod p (con p primo), o por extended gcd si no
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // when MOD is prime

```

```

7 long long modInv(long long a, long long mod){
8     return modpow(a, mod - 2, mod);
9 }
10
11 // general version using extended gcd (works for any mod coprime with a)
12 long long extGcd(long long a, long long b, long long& x, long long& y){
13     if(b == 0){ x = 1; y = 0; return a; }
14     long long x1, y1;
15     long long g = extGcd(b, a % b, x1, y1);
16     x = y1;
17     y = x1 - y1 * (a / b);
18     return g;
19 }
20
21 long long modInvGeneral(long long a, long long mod){
22     long long x, y;
23     long long g = extGcd(a, mod, x, y);
24     if(g != 1) return -1; // no inverse
25     return (x % mod + mod) % mod;
26 }
27
28 int main(){
29     // inverse of 3 mod 1e9+7
30     cout << modInv(3, 1000000007LL) << "\n";
31     // general (via extgcd)
32     cout << modInvGeneral(7, 26LL) << "\n"; // 7*15 = 105 = 1 mod 26
33     return 0;
34 }

```

## 2.9 Chinese Remainder Theorem

```

1 // TC: O(N * log(max(m))) donde N = num. congruencias
2 // (cada modpow cuesta O(log m[i]))
3 // SC: O(1) extra
4 // usa: resolver sistema de congruencias pairwise coprimes
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 // returns smallest x >= 0 such that x == r[i] (mod m[i]) for all i
9 // assumes m[i] are pairwise coprime
10 long long crt(const vector<long long>& r, const vector<long long>& m){
11     long long x = 0;
12     long long M = 1; // modular inverse
13     for(int i=0 ; i<r.size() ; i++){
14         long long diff = (r[i] - x) % m[i];
15         if(diff < 0) diff += m[i];
16         long long inv = modpow(M % m[i], m[i] - 2, m[i]); // (a^b) % m[i]
17         long long t = diff * inv % m[i];
18         x = x + M * t;
19         M *= m[i];
20         x %= M;
21         if(x < 0) x += M;
22     }
23     return x;
24 }

```

```

25 int main(){
26     // x == 2 (mod 3), x == 3 (mod 5), x == 2 (mod 7) -> x = 23
27     cout << crt({2, 3, 2}, {3, 5, 7}) << "\n";
28     return 0;
29 }
30

```

## 2.10 Euler totient precompute

```

1 // TC: O(N log log N)
2 // usa: queries tipo "phi(n) para varios n hasta N"
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> phi;
7
8 void phiSieve(int n){
9     phi.assign(n + 1, 0);
10    for(int i=0;i<=n;i++) phi[i] = i;
11    for(int i=2;i<=n;i++){
12        if(phi[i] == i){ // prime
13            for(int j=i;j<=n;j+=i){
14                phi[j] -= phi[j] / i;
15            }
16        }
17    }
18 }
19
20 // single phi via factorization (slower but no precompute)
21 long long phiSingle(long long n){
22     long long result = n;
23     for(long long p=2 ; p*p<=n ; p++){
24         if(n % p == 0){
25             while(n % p == 0) n /= p;
26             result -= result / p;
27         }
28     }
29     if(n > 1) result -= result / n;
30     return result;
31 }
32
33 int main(){
34     phiSieve(100);
35     // phi[10] = 4 (numbers coprime with 10: 1,3,7,9)
36     for(int i=1 ; i<=10 ; i++){
37         cout << "phi(" << i << ") = " << phi[i] << "\n";
38     }
39     // single phi
40     cout << "phi(36) = " << phiSingle(36) << "\n";
41     return 0;
42 }

```

## 2.11 Mobius precompute

```

1 // TC: O(N log log N)
2 // usa: inclusion-exclusion con Mobius, queries tipo sum_{d|n} mu(d)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> mu;
7
8 void mobiusSieve(int n){
9     mu.assign(n + 1, 1);
10    vector<bool> is_composite(n + 1, false);
11    vector<int> primes;
12    mu[0] = 0;
13    mu[1] = 1;
14    for(int i=2;i<=n;i++){
15        if(!is_composite[i]){
16            primes.push_back(i);
17            mu[i] = -1;
18        }
19        for(int p : primes){
20            long long v = 1LL * i * p;
21            if(v > n) break;
22            is_composite[v] = true;
23            if(i % p == 0){
24                mu[v] = 0;
25                break;
26            }else{
27                mu[v] = -mu[i];
28            }
29        }
30    }
31 }
32
33 int main(){
34     mobiusSieve(20);
35     for(int i=1 ; i<=10 ; i++){
36         cout << "mu(" << i << ") = " << mu[i] << "\n";
37     }
38     return 0;
39 }

```

## 2.12 Stern-Brocot (best fraction in interval)

```

1 // TC: O(log(max(a,b))) per query
2 // usa: encontrar la fraccion mas simple p/q en un rango (lo, hi)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct Frac { long long p, q; };
7
8 // simplest = minimize p+q; iterate through mediants
9 // finds the fraction p/q with smallest p+q in the open interval (a/b
10 // , c/d)
11 // assumes a/b < c/d.
12 Frac bestInInterval(long long a, long long b, long long c, long long d){
13     long long p = a + c, q = b + d;
14 }

```

```

13 while(true){
14     long long k = max((p - a) / b, (p - c) / d);
15     // this is a coarse approximation; a full implementation needs limits
16     if(k <= 1) break;
17     p = a + k * c;
18     q = b + k * d;
19 }
20 return {p, q};
21 }
22
23 int main(){
24     // simplest fraction in (0/1, 1/1) is 1/2
25     auto f = bestInInterval(0, 1, 1, 1);
26     cout << f.p << "/" << f.q << "\n";
27     return 0;
28 }

```

## 2.2 Combinatoria

### 2.1 Catalan (formula)

```

1 // TC: O(n^2)
2 vector<int> dp;
3 const int MOD=998244353;
4
5 void catalan(int n){ // from 1 to n
6     dp.resize(n+1);
7     dp[0]=dp[1]=1;
8     for(int i=2 ; i<=n ; i++){
9         for(int j=0 ; j<i ; j++){
10             dp[i]=(dp[i]+(dp[j]*dp[i-j-1]))%MOD)%MOD;
11         }
12     }
13 }
14
15 // TC: O(n)
16 // consider use of long long
17 int catalan(int n){
18     int ans=1;
19     for(int i=0 ; i<n ; i++){
20         ans=ans*2*(2*i+1)/(i+2);
21     }
22     return ans;
23 }
24
25 int main(){
26     cout << "catalan(10) = " << catalan(10) << "\n";
27     return 0;
28 }

```

### 2.2 Factoriales + nCr mod p

```

1 // TC: O(N) precompute, O(1) query
2 // usa: C(n, k) mod p, donde p es primo
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long MOD = 998244353LL;
7 vector<long long> fact, invFact;
8
9 void buildFact(int n){
10     fact.assign(n + 1, 1);
11     invFact.assign(n + 1, 1);
12     for(int i=1 ; i<=n ; i++) fact[i] = fact[i-1] * i % MOD;
13     invFact[n] = modpow(fact[n], MOD - 2, MOD);
14     for(int i=n ; i>=1 ; i--) invFact[i-1] = invFact[i] * i % MOD;
15 }
16
17 long long nCr(int n, int k){
18     if(k < 0 || k > n) return 0;
19     return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
20 }
21
22 int main(){
23     buildFact(100);
24     cout << "C(10,3) = " << nCr(10, 3) << "\n";
25     cout << "C(100,50) mod = " << nCr(100, 50) << "\n";
26     return 0;
27 }

```

### 2.3 Lucas theorem

```

1 // TC: O(log_p n) per query
2 // usa: C(n, k) mod p con p primo y n, k grandes
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // usa: C(n, k) mod p cuando n y k son gigantes y p es primo
7 // idea: escribir n y k en base p; C(n, k) congruente Productorio C(ni,
8 //         ki) mod p
9 //         (un producto por cada digito en base p)
10 // requires fact[] and invFact[] already built for MOD (small case)
11 long long nCrLucas(long long n, long long k, long long MOD){
12     if(k == 0) return 1;
13     long long ni = n % MOD, ki = k % MOD;
14     if(ki > ni) return 0;
15     return nCrLucas(n / MOD, k / MOD, MOD) * nCr((int)ni, (int)ki) % MOD;
16 }
17
18 int main(){
19     buildFact(MOD); // precompute factorials mod MOD
20     // C(1000, 100) mod 998244353 using Lucas
21     cout << nCrLucas(1000, 100, MOD) << "\n";
22     return 0;
23 }

```

## 2.4 Stirling S2 (particiones)

```
1 // TC:  $O(N^2)$ 
2 // usa: numero de formas de particionar N elementos en K subconjuntos no
   vacios
3 // rec:  $S2[i][j] = S2[i-1][j-1] + j * S2[i-1][j]$ 
4 //     el elem i forma un bloque nuevo, o se une a uno de los j bloques
   existentes
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 const long long MOD = 998244353LL;
9 vector<vector<long long>> S2;
10
11 void buildStirlingS2(int n){
12     S2.assign(n + 1, vector<long long>(n + 1, 0));
13     S2[0][0] = 1;
14     for(int i=1;i<=n;i++){
15         for(int j=1;j<=i;j++){
16             S2[i][j] = (S2[i-1][j-1] + j * S2[i-1][j]) % MOD;
17         }
18     }
19 }
20
21 int main(){
22     buildStirlingS2(10);
23     // partitions of 5 elements into 2 nonempty subsets
24     cout << "S2(5,2) = " << S2[5][2] << "\n";
25     return 0;
26 }
```

## 2.5 Stirling S1 (permutaciones por ciclos)

```
1 // TC:  $O(N^2)$ 
2 // usa: numero de permutaciones de N elementos con exactamente K ciclos
3 // rec:  $S1[i][j] = S1[i-1][j-1] + (i-1) * S1[i-1][j]$ 
4 //     el elem i forma un ciclo de 1, o se inserta en uno de los i-1
   huecos de ciclos previos
5 // nota: version sin signo; con signo  $s(n,k)=(-1)^{(n-k)}*S1[n][k]$  es
   coeficiente de  $(x)_n$ 
6 #include <bits/stdc++.h>
7 using namespace std;
8
9 const long long MOD = 998244353LL;
10 vector<vector<long long>> S1;
11
12 void buildStirlingS1(int n){
13     S1.assign(n + 1, vector<long long>(n + 1, 0));
14     S1[0][0] = 1;
15     for(int i=1;i<=n;i++){
16         for(int j=1;j<=i;j++){
17             S1[i][j] = (S1[i-1][j-1] + (long long)(i - 1) * S1[i-1][j]) % MOD;
18         }
19     }
20 }
```

```
20 }
21
22 int main(){
23     buildStirlingS1(10);
24     // permutations of 4 elements with exactly 2 cycles -> 11
25     cout << "S1(4,2) = " << S1[4][2] << "\n";
26     return 0;
27 }
```

## 2.6 Inclusion-exclusion over subsets

```
1 // TC:  $O(2^N * \text{cost\_of\_evaluating\_subset})$ 
2 // usa: cuando se quiere contar elementos que cumplen alguna combinacion
   de N propiedades
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // f(mask) returns the number of elements satisfying exactly the
7 //     properties in mask. Returns sum over non-empty masks with sign
    $(-1)^{(k+1)}$ .
8 template<class F>
9 long long inclusionExclusion(int n, F&& f){
10     long long result = 0;
11     int total = 1 << n;
12     for(int mask=1;mask<total;mask++){
13         long long cnt = f(mask);
14         int bits = __builtin_popcount(mask);
15         if(bits % 2 == 1) result += cnt;
16         else result -= cnt;
17     }
18     return result;
19 }
20
21 int main(){
22     // count numbers in [1,100] divisible by 2, 3 or 5 (must not be
   divisible by any)
23     // generic f: returns |union over mask|, signed sum via inclusion-
   exclusion
24     auto f = [&](int mask) -> long long {
25         long long cnt = 0;
26         int primes[] = {2,3,5};
27         int bits = __builtin_popcount(mask);
28         for(int p : primes) if(mask & (1<<(p==2?0:p==3?1:2))){ /* set bit */
29             cnt++;
30         }
31         // (full example in PLAN_03 central main)
32         return cnt;
33     }
```

## 2.3 Polinomios y transformadas

### 2.1 FFT (iterativo, complex)

```
1 // TC: O(N log N)
2 // usa: multiplicacion de polinomios con coeficientes double
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 using cd = complex<double>;
7 const double PI = acos(-1);
8
9 void fft(vector<cd>& a, bool invert){
10     int n = a.size();
11     // bit-reversal permutation
12     for(int i=1,j=0;i<n;i++){
13         int bit = n >> 1;
14         for(; j&bit; bit>>=1) j ^= bit;
15         j ^= bit;
16         if(i < j) swap(a[i], a[j]);
17     }
18     for(int len=2; len<=n; len<=1){
19         double ang = 2 * PI / len * (invert ? -1 : 1);
20         cd wlen(cos(ang), sin(ang));
21         for(int i=0; i<n; i+=len){
22             cd w(1);
23             for(int j=0; j<len/2; j++){
24                 cd u = a[i+j], v = a[i+j+len/2] * w;
25                 a[i+j] = u + v;
26                 a[i+j+len/2] = u - v;
27                 w *= wlen;
28             }
29         }
30     }
31     if(invert){
32         for(cd& x : a) x /= n;
33     }
34 }
35
36 vector<long long> multiplyFFT(const vector<long long>& a, const vector<
37     long long>& b){
38     vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
39     int n = 1;
40     while(n < (int)a.size() + (int)b.size()) n <= 1;
41     fa.resize(n); fb.resize(n);
42     fft(fa, false); fft(fb, false);
43     for(int i=0;i<n;i++) fa[i] *= fb[i];
44     fft(fa, true);
45     vector<long long> res(n);
46     for(int i=0;i<n;i++) res[i] = (long long)llround(fa[i].real());
47     return res;
48 }
49
50 int main(){
51     // multiply (1 + x + 2x^2) * (3 + 4x) = 3 + 7x + 10x^2 + 8x^3
52     auto res = multiplyFFT({1,1,2}, {3,4});
```

```
52     for(long long x : res) cout << x << " ";
53     cout << "\n";
54     return 0;
55 }
```

### 2.2 NTT (mod 998244353)

```
1 // TC: O(N log N)
2 // usa: multiplicacion exacta de polinomios con coeficientes mod
3 // 998244353
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 const long long MOD = 998244353LL;
8 const long long PRIMITIVE_ROOT = 3LL;
9
10 void ntt(vector<long long>& a, bool invert){
11     int n = a.size();
12     for(int i=1,j=0;i<n;i++){
13         int bit = n >> 1;
14         for(; j&bit; bit>>=1) j ^= bit;
15         j ^= bit;
16         if(i < j) swap(a[i], a[j]);
17     }
18     for(int len=2; len<=n; len<=1){
19         long long wlen = modpow(PRIMITIVE_ROOT, (MOD - 1) / len, MOD);
20         if(invert) wlen = modpow(wlen, MOD - 2, MOD);
21         for(int i=0; i<n; i+=len){
22             long long w = 1;
23             for(int j=0; j<len/2; j++){
24                 long long u = a[i+j], v = a[i+j+len/2] * w % MOD;
25                 a[i+j] = (u + v) % MOD;
26                 a[i+j+len/2] = (u - v + MOD) % MOD;
27                 w = w * wlen % MOD;
28             }
29         }
30     }
31     if(invert){
32         long long n_inv = modpow(n, MOD - 2, MOD);
33         for(long long& x : a) x = x * n_inv % MOD;
34     }
35 }
36
37 vector<long long> multiplyNTT(const vector<long long>& a, const vector<
38     long long>& b){
39     vector<long long> fa(a.begin(), a.end()), fb(b.begin(), b.end());
40     int n = 1;
41     while(n < (int)a.size() + (int)b.size()) n <= 1;
42     fa.resize(n); fb.resize(n);
43     ntt(fa, false); ntt(fb, false);
44     for(int i=0;i<n;i++) fa[i] = fa[i] * fb[i] % MOD;
45     ntt(fa, true);
46     return fa;
```

```

47 int main(){
48     // multiply (1 + x + 2x^2) * (3 + 4x) mod 998244353
49     auto res = multiplyNTT({1,1,2}, {3,4});
50     for(long long x : res) cout << x << " ";
51     cout << "\n";
52     return 0;
53 }

```

## 2.3 Polynomial utilities (mod p)

```

1  // TC: O(N log N) per operation (using NTT)
2  // usa: requiere multiplyNTT ya definido
3  #include <bits/stdc++.h>
4  using namespace std;
5
6  using Poly = vector<long long>;
7
8  Poly polyAdd(const Poly& a, const Poly& b){
9      Poly c(max(a.size(), b.size()));
10     for(size_t i=0;i<a.size();i++) c[i] = (c[i] + a[i]) % MOD;
11     for(size_t i=0;i<b.size();i++) c[i] = (c[i] + b[i]) % MOD;
12     return c;
13 }
14
15 Poly polySub(const Poly& a, const Poly& b){
16     Poly c(max(a.size(), b.size()));
17     for(size_t i=0;i<a.size();i++) c[i] = (c[i] + a[i]) % MOD;
18     for(size_t i=0;i<b.size();i++) c[i] = (c[i] - b[i] + MOD) % MOD;
19     return c;
20 }
21
22 // inverse: 1/B(x) mod x^n
23 Poly polyInv(const Poly& b, int n){
24     Poly r(1, modpow(b[0], MOD - 2, MOD));
25     int cur = 1;
26     while(cur < n){
27         cur <= 1;
28         Poly a(min((int)b.size(), cur));
29         for(int i=0;i<(int)a.size();i++) a[i] = b[i];
30         Poly nr = multiplyNTT(a, multiplyNTT(r, r));
31         r.resize(cur);
32         for(int i=0;i<cur;i++) r[i] = (2LL * r[i] - (i < (int)nr.size() ? nr[i] : 0) + MOD) % MOD;
33     }
34     r.resize(n);
35     return r;
36 }
37
38 // derivative
39 Poly polyDeriv(const Poly& a){
40     if(a.empty()) return {};
41     Poly b((int)a.size() - 1);
42     for(size_t i=1;i<a.size();i++) b[i-1] = a[i] * i % MOD;
43     return b;
44 }

```

```

45 // integral
46 Poly polyInteg(const Poly& a){
47     Poly b(a.size() + 1);
48     // modular inverse of i+1 for each i
49     static vector<long long> inv(1, 1);
50     while((int)inv.size() <= (int)a.size()) inv.push_back(modpow(inv.size()
51         , MOD - 2, MOD));
52     for(size_t i=0;i<a.size();i++) b[i+1] = a[i] * inv[i+1] % MOD;
53     return b;
54 }
55
56 int main(){
57     Poly a = {1, 2, 3};
58     Poly b = {4, 5};
59     Poly c = polyAdd(a, b);
60     Poly d = polySub(a, b);
61     Poly deriv = polyDeriv(a); // {2, 6}
62     Poly integ = polyInteg(a); // {0, 1, 1, 1}
63     return 0;
64 }

```

## 2.4 Algebra lineal

### 2.1 Gauss-Jordan (mod p)

```

1  // TC: O(N^3) where N is the matrix dimension
2  // usa: resolver sistemas lineales, calcular determinante y rango
3  #include <bits/stdc++.h>
4  using namespace std;
5
6  const long long MOD = 998244353LL;
7
8  // returns rank; mutates a in-place to row-echelon form
9  int gaussJordan(vector<vector<long long>>& a){
10     int n = a.size(), m = a[0].size(), rank = 0;
11     int row = 0;
12     for(int col=0; col<m && row<n; col++){
13         int sel = row;
14         for(int i=row;i<n;i++) if(a[i][col]){ sel = i; break; }
15         if(!a[sel][col]) continue;
16         swap(a[sel], a[row]);
17         long long inv = modpow(a[row][col], MOD - 2, MOD);
18         for(int j=col;j<m;j++) a[row][j] = a[row][j] * inv % MOD;
19         for(int i=0;i<n;i++){
20             if(i != row && a[i][col]){
21                 long long factor = a[i][col];
22                 for(int j=col;j<m;j++){
23                     a[i][j] = (a[i][j] - factor * a[row][j] % MOD + MOD) % MOD;
24                 }
25             }
26         }
27         row++; rank++;
28     }

```

```

29     return rank;
30 }
31
32 long long determinant(vector<vector<long long>> a){
33     int n = a.size();
34     long long det = 1;
35     for(int i=0;i<n;i++){
36         int sel = i;
37         for(int j=i;j<n;j++) if(a[j][i]){ sel = j; break; }
38         if(!a[sel][i]) return 0;
39         if(sel != i){ swap(a[sel], a[i]); det = (MOD - det) % MOD; }
40         det = det * a[i][i] % MOD;
41         long long inv = modpow(a[i][i], MOD - 2, MOD);
42         for(int j=i+1;j<n;j++){
43             long long factor = a[j][i] * inv % MOD;
44             for(int k=i;k<n;k++){
45                 a[j][k] = (a[j][k] - factor * a[i][k] % MOD + MOD) % MOD;
46             }
47         }
48     }
49     return det;
50 }
51
52 // --- Resolver Ax = b usando gaussJordan sobre la matriz aumentada ---
53 // gaussJordan devuelve el RANGO r de la matriz aumentada [A|b].
54 // - Si tras la REF existe alguna fila i con todos los coeficientes
55 //   en cero pero el termino independiente aug[i][n] != 0, el
56 //   sistema es INCONSISTENTE: no hay solucion.
57 // - Si r == n (rango == #incognitas), hay SOLUCION UNICA y vive
58 //   en la ultima columna de la RREF: x[i] = aug[i][n].
59 // - Si r < n, quedan (n - r) variables libres: hay INFINITAS
60 //   soluciones. La ultima columna da una solucion particular
61 //   poniendo las libres en 0.
62 // Retorna:
63 // 0 -> sin solucion
64 // 1 -> solucion unica (escrita en 'sol')
65 // 2 -> infinitas soluciones (escribe una particular en 'sol')
66 int solveSystem(vector<vector<long long>> A, vector<long long> b,
67     vector<long long>& sol){
68     int n = (int)A.size();
69     // 1) construir la matriz aumentada [A | b] de tamaño n x (n+1)
70     vector<vector<long long>> aug(n, vector<long long>(n + 1));
71     for(int i = 0; i < n; i++){
72         for(int j = 0; j < n; j++) aug[i][j] = A[i][j];
73         aug[i][n] = b[i];
74     }
75
76     // 2) RREF + rango
77     int r = gaussJordan(aug);
78
79     // 3) checar inconsistencia: fila nula con termino independiente != 0
80     for(int i = r; i < n; i++)
81         if(aug[i][n] != 0) return 0;    // sin solucion
82
83     // 4) rango < #incognitas -> variables libres -> infinitas soluciones
84     if(r < n) return 2;

```

```

85
86 // 5) solucion unica: se lee directo de la ultima columna
87 sol.assign(n, 0);
88 for(int i = 0; i < n; i++) sol[i] = aug[i][n];
89 return 1;
90 }
91
92 int main(){
93     vector<vector<long long>> A = {{1,2,3},{4,5,6},{7,8,10}};
94     int r = gaussJordan(A);
95     cout << "rank = " << r << "\n";
96     cout << "det = " << determinant({{1,2},{3,4}}) << "\n";    // -2 mod p
97
98     // Ejemplo: 2x + y = 5 , x - y = 1 -> x = 2, y = 1
99     vector<vector<long long>> M = {{2,1},{1,-1}};
100     vector<long long> b = {5,1}, x;
101     int status = solveSystem(M, b, x);    // status = 1, x = {2, 1}
102     for(long long v : x) cout << v << " ";
103     cout << "\n";
104     return 0;
105 }

```

## 2.2 Matrix exponentiation

```

1 // TC: O(N^3 log k)
2 // usa: recurrencias lineales (Fibonacci, contar caminos de longitud k,
3 //   etc.)
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 using Matrix = vector<vector<long long>>;
8
9 Matrix matMul(const Matrix& a, const Matrix& b, long long MOD){
10     int n = a.size();
11     Matrix c(n, vector<long long>(n, 0));
12     for(int i=0;i<n;i++)
13         for(int k=0;k<n;k++) if(a[i][k])
14             for(int j=0;j<n;j++)
15                 c[i][j] = (c[i][j] + a[i][k] * b[k][j]) % MOD;
16     return c;
17 }
18
19 Matrix matPow(Matrix base, long long exp, long long MOD){
20     int n = base.size();
21     Matrix res(n, vector<long long>(n, 0));
22     for(int i=0;i<n;i++) res[i][i] = 1;
23     while(exp){
24         if(exp & 1) res = matMul(res, base, MOD);
25         base = matMul(base, base, MOD);
26         exp >>= 1;
27     }
28     return res;
29 }
30
31 int main(){

```

```

31 // Fibonacci: F(n) = F(n-1) + F(n-2)
32 Matrix T = {{1,1},{1,0}};
33 Matrix Fn = matPow(T, 10, MOD);
34 cout << "F(10) = " << Fn[0][1] << "\n"; // 55
35 return 0;
36 }

```

## 2.5 Juegos y teoria de juegos

### 2.1 Sprague-Grundy template

```

1 // TC: depends on game; usually O(states * transitions)
2 // usa: juegos impartial, nim-heap decomposition
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 unordered_map<long long, long long> sgMemo;
7
8 long long grundy(long long state){
9     if(sgMemo.count(state)) return sgMemo[state];
10    set<long long> reachable;
11    // populate 'reachable' with grundy of all positions reachable from '
    state'
12    // ... (game-specific) ...
13    long long mex = 0;
14    while(reachable.count(mex)) mex++;
15    return sgMemo[state] = mex;
16 }
17
18 int main(){
19     // Nim with 3 piles of sizes 1, 2, 3
20     int g = grundy(1) ^ grundy(2) ^ grundy(3);
21     cout << (g ? "first player wins" : "second player wins") << "\n";
22     return 0;
23 }

```

### 2.2 Small game tables (Kayles SG values)

```

1 // TC: O(1) per query (table lookup)
2 // usa: cuando el juego tiene espacio de estados pequeno y conocido
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // Kayles grundy values for n=0..71 (precomputed table)
7 vector<int> kaylesSG = {
8     0,1,2,3,1,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,
9     1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,7,4,3,2,1,4,2,6,4,5,1,1,
10    7,4,3,2,1,4,2,6
11 };
12
13 int main(){
14     cout << "SG Kayles(7) = " << kaylesSG[7] << "\n";

```

```

15     return 0;
16 }

```

## 3 Estructuras de datos

### 3.1 Ordenado y contenedores

#### 3.1 Ordered Set (pbds)

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/trie_policy.hpp>
3
4 using namespace __gnu_pbds;
5
6 typedef tree<int, null_type, less<int>, rb_tree_tag,
7 tree_order_statistics_node_update> pbds;
8 // find_by_order, order_of_key ; greater<int>, less_equal<int>
9 // insert -> logn | delete -> | find kth element -> logn
10 // find # of elements smaller than x -> logn
11 // up and lb work opposite
12 // to erase in a multiset s.erase(s.upperbound(x))

```

#### 3.2 Basis (XOR basis)

```

1 // TC: O(D^2) por insercion (D = numero de bits, aca 64), O(D^2) total
    para n inserciones
2 // usa: representar conjunto de numeros como base de Gauss sobre GF(2);
    maximo xor subconjunto, pertenencia, k-esimo xor
3 struct Basis{
4     int d=64;
5     vector<int> basis;
6     int sz=0; // current size of the basis
7     Basis(){
8         basis.resize(d);
9     }
10    void insertVector(int mask){
11        for(int i=d-1 ; i>=0 ; i--){
12            if( (mask & (1LL<<i)) == 0 ) continue;
13            if( !basis[i] ){
14                basis[i]=mask;
15                ++sz;
16                return;
17            }
18            mask ^= basis[i];
19        }
20    }
21 };
22
23 int main(){
24     // ejemplo: dado un conjunto de numeros, encontrar el maximo xor de un
    subconjunto

```

```

25 vector<int> nums = {1, 2, 3, 5};
26 Basis b;
27 for(int x : nums) b.insertVector(x);
28 int ans = 0;
29 for(int i=b.d-1 ; i>=0 ; i--) ans = max(ans, ans ^ b.basis[i]);
30 // ans = maximo xor de un subconjunto de nums
31 return 0;
32 }

```

### 3.3 Trie

```

1 // TC: O(L) por operacion (L = largo de la palabra) en tiempo, O(N * L)
  // en memoria (N palabras)
2 // usa: insertar, buscar y verificar prefijos sobre un alfabeto (aca 26
  // letras)
3 struct Node{
4     Node *links[26];
5     bool flag=false;
6     bool containsKey(char ch){
7         return links[ch-'a'] != NULL;
8     }
9     void put(char ch, Node* node){
10         links[ch-'a'] = node;
11     }
12     Node* get(char ch){
13         return links[ch-'a'];
14     }
15     void setEnd(){
16         flag=true;
17     }
18     bool isEnd(){
19         return flag;
20     }
21 };
22 class Trie{
23     private:
24     Node *root;
25     public:
26     Trie(){
27         root = new Node();
28     }
29     // tc -> O(len)
30     void insert(string word){
31         Node* node = root;
32         for(int i=0 ; i<word.size() ; i++){
33             if(!node->containsKey(word[i])){
34                 node->put(word[i], new Node());
35             }
36             // moves to reference trie
37             node = node->get(word[i]);
38         }
39         node->setEnd();
40     }
41     // tc -> O(len)
42     bool search(string word){

```

```

43     Node* node = root;
44     for(int i=0 ; i<word.size() ; i++){
45         if(!node->containsKey(word[i])){
46             return false;
47         }
48         node = node->get(word[i]);
49     }
50     return node->isEnd();
51 }
52 // tc -> O(len)
53 bool startsWith(string prefix){
54     Node* node = root;
55     for(int i=0 ; i<prefix.size() ; i++){
56         if(!node->containsKey(prefix[i])){
57             return false;
58         }
59         node = node->get(prefix[i]);
60     }
61     return true;
62 }
63 };
64 int main(){
65     Trie trie;
66     trie.insert("hello");
67     trie.insert("world");
68     cout << trie.search("hello") << endl; // 1
69     cout << trie.search("hell") << endl; // 0
70     cout << trie.startsWith("hell") << endl; // 1
71     cout << trie.startsWith("wor") << endl; // 1
72     return 0;
73 }

```

## 3.2 Union-Find

### 3.1 DSU classico

```

1 class DisjointSet{
2     vector<int> parent, size;
3     public:
4     DisjointSet(int n){
5         parent.resize(n+1);
6         size.resize(n+1);
7         for(int i=0 ; i<=n ; i++){
8             parent[i]=i;
9             size[i]=1;
10        }
11    }
12    int findUPar(int node){
13        if(node == parent[node]) return node;
14        return parent[node]=findUPar(parent[node]);
15    }
16    void UnionBySize(int u, int v){
17        int ulp_u=findUPar(u);
18        int ulp_v=findUPar(v);

```

```

19     if(ulp_u == ulp_v) return;
20     if(size[ulp_u] < size[ulp_v]){
21         parent[ulp_u]=ulp_v;
22         size[ulp_v]+=size[ulp_u];
23     }else{
24         parent[ulp_v]=ulp_u;
25         size[ulp_u]+=size[ulp_v];
26     }
27 }
28 };

```

## 3.2 DSU con rollback

```

1 // TC: O(log N) amortized per union/find; rollback is O(1) undo of last
  union
2 // usa: divide & conquer offline, Mo con updates, undoable DSU
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct DSUrb{
7     vector<int> parent, sz;
8     vector<pair<int,int>> history; // stack of (node, old_size)
9     int components;
10
11     DSUrb(int n){
12         parent.resize(n);
13         sz.assign(n, 1);
14         iota(parent.begin(), parent.end(), 0);
15         components = n;
16     }
17
18     int find(int v){
19         // no path compression to keep rollback easy
20         while(v != parent[v]) v = parent[v];
21         return v;
22     }
23
24     bool unite(int a, int b){
25         a = find(a); b = find(b);
26         if(a == b) return false;
27         if(sz[a] > sz[b]) swap(a, b);
28         history.push_back({a, sz[a]});
29         history.push_back({b, sz[b]});
30         parent[a] = b;
31         sz[b] += sz[a];
32         components--;
33         return true;
34     }
35
36     void snapshot(){
37         history.push_back({-1, -1}); // marker
38     }
39
40     void rollback(){
41         // undo until last marker (or to empty if no markers)

```

```

42         while(!history.empty() && history.back().first != -1){
43             auto [v, s0ld] = history.back(); history.pop_back();
44             auto [w, s0] = history.back(); history.pop_back();
45             parent[v] = w;
46             sz[w] = s0;
47             components++;
48         }
49         if(!history.empty()) history.pop_back(); // pop marker
50     }
51 };
52
53 int main(){
54     DSUrb d(5);
55     d.unite(0,1); d.unite(1,2); // {0,1,2} merged
56     cout << d.components << "\n"; // 3
57     d.rollback();
58     cout << d.components << "\n"; // 5 (everything undone)
59     return 0;
60 }

```

## 3.3 Arboles de segmentos

### 3.1 Segment Tree Add + Sum

```

1 // TC: O(log n) por operacion, O(n) en memoria
2 // usa: sumar un valor a todos los elementos de un rango y consultar la
  suma en un rango
3 struct SegmentTree {
4     struct Node {
5         int val = 0, lazy = 0;
6     };
7
8     int n;
9     vector<Node> st;
10
11     SegmentTree(int size) {
12         n = 1;
13         while(__builtin_popcount(n) != 1) n++;
14         st.resize(2*n);
15     }
16
17     void push(int node, int l, int r) {
18         if(st[node].lazy == 0) return;
19         st[node].val += (r-l+1) * st[node].lazy;
20         if(l != r) {
21             st[node*2].lazy += st[node].lazy;
22             st[node*2+1].lazy += st[node].lazy;
23         }
24         st[node].lazy = 0;
25     }
26
27     void add_range(int node, int l, int r, int ql, int qr, int val) {
28         push(node, l, r);
29         if(ql <= l && r <= qr) {

```

```

30     st[node].lazy += val;
31     push(node, l, r);
32     return;
33 }
34 if(r < ql || qr < l) return;
35 int mid = l + (r-l)/2;
36 add_range(node*2, l, mid, ql, qr, val);
37 add_range(node*2+1, mid+1, r, ql, qr, val);
38 st[node].val = st[node*2].val + st[node*2+1].val;
39 }
40
41 int query(int node, int l, int r, int ql, int qr) {
42     push(node, l, r);
43     if(ql <= l && r <= qr) return st[node].val;
44     if(r < ql || qr < l) return 0;
45     int mid = l + (r-l)/2;
46     return query(node*2, l, mid, ql, qr)
47     + query(node*2+1, mid+1, r, ql, qr);
48 }
49
50 void add(int l, int r, int val) {
51     add_range(1, 0, n-1, l, r, val);
52 }
53 int sum(int l, int r) {
54     return query(1, 0, n-1, l, r);
55 }
56 };
57 int main(){
58     SegmentTree st(5);
59     st.add(0, 4, 1);           // [1,1,1,1,1]
60     st.add(1, 3, 2);           // [1,3,3,3,1]
61     cout << st.sum(0, 4) << endl; // 11
62     cout << st.sum(1, 3) << endl; // 9
63     st.add(0, 4, -1);          // [0,2,2,2,0]
64     cout << st.sum(0, 4) << endl; // 6
65     return 0;
66 }

```

### 3.2 Segment Tree Lazy Assign + Sum

```

1 // TC: O(log n) por operacion, O(n) en memoria
2 // usa: asignar un valor a un rango y consultar la suma en un rango
3 struct SegmentTree {
4     struct Node {
5         int val = 0;
6         int lazy = LLONG_MAX; // valor neutro
7     };
8
9     int n;
10    vector<Node> st;
11
12    SegmentTree(int size) {
13        n = size;
14        while(__builtin_popcount(n) != 1) n++;
15        st.resize(2*n);

```

```

16    }
17
18    void push(int node, int l, int r) {
19        if(st[node].lazy == LLONG_MAX) return;
20        st[node].val = (r-l+1) * st[node].lazy;
21        if(l != r) {
22            st[node*2].lazy = st[node].lazy;
23            st[node*2+1].lazy = st[node].lazy;
24        }
25        st[node].lazy = LLONG_MAX;
26    }
27
28    void assign_range(int node, int l, int r, int ql, int qr, int val) {
29        push(node, l, r);
30        if(ql <= l && r <= qr) {
31            st[node].lazy = val;
32            push(node, l, r);
33            return;
34        }
35        if(r < ql || qr < l) return;
36        int mid = l + (r-l)/2;
37        assign_range(node*2, l, mid, ql, qr, val);
38        assign_range(node*2+1, mid+1, r, ql, qr, val);
39        st[node].val = st[node*2].val + st[node*2+1].val;
40    }
41
42    int query(int node, int l, int r, int ql, int qr) {
43        push(node, l, r);
44        if(ql <= l && r <= qr) return st[node].val;
45        if(r < ql || qr < l) return 0;
46        int mid = l + (r-l)/2;
47        return query(node*2, l, mid, ql, qr)
48        + query(node*2+1, mid+1, r, ql, qr);
49    }
50
51    void assign(int l, int r, int val) {
52        assign_range(1, 0, n-1, l, r, val);
53    }
54    int sum(int l, int r) {
55        return query(1, 0, n-1, l, r);
56    }
57 };
58 int main(){
59     SegmentTree st(5);
60     st.assign(0, 4, 1);           // [1,1,1,1,1]
61     st.assign(1, 3, 2);           // [1,2,2,2,1]
62     cout << st.sum(0, 4) << endl; // 8
63     cout << st.sum(1, 3) << endl; // 6
64     st.assign(0, 4, 5);           // [5,5,5,5,5]
65     cout << st.sum(0, 4) << endl; // 25
66     return 0;
67 }

```

### 3.3 Segment Tree Add + lower bound index

```

1 struct SegmentTree {
2     struct Node {
3         int val = 0, lazy = 0;
4     };
5
6     int n;
7     vector<Node> st;
8
9     SegmentTree(int size) {
10         n = size;
11         while(__builtin_popcount(n) != 1) n++;
12         st.resize(2*n);
13     }
14
15     void push(int node, int l, int r) {
16         if(st[node].lazy == 0) return;
17         st[node].val += st[node].lazy;
18         if(l != r) {
19             st[node*2].lazy += st[node].lazy;
20             st[node*2+1].lazy += st[node].lazy;
21         }
22         st[node].lazy = 0;
23     }
24
25     void add_range(int node, int l, int r, int ql, int qr, int val) {
26         push(node, l, r);
27         if(ql <= l && r <= qr) {
28             st[node].lazy = val;
29             push(node, l, r);
30             return;
31         }
32         if(r < ql || qr < l) return;
33         int mid = l + (r-l)/2;
34         add_range(node*2, l, mid, ql, qr, val);
35         add_range(node*2+1, mid+1, r, ql, qr, val);
36         st[node].val = max(st[node*2].val, st[node*2+1].val);
37     }
38
39     int query(int node, int l, int r, int val, int bound) {
40         push(node, l, r);
41         if(r < bound || st[node].val < val) return -1;
42         if(l == r) return l;
43         int mid = l + (r-l)/2;
44         int left = query(node*2, l, mid, val, bound);
45         if(left == -1) return query(node*2+1, mid+1, r, val, bound);
46         return left;
47     }
48
49     void add(int l, int r, int val) {
50         add_range(1, 0, n-1, l, r, val);
51     }
52
53     int lower_bound_index(int val, int bound) {
54         return query(1, 0, n-1, val, bound);
55     }
56 };

```

### 3.4 Segment Tree Lazy Add + Set + Sum

```

1 // TC: O(log n) por operacion, O(n) en memoria
2 // usa: sumar o asignar valores en rangos y consultar sumas
3 // lazy_set = LLONG_MAX indica "no hay set pendiente" (sirve para set a 0)
4 struct SegmentTree {
5     struct Node {
6         int val = 0, lazy_add = 0;
7         int lazy_set = LLONG_MAX;
8         Node(int x=0) { val = x; lazy_add = 0; lazy_set = LLONG_MAX; }
9     };
10
11     int n;
12     vector<Node> st;
13
14     SegmentTree(const vector<int>& a) {
15         n = a.size();
16         while(__builtin_popcount(n) != 1) n++;
17         st.resize(2*n);
18         for(int i=0 ; i<(int)a.size() ; i++)
19             st[n+i] = Node(a[i]);
20         for(int i=n-1 ; i>=1; i--)
21             st[i] = merge(st[2*i], st[2*i+1]);
22     }
23
24     Node merge(Node &left, Node &right) {
25         Node ans;
26         ans.val = left.val + right.val;
27         return ans;
28     }
29
30     void push(int node, int l, int r) {
31         if(st[node].lazy_set != LLONG_MAX) {
32             st[node].val = st[node].lazy_set * (r-l+1);
33             if(l != r) {
34                 st[node*2].lazy_set = st[node].lazy_set;
35                 st[node*2].lazy_add = 0;
36                 st[node*2+1].lazy_set = st[node].lazy_set;
37                 st[node*2+1].lazy_add = 0;
38             }
39             st[node].lazy_set = LLONG_MAX;
40         }
41         if(st[node].lazy_add != 0) {
42             st[node].val += st[node].lazy_add * (r-l+1);
43             if(l != r) {
44                 st[node*2].lazy_add += st[node].lazy_add;
45                 st[node*2+1].lazy_add += st[node].lazy_add;
46             }
47             st[node].lazy_add = 0;
48         }
49     }
50
51     void update(int node, int l, int r, int ql, int qr, int val,
52                int type) {
53         push(node, l, r);

```

```

54 if(ql <= l && r <= qr) {
55     if(type == 1) st[node].lazy_add += val;
56     else st[node].lazy_set = val;
57     push(node, l, r);
58     return;
59 }
60 if(r < ql || qr < l) return;
61 int mid = l + (r-l)/2;
62 update(node*2, l, mid, ql, qr, val, type);
63 update(node*2+1, mid+1, r, ql, qr, val, type);
64 st[node].val = st[node*2].val + st[node*2+1].val;
65 }
66
67 int query(int node, int l, int r, int ql, int qr) {
68     push(node, l, r);
69     if(ql <= l && r <= qr) return st[node].val;
70     if(r < ql || qr < l) return 0;
71     int mid = l + (r-l)/2;
72     return query(node*2, l, mid, ql, qr)
73     + query(node*2+1, mid+1, r, ql, qr);
74 }
75
76 // Funciones publicas
77 void add_range(int l, int r, int val) {
78     update(1, 0, n-1, l, r, val, 1);
79 }
80 void set_range(int l, int r, int val) {
81     update(1, 0, n-1, l, r, val, 2);
82 }
83 int sum(int l, int r) {
84     return query(1, 0, n-1, l, r);
85 }
86 };
87 int main(){
88     SegmentTree st({1, 2, 3, 4, 5});
89     st.add_range(0, 4, 2); // [3,4,5,6,7]
90     cout << st.sum(0, 4) << endl; // 25
91     st.set_range(1, 3, 0); // [3,0,0,0,7]
92     st.add_range(0, 4, 1); // [4,1,1,1,8]
93     cout << st.sum(0, 4) << endl; // 15
94     st.set_range(2, 4, 5); // [4,1,5,5,5]
95     cout << st.sum(2, 4) << endl; // 15
96     return 0;
97 }

```

### 3.5 MO

```

1 // TC: O((n+q) * sqrt(n)) offline. Ordena queries por bloque de l, y
2 // dentro del bloque por r alternado.
3 // usa: responder muchas queries sobre subarreglos cuando se puede
4 // agregar/quitar un elemento en O(1)
5 // (ej: contar distintos, suma en rango, frecuencia, xor, etc).
6 struct Q{
7     int l,r,idx;
8 };

```

```

7
8 int main(){
9     int n,m; cin >> n >> m;
10    vector<int> a(n);
11    for(int i=0 ; i<n ; i++) cin >> a[i];
12
13    vector<Q> q(m);
14    for(int i=0 ; i<m ; i++){
15        int l,r; cin >> l >> r;
16        q[i].l = l-1;
17        q[i].r = r-1;
18        q[i].idx = i;
19    }
20
21    int rootN = sqrtl(n);
22    sort(q.begin(), q.end(), [&](Q x, Q y){
23        int bx = x.l / rootN, by = y.l / rootN;
24        if(bx != by) return bx < by;
25        return (bx & 1) ? (x.r > y.r) : (x.r < y.r);
26    });
27
28    vector<int> freq(1000001, 0);
29    long long curr = 0;
30    vector<long long> ans(m);
31
32    auto add = [&](int pos){
33        int x = a[pos];
34        if(freq[x] == 0) curr++;
35        freq[x]++;
36    };
37    auto del = [&](int pos){
38        int x = a[pos];
39        freq[x]--;
40        if(freq[x] == 0) curr--;
41    };
42
43    int l=0, r=-1;
44    for(int i=0 ; i<m ; i++){
45        int ql=q[i].l, qr=q[i].r;
46        while(r < qr){ add(++r); }
47        while(r > qr){ del(r--); }
48        while(l < ql){ del(l++); }
49        while(l > ql){ add(--l); }
50        ans[q[i].idx] = curr; // cuenta los distintos en [ql, qr]
51    }
52
53    for(int i=0 ; i<m ; i++) cout << ans[i] << "\n";
54    return 0;
55 }

```

### 3.6 Segment Tree 2D

```

1 // TC: O(log N * log M) per query
2 // usa: matriz con queries suma en subrectangulo y updates punto
3 #include <bits/stdc++.h>

```

```

4 using namespace std;
5
6 struct SegTree2D{
7     int n, m;
8     vector<vector<int>> tree;
9     SegTree2D(int n, int m): n(n), m(m){
10         tree.assign(4*n, vector<int>(4*m, 0));
11     }
12     void updateY(int vx, int lx, int rx, int vy, int ly, int ry, int x, int
        y, int val){
13         if(ly == ry){
14             if(lx == rx) tree[vx][vy] = val;
15             else tree[vx][vy] = tree[vx*2][vy] + tree[vx*2+1][vy];
16         }else{
17             int my = (ly + ry) / 2;
18             if(y <= my) updateY(vx, lx, rx, vy*2, ly, my, x, y, val);
19             else updateY(vx, lx, rx, vy*2+1, my+1, ry, x, y, val);
20             tree[vx][vy] = tree[vx][vy*2] + tree[vx][vy*2+1];
21         }
22     }
23     void updateX(int vx, int lx, int rx, int x, int y, int val){
24         if(lx != rx){
25             int mx = (lx + rx) / 2;
26             if(x <= mx) updateX(vx*2, lx, mx, x, y, val);
27             else updateX(vx*2+1, mx+1, rx, x, y, val);
28         }
29         updateY(vx, lx, rx, 1, 0, m-1, x, y, val);
30     }
31     void update(int x, int y, int val){
32         updateX(1, 0, n-1, x, y, val);
33     }
34     int queryY(int vx, int vy, int ly, int ry, int y1, int y2){
35         if(y1 > ry || y2 < ly) return 0;
36         if(y1 <= ly && ry <= y2) return tree[vx][vy];
37         int my = (ly + ry) / 2;
38         return queryY(vx, vy*2, ly, my, y1, y2) + queryY(vx, vy*2+1, my+1, ry
            , y1, y2);
39     }
40     int queryX(int vx, int lx, int rx, int x1, int x2, int y1, int y2){
41         if(x1 > rx || x2 < lx) return 0;
42         if(x1 <= lx && rx <= x2) return queryY(vx, 1, 0, m-1, y1, y2);
43         int mx = (lx + rx) / 2;
44         return queryX(vx*2, lx, mx, x1, x2, y1, y2) + queryX(vx*2+1, mx+1, rx
            , x1, x2, y1, y2);
45     }
46     int query(int x1, int y1, int x2, int y2){
47         return queryX(1, 0, n-1, x1, x2, y1, y2);
48     }
49 };
50
51 int main(){
52     SegTree2D st(8, 8);
53     st.update(2, 3, 5);
54     st.update(5, 5, 7);
55     cout << st.query(0, 0, 7, 7) << "\n"; // 12
56     return 0;

```

```

57 }

```

### 3.7 Segment Tree persistente

```

1 // TC: O(log N) per query, O(log N) per update (creates new root)
2 // usa: queries en versiones historicas del arreglo
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct PNode{
7     long long val;
8     PNode *l, *r;
9     PNode(long long v = 0, PNode* l = nullptr, PNode* r = nullptr): val(v),
        l(l), r(r){}
10 };
11
12 PNode* pBuild(int l, int r, const vector<long long>& a){
13     if(l == r) return new PNode(a[l]);
14     int m = (l + r) / 2;
15     return new PNode(0, pBuild(l, m, a), pBuild(m+1, r, a));
16 }
17
18 PNode* pUpdate(PNode* node, int l, int r, int pos, long long val){
19     if(l == r) return new PNode(val);
20     int m = (l + r) / 2;
21     if(pos <= m) return new PNode(0, pUpdate(node->l, l, m, pos, val), node
        ->r);
22     else return new PNode(0, node->l, pUpdate(node->r, m+1, r, pos, val));
23 }
24
25 long long pQuery(PNode* node, int l, int r, int ql, int qr){
26     if(!node || ql > r || qr < l) return 0;
27     if(ql <= l && r <= qr) return node->val;
28     int m = (l + r) / 2;
29     return pQuery(node->l, l, m, ql, qr) + pQuery(node->r, m+1, r, ql, qr);
30 }
31
32 int main(){
33     vector<long long> a = {1, 2, 3, 4, 5};
34     PNode* root = pBuild(0, 4, a);
35     PNode* v2 = pUpdate(root, 0, 4, 2, 10); // set a[2] = 10
36     cout << pQuery(root, 0, 4, 0, 4) << "\n"; // 15 (v1)
37     cout << pQuery(v2, 0, 4, 0, 4) << "\n"; // 22 (v2)
38     return 0;
39 }

```

### 3.8 Segment Tree iterativo

```

1 // TC: O(log N) per operation
2 // usa: cuando quieres evitar recursion (mas rapido en practice)
3 #include <bits/stdc++.h>
4 using namespace std;

```

```

5 struct SegTreeIter{
6     int n;
7     vector<long long> t;
8     SegTreeIter(int n){
9         this->n = n;
10        t.assign(2*n, 0);
11    }
12    void build(const vector<long long>& a){
13        for(int i=0;i<(int)a.size();i++) t[n+i] = a[i];
14        for(int i=n-1;i>=1;i--) t[i] = t[2*i] + t[2*i+1];
15    }
16    void set(int p, long long val){
17        p += n;
18        t[p] = val;
19        for(p >>= 1; p >= 1; p >>= 1) t[p] = t[2*p] + t[2*p+1];
20    }
21    long long query(int l, int r){
22        long long res = 0;
23        for(l += n, r += n+1; l < r; l >>= 1, r >>= 1){
24            if(l & 1) res += t[l++];
25            if(r & 1) res += t[--r];
26        }
27        return res;
28    }
29 };
30
31 int main(){
32     SegTreeIter st(5);
33     st.build({1, 2, 3, 4, 5});
34     cout << st.query(1, 3) << "\n"; // 9
35     st.set(2, 10);
36     cout << st.query(1, 3) << "\n"; // 16
37     return 0;
38 }
39

```

## 3.4 Fenwick / BIT

### 3.1 BIT point update + prefix sum

```

1 // TC: O(log N) per operation
2 // usa: queries de suma en prefijo
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct BIT{
7     int n;
8     vector<long long> t;
9     BIT(int n): n(n), t(n+1, 0){}
10    void add(int i, long long delta){
11        for(; i <= n; i += i & -i) t[i] += delta;
12    }
13    long long sum(int i){
14        long long r = 0;
15

```

```

15        for(; i > 0; i -= i & -i) r += t[i];
16        return r;
17    }
18    long long rangeSum(int l, int r){
19        return sum(r) - sum(l - 1);
20    }
21 };
22
23 int main(){
24     BIT bit(10);
25     bit.add(3, 5);
26     bit.add(7, 2);
27     cout << bit.sum(5) << "\n"; // 5
28     cout << bit.rangeSum(1, 10) << "\n"; // 7
29     return 0;
30 }

```

### 3.2 BIT range add + point query

```

1 // TC: O(log N) per operation
2 // usa: sumar delta a [l, r] y consultar un punto
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct BITRangeAdd{
7     int n;
8     vector<long long> t;
9     BITRangeAdd(int n): n(n), t(n+1, 0){}
10    void add(int i, long long delta){
11        for(; i <= n; i += i & -i) t[i] += delta;
12    }
13    void rangeAdd(int l, int r, long long delta){
14        add(l, delta);
15        if(r + 1 <= n) add(r + 1, -delta);
16    }
17    long long pointQuery(int i){
18        long long r = 0;
19        for(; i > 0; i -= i & -i) r += t[i];
20        return r;
21    }
22 };
23
24 int main(){
25     BITRangeAdd bit(10);
26     bit.rangeAdd(2, 5, 3);
27     cout << bit.pointQuery(1) << "\n"; // 0
28     cout << bit.pointQuery(3) << "\n"; // 3
29     cout << bit.pointQuery(6) << "\n"; // 0
30     return 0;
31 }

```

### 3.3 BIT range add + range sum

```

1 // TC: O(log N) per operation
2 // usa: sumar delta a [l, r] y consultar suma de prefijo
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct BIT2{
7     int n;
8     vector<long long> t1, t2;
9     BIT2(int n): n(n), t1(n+1, 0), t2(n+1, 0){}
10    void add(vector<long long>& t, int i, long long delta){
11        for(; i <= n; i += i & -i) t[i] += delta;
12    }
13    long long sum(const vector<long long>& t, int i){
14        long long r = 0;
15        for(; i > 0; i -= i & -i) r += t[i];
16        return r;
17    }
18    void rangeAdd(int l, int r, long long delta){
19        add(t1, l, delta);
20        add(t1, r + 1, -delta);
21        add(t2, l, delta * (1 - 1));
22        add(t2, r + 1, -delta * r);
23    }
24    long long prefixSum(int i){
25        return sum(t1, i) * i - sum(t2, i);
26    }
27    long long rangeSum(int l, int r){
28        return prefixSum(r) - prefixSum(l - 1);
29    }
30 };
31
32 int main(){
33     BIT2 bit(10);
34     bit.rangeAdd(2, 5, 3);
35     cout << bit.rangeSum(1, 10) << "\n"; // 12
36     cout << bit.rangeSum(3, 5) << "\n"; // 9
37     return 0;
38 }

```

## 3.5 RMQ y sparse table

### 3.1 Sparse Table

```

1 // TC: O(N log N) build, O(1) query
2 // usa: min/max/gcd en rango estatico (sin updates)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct SparseTable{
7     int n, LOG;
8     vector<vector<int>>> st;
9     vector<int> lg;
10    SparseTable(const vector<int>& a){
11        n = a.size();

```

```

12        LOG = 32 - __builtin_clz(n);
13        st.assign(LOG, vector<int>(n));
14        st[0] = a;
15        for(int k=1; k<LOG; k++){
16            for(int i=0; i + (1<<k) <= n; i++){
17                st[k][i] = min(st[k-1][i], st[k-1][i + (1<<(k-1))]);
18            }
19        }
20        lg.assign(n + 1, 0);
21        for(int i=2; i<=n; i++) lg[i] = lg[i/2] + 1;
22    }
23    int query(int l, int r){
24        int k = lg[r - l + 1];
25        return min(st[k][l], st[k][r - (1<<k) + 1]);
26    }
27 };
28
29 int main(){
30     SparseTable st({3, 1, 4, 1, 5, 9, 2, 6});
31     cout << st.query(0, 4) << "\n"; // 1 (min in [3,1,4,1,5])
32     cout << st.query(2, 7) << "\n"; // 2
33     return 0;
34 }

```

### 3.2 Disjoint Sparse Table

```

1 // TC: O(N log N) build, O(1) query
2 // usa: queries idempotentes en arreglos estaticos (alternativa a Sparse
3 //      Table)
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 struct DisjointSparseTable{
8     int n, LOG;
9     vector<vector<int>>> st;
10    DisjointSparseTable(const vector<int>& a){
11        n = a.size();
12        LOG = (n <= 1 ? 1 : 32 - __builtin_clz(n - 1) + 1);
13        st.assign(LOG, a);
14        for(int k=1; k<LOG; ++k){
15            int half = 1 << k;
16            for(int mid = half; mid < n; mid += 2 * half){
17                for(int i = mid - 2; i >= mid - half; --i)
18                    st[k][i] = min(a[i], st[k][i + 1]);
19                int lim = min(n, mid + half);
20                for(int i = mid + 1; i < lim; ++i)
21                    st[k][i] = min(st[k][i - 1], a[i]);
22            }
23        }
24    }
25    int query(int l, int r){
26        if(l == r) return st[0][l];
27        int k = 31 - __builtin_clz(1 ^ r);
28        return min(st[k][l], st[k][r]);
29    }

```

```

29 };
30
31 int main(){
32     DisjointSparseTable dst({3, 1, 4, 1, 5, 9, 2, 6});
33     cout << dst.query(0, 4) << "\n"; // 1
34     cout << dst.query(2, 7) << "\n"; // 2
35     return 0;
36 }

```

## 3.6 Trees y DP en arbol

### 3.1 Binary Lifting / LCA

```

1 // TC: O(N log N) build, O(log N) per LCA query
2 // usa: lowest common ancestor en arbol estatico (rooted)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct LCA{
7     int n, LOG;
8     vector<vector<int>> up;
9     vector<int> depth;
10    vector<vector<int>> adj;
11    LCA(int n): n(n){
12        LOG = 32 - __builtin_clz(n + 1);
13        up.assign(LOG, vector<int>(n, -1));
14        depth.assign(n, 0);
15        adj.assign(n, {});
16    }
17    void addEdge(int u, int v){
18        adj[u].push_back(v);
19        adj[v].push_back(u);
20    }
21    void dfs(int v, int p){
22        up[0][v] = p;
23        for(int k=1; k<LOG; k++){
24            if(up[k-1][v] != -1)
25                up[k][v] = up[k-1][up[k-1][v]];
26        }
27        for(int u : adj[v]){
28            if(u == p) continue;
29            depth[u] = depth[v] + 1;
30            dfs(u, v);
31        }
32    }
33    void build(int root){
34        dfs(root, -1);
35    }
36    int lca(int a, int b){
37        if(depth[a] < depth[b]) swap(a, b);
38        int diff = depth[a] - depth[b];
39        for(int k=0; k<LOG; k++) if(diff & (1<<k)) a = up[k][a];
40        if(a == b) return a;
41        for(int k=LOG-1; k>=0; k--){

```

```

42            if(up[k][a] != up[k][b]){
43                a = up[k][a];
44                b = up[k][b];
45            }
46        }
47        return up[0][a];
48    }
49 };
50
51 int main(){
52     LCA lca(5);
53     lca.addEdge(0,1); lca.addEdge(1,2); lca.addEdge(1,3); lca.addEdge(3,4);
54     lca.build(0);
55     cout << "LCA(2,4) = " << lca.lca(2,4) << "\n"; // 1
56     return 0;
57 }

```

### 3.2 Heavy-Light Decomposition (HLD)

```

1 // TC: O(N) build, O(log^2 N) per path query (with segtree/BIT)
2 // usa: queries en camino entre u y v
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct HLD{
7     int n, curPos;
8     vector<int> parent, depth, heavy, head, pos;
9     vector<vector<int>> adj;
10    HLD(int n): n(n){
11        parent.assign(n, -1);
12        depth.assign(n, 0);
13        heavy.assign(n, -1);
14        head.assign(n, 0);
15        pos.assign(n, 0);
16        adj.assign(n, {});
17        curPos = 0;
18    }
19    void addEdge(int u, int v){
20        adj[u].push_back(v);
21        adj[v].push_back(u);
22    }
23    int dfs(int v){
24        int size = 1, maxSub = 0;
25        for(int u : adj[v]){
26            if(u == parent[v]) continue;
27            parent[u] = v;
28            depth[u] = depth[v] + 1;
29            int sub = dfs(u);
30            if(sub > maxSub){ maxSub = sub; heavy[v] = u; }
31            size += sub;
32        }
33        return size;
34    }
35    void decompose(int v, int h){
36        head[v] = h;

```

```

37 pos[v] = curPos++;
38 if(heavy[v] != -1) decompose(heavy[v], h);
39 for(int u : adj[v]){
40     if(u == parent[v] || u == heavy[v]) continue;
41     decompose(u, u);
42 }
43 }
44 void build(int root = 0){
45     dfs(root);
46     decompose(root, root);
47 }
48 // returns the contiguous segments covering the path u->v in base array
49 // order
50 vector<pair<int,int>> pathSegments(int u, int v){
51     vector<pair<int,int>> segs;
52     while(head[u] != head[v]){
53         if(depth[head[u]] < depth[head[v]]) swap(u, v);
54         segs.push_back({pos[head[u]], pos[u]});
55         u = parent[head[u]];
56     }
57     if(depth[u] > depth[v]) swap(u, v);
58     segs.push_back({pos[u], pos[v]});
59     return segs;
60 }
61 };
62 int main(){
63     HLD hld(5);
64     hld.addEdge(0,1); hld.addEdge(1,2); hld.addEdge(1,3); hld.addEdge(3,4);
65     hld.build(0);
66     // path segments from 2 to 4 -> [(2,2), (1,3)] in pos[] order
67     auto segs = hld.pathSegments(2, 4);
68     for(auto [l,r] : segs) cout << "[" << l << ", " << r << "]" << " ";
69     cout << "\n";
70     return 0;
71 }

```

### 3.3 Centroid Decomposition

```

1 // TC: O(N log N) build; queries depend on problem
2 // usa: problemas tipo "para cada nodo, encontrar el mas cercano que
3 // cumple X"
4 #include <bits/stdc++.h>
5 using namespace std;
6 struct CentroidDecomp{
7     int n;
8     vector<vector<int>> adj;
9     vector<int> subSize;
10    vector<bool> removed;
11    vector<int> centroidParent;
12    CentroidDecomp(int n): n(n){
13        adj.assign(n, {});
14        subSize.assign(n, 0);
15        removed.assign(n, false);

```

```

16    centroidParent.assign(n, -1);
17 }
18 void addEdge(int u, int v){
19     adj[u].push_back(v);
20     adj[v].push_back(u);
21 }
22 int dfsSize(int v, int p){
23     subSize[v] = 1;
24     for(int u : adj[v]) if(u != p && !removed[u]) subSize[v] += dfsSize(u, v);
25     return subSize[v];
26 }
27 int dfsCentroid(int v, int p, int total){
28     for(int u : adj[v]){
29         if(u != p && !removed[u] && subSize[u] > total / 2){
30             return dfsCentroid(u, v, total);
31         }
32     }
33     return v;
34 }
35 void build(int v = 0, int p = -1){
36     int total = dfsSize(v, -1);
37     int c = dfsCentroid(v, -1, total);
38     centroidParent[c] = p;
39     removed[c] = true;
40     for(int u : adj[c]){
41         if(!removed[u]) build(u, c);
42     }
43 }
44 };
45
46 int main(){
47     CentroidDecomp cd(5);
48     cd.addEdge(0,1); cd.addEdge(1,2); cd.addEdge(2,3); cd.addEdge(3,4);
49     cd.build();
50     cout << "centroid parent of node 0: " << cd.centroidParent[0] << "\n";
51     return 0;
52 }

```

### 3.4 Euler Tour + range queries

```

1 // TC: O(N) build, O(log N) per query (with BIT/segtree)
2 // usa: queries en subarbol (suma de valores en el subtree de v)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct EulerTour{
7     int n, timer;
8     vector<int> tin, tout;
9     vector<vector<int>> adj;
10    EulerTour(int n): n(n){
11        tin.assign(n, 0);
12        tout.assign(n, 0);
13        adj.assign(n, {});
14        timer = 0;

```

```

15 }
16 void addEdge(int u, int v){
17     adj[u].push_back(v);
18     adj[v].push_back(u);
19 }
20 void dfs(int v, int p){
21     tin[v] = timer++;
22     for(int u : adj[v]) if(u != p) dfs(u, v);
23     tout[v] = timer - 1;
24 }
25 void build(int root = 0){ dfs(root, -1); }
26 // subtree of v corresponds to range [tin[v], tout[v]] in base array
27 };
28
29 int main(){
30     EulerTour et(5);
31     et.addEdge(0,1); et.addEdge(1,2); et.addEdge(1,3); et.addEdge(3,4);
32     et.build(0);
33     // subtree of node 1 maps to range [tin[1], tout[1]]
34     cout << "tin[1]=" << et.tin[1] << " tout[1]=" << et.tout[1] << "\n";
35     return 0;
36 }

```

## 3.7 BST balanceado

### 3.1 Treap implicito

```

1 // TC: O(log N) per operation (amortized)
2 // usa: secuencia con insert/delete por posicion, split/merge
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct Treap{
7     struct Node{
8         int val, pr, sz;
9         Node *l, *r;
10         Node(int v): val(v), pr(rand()), sz(1), l(nullptr), r(nullptr){}
11     };
12     Node* root = nullptr;
13     int sz(Node* t){ return t ? t->sz : 0; }
14     void upd(Node* t){ if(t) t->sz = 1 + sz(t->l) + sz(t->r); }
15     // split by position: [0, k) and [k, n)
16     void split(Node* t, int k, Node*& l, Node*& r){
17         if(!t){ l = r = nullptr; return; }
18         if(sz(t->l) >= k){
19             split(t->l, k, l, t->l);
20             r = t;
21             upd(r);
22         }else{
23             split(t->r, k - sz(t->l) - 1, t->r, r);
24             l = t;
25             upd(l);
26         }
27     }

```

```

28 // merge two treaps where all in l come before all in r
29 Node* merge(Node* l, Node* r){
30     if(!l || !r) return l ? l : r;
31     if(l->pr > r->pr){
32         l->r = merge(l->r, r);
33         upd(l);
34         return l;
35     }else{
36         r->l = merge(l, r->l);
37         upd(r);
38         return r;
39     }
40 }
41 // insert value at position pos
42 void insert(int pos, int val){
43     Node *l, *r;
44     split(root, pos, l, r);
45     root = merge(merge(l, new Node(val)), r);
46 }
47 // erase element at position pos
48 void erase(int pos){
49     Node *l, *m, *r;
50     split(root, pos, l, m);
51     split(m, 1, m, r);
52     delete m;
53     root = merge(l, r);
54 }
55 };
56
57 int main(){
58     Treap t;
59     t.insert(0, 10); t.insert(1, 20); t.insert(2, 30);
60     // erase position 1
61     t.erase(1);
62     return 0;
63 }

```

## 3.8 Otras

### 3.1 Mo con updates

```

1 // TC: O(N^{5/3}) per query
2 // usa: queries offline con updates mezclados
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct MoWithUpdates{
7     struct Query{
8         int l, r, t, idx;
9     };
10    struct Update{
11        int pos, val;
12    };
13    int n, q;

```

```

14 vector<Query> queries;
15 vector<Update> updates;
16 vector<int> a;
17 long long curAns = 0;
18 vector<long long> ans;
19
20 void addPos(int x){
21     // update curAns given that position x is added to current range
22 }
23 void removePos(int x){
24     // update curAns given that position x is removed from current range
25 }
26 void applyUpdate(const Update& u){
27     // apply update; if u.pos is in [curL, curR], update state
28 }
29 void revertUpdate(const Update& u){
30     // revert update; mirror of applyUpdate
31 }
32 void solve(){
33     int blockSize = pow(n, 2.0/3.0);
34     sort(queries.begin(), queries.end(), [&](const Query& A, const Query&
35         B){
36         int bA = A.l / blockSize, bB = B.l / blockSize;
37         if(bA != bB) return bA < bB;
38         int bAr = A.r / blockSize, bBr = B.r / blockSize;
39         if(bAr != bBr) return bAr < bBr;
40         return A.t < B.t;
41     });
42     ans.assign(queries.size(), 0);
43     int curL = 0, curR = -1, curT = -1;
44     for(const auto& qry : queries){
45         while(curL > qry.l) addPos(--curL);
46         while(curR < qry.r) addPos(++curR);
47         while(curL < qry.l) removePos(curL++);
48         while(curR > qry.r) removePos(curR--);
49         while(curT < qry.t) applyUpdate(updates[++curT]);
50         while(curT > qry.t) revertUpdate(updates[curT--]);
51         ans[qry.idx] = curAns;
52     }
53 };
54
55 int main(){
56     // skeleton: addPos/removePos/applyUpdate/revertUpdate must be filled
57     // per problem; see standard Mo's algorithm tutorial.
58     MoWithUpdates mwu;
59     // mwu.addPos, mwu.removePos, mwu.applyUpdate, mwu.revertUpdate need
60     impl
61     // mwu.solve();
62     return 0;
63 }

```

### 3.2 sqrt-decomposition generico

```

1 // TC: O(sqrt(N)) per operation
2 // usa: cuando quieras una EDD simple y lazy
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 template<class T>
7 struct SqrtDecomp{
8     int n, B;
9     vector<T> a, bucket;
10     SqrtDecomp(const vector<T>& v): a(v){
11         n = v.size();
12         B = (int)sqrt(n) + 1;
13         bucket.assign(B, 0);
14         for(int i=0;i<n;i++) bucket[i / B] += v[i];
15     }
16     void pointUpdate(int p, T val){
17         bucket[p / B] += val - a[p];
18         a[p] = val;
19     }
20     // range sum query
21     T rangeSum(int l, int r){
22         T res = 0;
23         int bl = l / B, br = r / B;
24         if(bl == br){
25             for(int i=l;i<=r;i++) res += a[i];
26         }else{
27             for(int i=l;i<(bl+1)*B;i++) res += a[i];
28             for(int i=bl+1;i<br;i++) res += bucket[i];
29             for(int i=br*B;i<=r;i++) res += a[i];
30         }
31         return res;
32     }
33 };
34
35 int main(){
36     SqrtDecomp<int> sd({1, 2, 3, 4, 5, 6, 7, 8, 9, 10});
37     cout << sd.rangeSum(2, 7) << "\n"; // 3+4+5+6+7+8 = 33
38     sd.pointUpdate(4, 100);
39     cout << sd.rangeSum(2, 7) << "\n"; // 128
40     return 0;
41 }

```

## 4 Strings

### 4.1 Hashing

#### 4.1 Rolling Hash (doble modulo)

```

1 // TC: precompute O(N), getHash O(1) (N = |s|)
2 // usar: comparar substrings, palindromos, longest common substring,
3 //         contar substrings distintos, hashing de paths en trees, etc.
4 // cuidado: doble hash reduce colisiones a casi cero pero no las elimina
5 struct pair_hash {

```

```

6  size_t operator()(const pair<int,int>& p) const {
7      return ((uint64_t)p.first << 32) ^ (uint64_t)p.second;
8  }
9  };
10 struct H{
11     vector<pair<int,int>> h,p;
12     const int MOD1=998244353;
13     const int MOD2=1e9+7;
14     const int A=31;
15     string s;
16     H(string &a){
17         s=a;
18         int n=s.size();
19         h.resize(n+1); p.resize(n+1,{1,1});
20         for(int i=1 ; i<=n ; i++){
21             h[i].F=((h[i-1].F*A)%MOD1 + s[i-1])%MOD1;
22             h[i].S=((h[i-1].S*A)%MOD2 + s[i-1])%MOD2;
23             p[i].F=(p[i-1].F*A)%MOD1;
24             p[i].S=(p[i-1].S*A)%MOD2;
25         }
26     }
27     pair<int,int> getHash(int l, int r){
28         pair<int,int> ans;
29         ans.F=(h[r].F-(h[l-1].F*p[r-l+1].F)%MOD1)%MOD1;
30         ans.S=(h[r].S-(h[l-1].S*p[r-l+1].S)%MOD2)%MOD2;
31         ans.F=(ans.F+MOD1)%MOD1;
32         ans.S=(ans.S+MOD2)%MOD2;
33         return ans;
34     }
35 };
36 // unordered_set<pair<int,int>, pair_hash> st;
37
38 int main(){
39     string s = "banana";
40     H h(s);
41     // comparar substrings en O(1) por hash (1-indexed, inclusivo)
42     auto a = h.getHash(1, 3); // "ban"
43     auto b = h.getHash(4, 6); // "ana"
44     cout << (a == b ? "iguales" : "distintos") << "\n";
45     return 0;
46 }

```

## 4.2 Patrones

### 4.1 KMP + LPS

```

1 // TC: O(N + M) (N = |s|, M = |p|)
2 vector<int> create_lps(string &needle){
3     int n=(int)needle.size();
4     vector<int> lps(n); // longest prefix suffix
5     int i=1 , prevLPS=0;
6     while(i < n){
7         if(needle[i] == needle[prevLPS]){
8             lps[i]=prevLPS+1;

```

```

9         i++; prevLPS++;
10    }else if(prevLPS == 0){
11        lps[i]=0; i++;
12    }else{
13        prevLPS=lps[prevLPS-1];
14    }
15 }
16 return lps;
17 }
18
19 void solve(){
20     string s,p; cin >> s >> p;
21
22     int sz_text=(int)s.size() , sz_pattern=(int)p.size();
23     vector<int> lps=create_lps(p);
24     int cnt=0;
25     int i=0; // ptr text
26     int j=0; // ptr pattern
27     while(i<sz_text){
28         if(s[i] == p[j]){
29             i++; j++;
30         }else if(j == 0) i++;
31         else j=lps[j-1];
32         if(j == sz_pattern){
33             cnt++;
34             j=lps[j-1];
35         }
36     }
37     cout<< cnt << "\n";
38 }

```

### 4.2 Z-algorithm

```

1 // TC: O(N) build
2 // usa: encontrar todas las ocurrencias de un patron en un texto
   concatenado
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> zFunction(const string& s){
7     int n = s.size();
8     vector<int> z(n, 0);
9     int l = 0, r = 0;
10    for(int i=1;i<n;i++){
11        if(i <= r) z[i] = min(r - i + 1, z[i - l]);
12        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
13        if(i + z[i] - 1 > r){ l = i; r = i + z[i] - 1; }
14    }
15    return z;
16 }
17
18 int main(){
19     string s = "abacaba";
20     int n = s.size();
21     auto z = zFunction(s);

```

```

22 for(int x : z) cout << x << " "; // 7 0 3 0 1 0 1
23 cout << "\n";
24
25 // (1) LCP entre s y cualquier sufijo s[i..]: max(z[i+1..n-1])
26 int best = 0;
27 for(int i = 1; i < n; i++) best = max(best, z[i]);
28 cout << "max LCP(s, s[i..]) = " << best << "\n"; // 3 ("aba")
29
30 // (2) ocurrencias de cada prefijo de largo k como substring (diff
31 array)
32 vector<int> prefCnt(n + 2, 0);
33 for(int i = 1; i < n; i++){ prefCnt[i]++; prefCnt[z[i] + 1]--; }
34 for(int k = 1; k <= n; k++) prefCnt[k] += prefCnt[k - 1];
35 for(int k = 1; k <= n; k++)
36     cout << "prefijo largo " << k << " aparece " << prefCnt[k] << " veces
37     \n";
38
39 // (3) palindromos impares: z sobre s + "#" + reverse(s)
40 // palindromo centrado en c tiene largo zp[n+1+(n-1-2*c)]
41 string rs(s.rbegin(), s.rend());
42 auto zp = zFunction(s + "#" + rs);
43 cout << "palindromos impares por centro:\n";
44 for(int c = 0; c < n; c++){
45     int idx = n + 1 + (n - 1 - 2*c);
46     if(idx >= 0 && idx < (int)zp.size())
47         cout << " centro " << c << " -> largo " << zp[idx] << "\n";
48 }
49
50 // (4) pattern matching: z de (patron + '#' + texto)
51 string p = "aba", t = "abacabacaba";
52 auto z2 = zFunction(p + "#" + t);
53 for(int i = 0; i < (int)z2.size(); i++)
54     if(z2[i] == (int)p.size()) cout << "match at " << i - p.size() - 1 <<
55     "\n";
56 return 0;
57 }

```

### 4.3 Rabin-Karp (single hash)

```

1 // TC: O(N + M) expected
2 // usa: encontrar patron con hash rodante; mejor con doble hash
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long BASE = 91138233LL;
7 const long long MOD = 1000000007LL;
8
9 // returns all starting positions of pattern in text
10 vector<int> rabinKarp(const string& text, const string& pattern){
11     int n = text.size(), m = pattern.size();
12     if(m > n) return {};
13     long long h = 1;
14     for(int i=0;i<m-1;i++) h = h * BASE % MOD;
15     long long pHash = 0, tHash = 0;
16     for(int i=0;i<m;i++){

```

```

17         pHash = (pHash * BASE + pattern[i]) % MOD;
18         tHash = (tHash * BASE + text[i]) % MOD;
19     }
20     vector<int> occ;
21     for(int i=0;i<=n-m;i++){
22         if(pHash == tHash){
23             bool match = true;
24             for(int j=0;j<m;j++) if(text[i+j] != pattern[j]){ match = false;
25                 break; }
26             if(match) occ.push_back(i);
27         }
28         if(i < n - m){
29             tHash = ((tHash - text[i] * h % MOD + MOD) * BASE + text[i+m]) %
30             MOD;
31         }
32     }
33     return occ;
34 }
35
36 int main(){
37     auto occ = rabinKarp("hello world", "world");
38     for(int pos : occ) cout << "found at " << pos << "\n";
39     return 0;
40 }

```

### 4.4 Aho-Corasick (basic)

```

1 // TC: O(sum |patterns| + |text|)
2 // usa: encontrar todos los patterns en un texto
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct AhoCorasick{
7     struct Node{
8         int next[26];
9         int link;
10         vector<int> out; // indices of patterns ending here
11         Node(){
12             fill(begin(next), end(next), -1);
13             link = -1;
14         }
15     };
16     vector<Node> t;
17     AhoCorasick(){ t.push_back(Node()); }
18     void addString(const string& s, int id){
19         int v = 0;
20         for(char c : s){
21             int k = c - 'a';
22             if(t[v].next[k] == -1){
23                 t[v].next[k] = (int)t.size();
24                 t.push_back(Node());
25             }
26             v = t[v].next[k];
27         }
28         t[v].out.push_back(id);

```

```

29 }
30 void build(){
31     queue<int> q;
32     t[0].link = 0;
33     for(int k=0;k<26;k++){
34         if(t[0].next[k] != -1){
35             t[t[0].next[k]].link = 0;
36             q.push(t[0].next[k]);
37         }else{
38             t[0].next[k] = 0;
39         }
40     }
41     while(!q.empty()){
42         int v = q.front(); q.pop();
43         for(int k=0;k<26;k++){
44             int u = t[v].next[k];
45             if(u != -1){
46                 t[u].link = t[t[v].link].next[k];
47                 // propagate outputs from link
48                 for(int pid : t[t[u].link].out) t[u].out.push_back(pid);
49                 q.push(u);
50             }else{
51                 t[v].next[k] = t[t[v].link].next[k];
52             }
53         }
54     }
55 }
56 // returns count of occurrences per pattern id
57 vector<int> countOccurrences(const string& text, int numPatterns){
58     vector<int> cnt(numPatterns, 0);
59     int v = 0;
60     for(char c : text){
61         v = t[v].next[c - 'a'];
62         for(int pid : t[v].out) cnt[pid]++;
63     }
64     return cnt;
65 }
66 };
67
68 int main(){
69     AhoCorasick ac;
70     ac.addString("he", 0);
71     ac.addString("she", 1);
72     ac.addString("his", 2);
73     ac.build();
74     auto cnt = ac.countOccurrences("ushers", 3);
75     for(int c : cnt) cout << c << " ";
76     cout << "\n";
77     return 0;
78 }

```

#### 4.5 Aho-Corasick DP for forbidden patterns

```

1 // TC: O(L * states * alphabet) per DP step
2 // usa: contar strings de longitud L evitando ciertos patterns

```

```

3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long AC_MOD = 998244353LL;
7
8 // assumes AhoCorasick is built; transition automaton is already computed
9
10 long long countAvoiding(const AhoCorasick& ac, int L){
11     int S = (int)ac.t.size();
12     vector<long long> dp(S, 0), ndp(S, 0);
13     if(ac.t[0].out.empty()) dp[0] = 1;
14     for(int i=0;i<L;i++){
15         fill(ndp.begin(), ndp.end(), 0);
16         for(int v=0;v<S;v++){
17             if(dp[v] == 0) continue;
18             for(int k=0;k<26;k++){
19                 int u = ac.t[v].next[k];
20                 if(ac.t[u].out.empty()){
21                     ndp[u] = (ndp[u] + dp[v]) % AC_MOD;
22                 }
23             }
24         }
25         dp = ndp;
26     }
27     long long total = 0;
28     for(long long x : dp) total = (total + x) % AC_MOD;
29     return total;
30 }
31
32 int main(){
33     // count strings of length L over [a-z] avoiding all patterns in ac
34     AhoCorasick ac;
35     ac.addString("ab", 0);
36     ac.build();
37     cout << countAvoiding(ac, 3) << "\n"; // 26^3 - count with "ab"
38     return 0;
39 }

```

### 4.3 Estructuras avanzadas

#### 4.1 Suffix Array

```

1 // TC: O(N log N) build, O(N) memoria
2 // usa: comparar substrings en O(1) (con LCP), contar substrings
3 //     distintos,
4 //     longest repeated substring, longest common substring entre 2
5 //     strings,
6 //     pattern matching O(|p| log N) con binary search sobre SA,
7 //     encontrar el k-esimo substring lex, concatenar strings y ordenar
8 //     sufijos
9 // sa.p[i] = posicion en s del i-esimo sufijo en orden lexicografico
10 // sa.c[i] = clase de equivalencia (rank) del sufijo que empieza en i
11 struct SuffArr{
12     vector<int> p,c;

```

```

10 int n;
11 string s;
12 SuffArr(string &a){
13     s=a;
14     n=s.size();
15     p.resize(n) ; c.resize(n);
16     build();
17 }
18 void count_sort(vector<int> &p, vector<int> &c){
19     int n=p.size();
20     vector<int> cnt(n);
21     for(auto x : c) cnt[x]++;
22     vector<int> p_new(n) , pos(n);
23     pos[0]=0;
24     for(int i=1 ; i<n ; i++){
25         pos[i]=pos[i-1]+cnt[i-1];
26     }
27     for(auto x : p){
28         int i=c[x];
29         p_new[pos[i]]=x;
30         pos[i]++;
31     }
32     p=p_new;
33 }
34 void build(){
35     {
36         vector<pair<int,int>> tmp(n);
37         for(int i=0 ; i<n ; i++){
38             tmp[i]={s[i],i};
39         }
40         sort(all(tmp));
41         for(int i=0 ; i<n ; i++){
42             p[i]=tmp[i].S;
43         }
44         c[p[0]]=0;
45         for(int i=1 ; i<n ; i++){
46             if(tmp[i].F == tmp[i-1].F){
47                 c[p[i]]=c[p[i-1]];
48             }else{
49                 c[p[i]]=c[p[i-1]]+1;
50             }
51         }
52     }
53     int k=0;
54     while((1<<k) < n){
55         // k -> k+1
56         for(int i=0; i<n ; i++){
57             p[i]=(p[i]-(1<<k) + n)%n;
58         }
59
60         count_sort(p,c);
61
62         vector<int> c_new(n);
63         c_new[p[0]]=0;
64         for(int i=1 ; i<n ; i++){
65             pair<int,int> prev={c[p[i-1]],c[(p[i-1]+(1<<k))%n]};

```

```

66             pair<int,int> now={c[p[i]],c[(p[i]+(1<<k))%n]};
67             if(now == prev){
68                 c_new[p[i]]=c_new[p[i-1]];
69             }else{
70                 c_new[p[i]]=c_new[p[i-1]]+1;
71             }
72         }
73         c=c_new;
74         k++;
75     }
76 }
77 };
78
79 int main(){
80     string s = "banana";
81     SuffArr sa(s);
82
83     // (1) imprimir SA y sufijos en orden lex
84     cout << "SA: ";
85     for(int i = 0; i < sa.n; i++) cout << sa.p[i] << " "; // 5 3 1 0 4 2
86     cout << "\nsufijos ordenados:\n";
87     for(int i = 0; i < sa.n; i++)
88         cout << " " << sa.s.substr(sa.p[i]) << "\n";
89
90     // (2) contar ocurrencias de un patron en O(|p| log N) con binary
91     search
92     string p = "ana";
93     auto cmpLess = [&](int suf){
94         return sa.s.compare(suf, (int)p.size(), p) < 0;
95     };
96     int lo = 0, hi = sa.n;
97     while(lo < hi){ int m = (lo+hi)/2; if(cmpLess(sa.p[m])) lo = m+1; else
98         hi = m; }
99     int first = lo;
100     // upper bound: lower_bound(p + '{') (cualquier char > los del alfabeto
101     )
102     string pNext = p + "{";
103     auto cmpLess2 = [&](int suf){
104         return sa.s.compare(suf, (int)pNext.size(), pNext) < 0;
105     };
106     lo = first; hi = sa.n;
107     while(lo < hi){ int m = (lo+hi)/2; if(cmpLess2(sa.p[m])) lo = m+1; else
108         hi = m; }
109     int last = lo;
110     cout << "ocurrencias de \"" << p << "\": " << (last - first) << "\n";
111     // 2
112
113     // (3) longest repeated substring: max(lcp[i]) sobre todo el SA
114     // (combinado con el LCP de Kasai - ver archivo lcp.tex)
115     return 0;
116 }

```

## 4.2 LCP Array (Kasai)

```

1 // TC: O(N)
2 // usa: longest common prefix entre sufijos adyacentes (necesita SA
   precomputado)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> buildLCP(const string& s, const vector<int>& sa){
7     int n = s.size();
8     vector<int> rank_(n);
9     for(int i=0;i<n;i++) rank_[sa[i]] = i;
10    vector<int> lcp(max(0, n - 1), 0);
11    int h = 0;
12    for(int i=0;i<n;i++){
13        if(rank_[i] > 0){
14            int j = sa[rank_[i] - 1];
15            while(i + h < n && j + h < n && s[i+h] == s[j+h]) h++;
16            lcp[rank_[i] - 1] = h;
17            if(h > 0) h--;
18        }
19    }
20    return lcp;
21 }
22
23 int main(){
24     string s = "banana";
25     vector<int> sa = {5,3,1,0,4,2}; // assume precomputed SA
26     auto lcp = buildLCP(s, sa);
27     for(int x : lcp) cout << x << " ";
28     cout << "\n";
29     return 0;
30 }

```

### 4.3 Suffix Automaton

```

1 // TC: O(N) build
2 // usa: contar substrings distintas, longest common substring, etc.
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct SAM{
7     struct State{
8         int next[26];
9         int link, len;
10        long long occ; // number of end positions (for counting)
11        State(){
12            fill(begin(next), end(next), -1);
13            link = -1;
14            len = 0;
15            occ = 0;
16        }
17    };
18    vector<State> st;
19    int last;
20    SAM(int maxLen){
21        st.reserve(2 * maxLen);

```

```

22        st.push_back(State());
23        st[0].link = -1;
24        last = 0;
25    }
26    void extend(char c){
27        int k = c - 'a';
28        int cur = (int)st.size();
29        st.push_back(State());
30        st[cur].len = st[last].len + 1;
31        st[cur].occ = 1;
32        int p = last;
33        while(p != -1 && st[p].next[k] == -1){
34            st[p].next[k] = cur;
35            p = st[p].link;
36        }
37        if(p == -1){
38            st[cur].link = 0;
39        }else{
40            int q = st[p].next[k];
41            if(st[p].len + 1 == st[q].len){
42                st[cur].link = q;
43            }else{
44                int clone = (int)st.size();
45                st.push_back(st[q]);
46                st[clone].len = st[p].len + 1;
47                st[clone].occ = 0; // clone doesn't count as new end
48                while(p != -1 && st[p].next[k] == q){
49                    st[p].next[k] = clone;
50                    p = st[p].link;
51                }
52                st[q].link = st[cur].link = clone;
53            }
54        }
55        last = cur;
56    }
57    void build(const string& s){
58        for(char c : s) extend(c);
59        // propagate occ counts in reverse order of len
60        vector<int> order(st.size());
61        iota(order.begin(), order.end(), 0);
62        sort(order.begin(), order.end(), [&](int a, int b){ return st[a].len
> st[b].len; });
63        for(int v : order){
64            if(st[v].link != -1) st[st[v].link].occ += st[v].occ;
65        }
66    }
67    // returns number of distinct substrings
68    long long countDistinct(){
69        long long ans = 0;
70        for(size_t i=1;i<st.size();i++){
71            ans += st[i].len - st[st[i].link].len;
72        }
73        return ans;
74    }
75    // longest common substring of s with the SAM's string
76    int longestCommonSubstring(const string& s){

```

```

77     int v = 0, l = 0, best = 0;
78     for(char c : s){
79         int k = c - 'a';
80         while(v != 0 && st[v].next[k] == -1){
81             v = st[v].link;
82             l = st[v].len;
83         }
84         if(st[v].next[k] != -1){
85             v = st[v].next[k];
86             l++;
87         }
88         best = max(best, l);
89     }
90     return best;
91 }
92 };
93
94 int main(){
95     SAM sam(100);
96     sam.build("ababa");
97     cout << "distinct substrings: " << sam.countDistinct() << "\n";
98     cout << "LCS with 'baba': " << sam.longestCommonSubstring("baba") << "\n";
99     return 0;
100 }

```

## 4.4 Palindromos

### 4.1 Manacher

```

1 // TC: O(N) para construir odd/even; cada query O(1) con los helpers
2 // usa: longest palindromic substring, contar palindromos, queries O(1) "
  es s[l..r] palindromo?"
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // Convencion de ESTE codigo (distinta a la "clasica de radio"):
7 //   odd[i] = k (k >= 1): el palindromo IMPAR mas largo centrado en i
8 //           es s[i - k + 1 .. i + k - 1] (longitud 2k - 1).
9 //           -> cuenta ademas cuantos palindromos impares hay centrados
10 //              en i:
11 //                 k palindromos (los de longitud 1, 3, 5, ..., 2k-1).
12 //   even[i] = k (k >= 0): el palindromo PAR mas largo "centrado entre i
  -1 e i"
13 //           es s[i - k .. i + k - 1] (longitud 2k). Si k == 0 no hay
  palindromo par ahi.
14 //           -> cuenta cuantos palindromos pares hay "centrados en i":
  k.
15 //   Equivalencias utiles:
16 //       palindromo impar centrado en i de longitud L impar -> odd[i] >=
  (L+1)/2
17 //       palindromo par "centrado entre i-1 e i" de long 2k -> even[i]
  >= k
18 pair<vector<int>, vector<int>> manacher(const string& s){

```

```

18     int n = (int)s.size();
19     vector<int> odd(n, 0);
20     int l = 0, r = -1;
21     for(int i = 0; i < n; i++){
22         int k = (i > r) ? 1 : min(odd[l + r - i], r - i + 1);
23         while(i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
24         odd[i] = k;
25         if(i + k - 1 > r){ l = i - k + 1; r = i + k - 1; }
26     }
27     vector<int> even(n, 0);
28     l = 0; r = -1;
29     for(int i = 0; i < n; i++){
30         int k = (i > r) ? 0 : min(even[l + r - i + 1], r - i + 1);
31         while(i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
32         even[i] = k;
33         if(i + k - 1 > r){ l = i - k; r = i + k - 1; }
34     }
35     return {odd, even};
36 }
37
38 // LONGEST PALINDROMIC SUBSTRING
39 string longestPalindrome(const string& s,
40     const vector<int>& odd,
41     const vector<int>& even){
42     int n = (int)s.size(), bestL = 0, bestLen = 1;
43     for(int i = 0; i < n; i++){
44         // impar centrado en i: longitud 2*odd[i] - 1
45         int len = 2 * odd[i] - 1;
46         if(len > bestLen){ bestLen = len; bestL = i - odd[i] + 1; }
47         // par centrado entre i-1 e i: longitud 2*even[i]
48         len = 2 * even[i];
49         if(len > bestLen){ bestLen = len; bestL = i - even[i]; }
50     }
51     return s.substr(bestL, bestLen);
52 }
53
54 // COUNT PALINDROMES
55 long long countPalindromes(const vector<int>& odd,
56     const vector<int>& even){
57     long long total = 0;
58     for(int x : odd) total += x;
59     for(int x : even) total += x;
60     return total;
61 }
62
63 // PALINDROMIC SUBSTRING QUERIES
64 //   Devuelve true sii s[l..r] (0-indexed, inclusivo) es palindromo.
65 bool isPalindrome(int l, int r,
66     const vector<int>& odd,
67     const vector<int>& even){
68     int len = r - l + 1;
69     if(len <= 1) return true;
70     if(len % 2 == 1){
71         int c = (l + r) / 2; // centro del palindromo impar
72         return odd[c] >= (len + 1) / 2;
73     } else {

```

```

74     int c = (1 + r + 1) / 2;        // borde derecho del centro par
75     return even[c] >= len / 2;
76 }
77 }
78
79 int main(){
80     string s = "abacaba";
81     auto [odd, even] = manacher(s);
82
83     // imprime los radios
84     for(int x : odd) cout << x << " "; cout << "\n";
85     for(int x : even) cout << x << " "; cout << "\n";
86
87     // 1) longest palindromic substring -> "abacaba"
88     cout << "longest = " << longestPalindrome(s, odd, even) << "\n";
89
90     // 2) count palindromes -> 12 para "abacaba"
91     cout << "count = " << countPalindromes(odd, even) << "\n";
92
93     // 3) queries O(1) "es s[l..r] palindromo?"
94     // (0-indexed, inclusivo)
95     auto Q = [&](int l, int r){
96         return isPalindrome(l, r, odd, even);
97     };
98     cout << "s[0..6] aba...? " << Q(0, 6) << "\n"; // 1 -> "abacaba"
99     cout << "s[0..5] aba...? " << Q(0, 5) << "\n"; // 0
100    cout << "s[0..2] aba? " << Q(0, 2) << "\n"; // 1 -> "aba"
101    cout << "s[1..3] bac? " << Q(1, 3) << "\n"; // 0
102    cout << "s[2..4] aca? " << Q(2, 4) << "\n"; // 1 -> "aca"
103    cout << "s[0..1] ab? " << Q(0, 1) << "\n"; // 0
104    cout << "s[0..0] a? " << Q(0, 0) << "\n"; // 1
105    return 0;
106 }

```

## 4.5 Otros

### 4.1 Lyndon factorization (Duval)

```

1 // TC: O(N)
2 // usa: descomponer un string en factores de Lyndon
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> lyndonFactorization(const string& s){
7     // returns ending positions of each Lyndon factor (0-indexed)
8     int n = s.size();
9     vector<int> cuts;
10    int i = 0;
11    while(i < n){
12        int j = i, k = i + 1;
13        while(k < n && s[j] <= s[k]){
14            if(s[j] < s[k]) j = i;
15            else j++;
16            k++;

```

```

17    }
18    int len = k - j;
19    while(i <= j){
20        cuts.push_back(i + len - 1);
21        i += len;
22    }
23 }
24 return cuts;
25 }
26
27 int main(){
28     string s = "ababa";
29     auto cuts = lyndonFactorization(s);
30     for(int c : cuts) cout << c << " "; // 0 2 4
31     cout << "\n";
32     return 0;
33 }

```

## 5 Grafos

### 5.1 Caminos minimos

#### 5.1 Dijkstra

```

1 // TC: O((N + M) log N)
2 // usa: camino minimo desde s con pesos no negativos; requiere pesos >= 0
3 vector<int> dijkstra(int n, vector<vector<int>> adj[], int s){
4     set<array<int,2>> st;
5     vector<int> dist(n,1e18);
6     dist[s]=0;
7     st.insert({0,s});
8     while(!st.empty()){
9         auto it=*st.begin();
10        int dis=it[0], node=it[1];
11        st.erase(it);
12        for(auto [adjN,adjW] : adj[node]){
13            if(dis+adjW < dist[adjN]){
14                if(dist[adjN] != 1e18){
15                    st.erase({dist[adjN],adjN});
16                }
17                dist[adjN]=dis+adjW;
18                st.insert({dist[adjN],adjN});
19            }
20        }
21    }
22    return dist;
23 }
24 // TC: O((N + M) log M)
25 // usa: camino minimo desde s con pesos no negativos; inserta duplicados
    y descarta entradas viejas al sacar
26 vector<int> dijkstraPQ(int n, vector<vector<int>> adj[], int s){
27     priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,
    int>>> pq;

```

```

28 vector<int> dist(n,1e18);
29 dist[s]=0;
30 pq.push({0,s});
31 while(!pq.empty()){
32     auto [dis,node]=pq.top(); pq.pop();
33     if(dis>dist[node]) continue;
34     for(auto [adjN,adjW] : adj[node]){
35         if(dis+adjW < dist[adjN]){
36             dist[adjN]=dis+adjW;
37             pq.push({dist[adjN],adjN});
38         }
39     }
40 }
41 return dist;
42 }

```

## 5.2 Bellman-Ford

```

1 // TC: O(N * M)
2 // 1. Camino minimo desde s y deteccion de ciclo negativo alcanzable
3 vector<long long> bellman_ford(int n, vector<array<int,3>>& edges, int s,
4     bool& has_cycle){
5     const long long INF = 1e18;
6     vector<long long> dist(n, INF);
7     dist[s] = 0;
8     for(int i = 0; i < n - 1; i++){
9         for(auto [u, v, w] : edges){
10             if(dist[u] < INF && dist[u] + w < dist[v]){
11                 dist[v] = max(-INF, dist[u] + w);
12             }
13         }
14     }
15     has_cycle = false;
16     for(auto [u, v, w] : edges){
17         if(dist[u] < INF && dist[u] + w < dist[v]){
18             has_cycle = true; break;
19         }
20     }
21     return dist;
22 }
23 // 2. Reconstruir ciclo negativo (ej. CSES - Cycle Finding)
24 // Retorna vertices del ciclo {v1, v2, ..., v1} en orden dirigido, o
25 // vacio si no hay.
26 // s = -1: detecta CUALQUIER ciclo en el grafo (dist=0 para todo nodo).
27 // s >= 0: detecta ciclo negativo alcanzable desde s.
28 vector<int> get_negative_cycle(int n, vector<array<int,3>>& edges, int s
29     = -1){
30     const long long INF = 1e18;
31     vector<long long> dist(n, s == -1 ? 0 : INF);
32     vector<int> parent(n, -1);
33     if(s != -1) dist[s] = 0;
34     int last_relaxed = -1;

```

```

35     for(int rep = 0; rep < n; rep++){
36         last_relaxed = -1;
37         for(auto [u, v, w] : edges){
38             if(dist[u] < INF && dist[u] + w < dist[v]){
39                 dist[v] = max(-INF, dist[u] + w);
40                 parent[v] = u;
41                 last_relaxed = v;
42             }
43         }
44     }
45     if(last_relaxed == -1) return {}; // No hay ciclo negativo
46
47     // Retroceder N veces: el ultimo vertice relajado podria estar fuera
48     // del ciclo
49     // (en una rama alcanzable). Al retroceder N veces por parent se entra
50     // seguro al ciclo.
51     for(int i = 0; i < n; i++) last_relaxed = parent[last_relaxed];
52
53     vector<int> cycle;
54     for(int cur = last_relaxed; ; cur = parent[cur]){
55         cycle.push_back(cur);
56         if(cur == last_relaxed && cycle.size() > 1) break;
57     }
58     reverse(cycle.begin(), cycle.end());
59     return cycle;
60 }

```

## 5.3 Floyd-Warshall

```

1 // TC: O(N^3)
2 // usa: camino minimo entre todo par de nodos; permite pesos negativos;
3 // detecta ciclo negativo si ans[i][i] < 0
4 vector<vector<int>> floyd(int n, vector<vector<int>> adj){
5     vector<vector<int>> ans(n,vector<int>(n,1e18));
6     for(int i=0 ; i<n ; i++) ans[i][i]=0;
7     for(int i=0 ; i<n ; i++){
8         for(auto x : adj[i]){
9             int adjN=x[0] , adjW=x[1];
10            ans[i][adjN]=adjW;
11            ans[adjN][i]=adjW;
12        }
13    }
14
15    for(int via=0 ; via<n ; via++){
16        for(int i=0 ; i<n ; i++){
17            for(int j=0 ; j<n ; j++){
18                ans[i][j]=min(ans[i][j],
19                    ans[i][via]+ans[via][j]);
20            }
21        }
22    }
23
24    // Detect negative cycle: if any ans[i][i] < 0 after the algorithm,
25    // then there is a negative cycle reachable from i.
26    for(int i=0 ; i<n ; i++){

```

```

26     if(ans[i][i] < 0){
27         // negative cycle reachable from source i
28     }
29 }
30 return ans;
31 }

```

## 5.4 0-1 BFS

```

1 // TC: O(N + M)
2 // usa: grafos con aristas 0 o 1
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long INF = 1e18;
7
8 vector<long long> zeroOneBFS(int n, const vector<vector<pair<int,int>>>&
9     adj, int start){
10     vector<long long> dist(n, INF);
11     deque<int> q;
12     dist[start] = 0;
13     q.push_back(start);
14     while(!q.empty()){
15         int node = q.front(); q.pop_front();
16         for(auto [adjN, adjW] : adj[node]){
17             if(dist[node] + adjW < dist[adjN]){
18                 dist[adjN] = dist[node] + adjW;
19                 if(adjW == 0) q.push_front(adjN);
20                 else q.push_back(adjN);
21             }
22         }
23     }
24     return dist;
25 }
26
27 int main(){
28     vector<vector<pair<int,int>>> adj(4);
29     adj[0] = {{1,0}, {2,1}};
30     adj[1] = {{2,0}, {3,1}};
31     adj[2] = {{3,1}};
32     auto d = zeroOneBFS(4, adj, 0);
33     for(long long x : d) cout << x << " "; // 0 0 0 1
34     cout << "\n";
35     return 0;
36 }

```

## 5.2 Componentes y cortes

### 5.1 Kosaraju SCC

```

1 // TC: O(N + M)
2 // usa: encuentra componentes fuertemente conexas (SCC) usando dos DFS:
   una por orden de salida y otra sobre el grafo transpuesto

```

```

3 void dfs(int node, vector<pair<int,int>> adj[], vector<bool> &vis,
4     stack<int> &st){
5     vis[node]=1;
6     for(auto [adjN,adjW] : adj[node]){
7         if(!vis[adjN]){
8             dfs(adjN , adj , vis , st);
9         }
10    }
11    st.push(node);
12 }
13
14 void dfs2(int node, vector<pair<int,int>> adj[], vector<bool> &vis,
15     vector<int> &scc, int scc_number){
16     vis[node]=1;
17     scc[node]=scc_number;
18     for(auto [adjN,adjW] : adj[node]){
19         if(!vis[adjN]){
20             dfs2(adjN , adj , vis , scc , scc_number);
21         }
22    }
23 }
24
25 void solve(){
26     int n,m; cin >> n >> m;
27     vector< pair<int,int> > adj[n];
28     for(int i=0 ; i<m ; i++){
29         int u,v,w; cin >> u >> v >> w;
30         u--; v--;
31         adj[u].emplace_back(v,w);
32     }
33
34     vector<bool> vis(n);
35     stack<int> st;
36     for(int i=0 ; i<n ; i++){
37         if(!vis[i]){
38             dfs(i , adj , vis , st);
39         }
40     }
41
42     vector<pair<int,int>> adjT[n];
43     for(int i=0 ; i<n ; i++){
44         vis[i]=0;
45         for(auto [adjN,adjW] : adj[i]){
46             adjT[adjN].push_back({i,adjW});
47         }
48     }
49
50     vector<int> scc(n);
51     int scc_number=0;
52     while(!st.empty()){
53         int node=st.top();
54         st.pop();
55         if(!vis[node]){
56             scc_number++;
57             dfs2(node, adjT, vis, scc, scc_number);
58         }

```

```

59 }
60
61 vector<pair<int,int>> adjCond[scc_number];
62 for(int i=0 ; i<n ; i++){
63     for(auto [adjN,adjW] : adj[i]){
64         if(scc[i] != scc[adjN]){
65             adjCond[scc[i]-1].emplace_back(scc[adjN]-1,adjW);
66         }
67     }
68 }
69 }

```

## 5.2 Tarjan (puentes)

```

1 // TC: O(N + M)
2 // usa: encuentra puentes (aristas que al remover desconectan el grafo)
   usando tiempos de entrada y lowlink
3 void dfs(int node, int parent, vector<int> adj[], vector<int> &time,
4 vector<int> &low, vector<array<int,2>> &ans, int &t){
5     time[node] = low[node] = t;
6     t++;
7     for(auto adjN : adj[node]){
8         if(adjN == parent) continue;
9         if(!time[adjN]){
10             dfs(adjN, node, adj, time, low, ans, t);
11             low[node]=min(low[node],low[adjN]);
12             if(time[node] < low[adjN]){
13                 int mn=min(node,adjN) , mx=max(node,adjN);
14                 ans.push_back({mn,mx});
15             }
16         }else{
17             low[node]=min(low[node],low[adjN]);
18         }
19     }
20 }
21
22 void solve(){
23     int n,m; cin >> n >> m;
24     vector<int> adj[n+1];
25     for(int i=0 ; i<m ; i++){
26         int u,v; cin >> u >> v;
27         adj[u].push_back(v);
28         adj[v].push_back(u);
29     }
30
31     vector<int> time(n+1) , low(n+1);
32     vector<array<int,2>> ans;
33     int t=1;
34     dfs(1, -1, adj, time, low, ans, t);
35 }

```

## 5.3 Tarjan (puntos de articulacion)

```

1 // TC: O(N + M)
2 // usa: encuentra puntos de articulacion (nodos que al remover
   desconectan el grafo) usando tiempos de entrada y lowlink
3 void dfs(int node, int parent, vector<int> adj[], vector<int> &tin,
4 vector<int> &low, vector<bool> &artPoint, int &t){
5     tin[node]=low[node]=++t;
6     int child=0;
7     for(auto adjN : adj[node]){
8         if(adjN == parent) continue;
9         if(!tin[adjN]){
10             dfs(adjN, node, adj, tin, low, artPoint, t);
11             low[node]=min(low[node],low[adjN]);
12             if(low[adjN] >= tin[node] && parent != -1){
13                 artPoint[node]=true;
14             }
15             child++;
16         }else{
17             low[node]=min(low[node],tin[adjN]);
18         }
19     }
20     if(child>1 && parent!=-1) artPoint[node]=true;
21 }
22
23 void solve(){
24     int n,m; cin >> n >> m;
25     vector<int> adj[n+1];
26     for(int i=0 ; i<m ; i++){
27         int u,v; cin >> u >> v;
28         adj[u].push_back(v);
29         adj[v].push_back(u);
30     }
31
32     vector<int> tin(n+1) , low(n+1);
33     vector<bool> artPoint(n+1);
34     int t=0;
35     for(int i=1 ; i<=n ; i++){
36         if(tin[i] == 0){
37             dfs(i , -1 , adj , tin , low , artPoint, t);
38         }
39     }
40 }

```

## 5.4 Tarjan SCC

```

1 // TC: O(N + M)
2 // usa: encontrar componentes fuertemente conexos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct TarjanSCC{
7     int n, timer, numComps;
8     vector<int> comp, low, disc, st;
9     vector<bool> inStack;
10     const vector<vector<int>>& adj;

```

```

11 TarjanSCC(int n, const vector<vector<int>>& adj): n(n), adj(adj){
12     comp.assign(n, -1);
13     low.assign(n, 0);
14     disc.assign(n, 0);
15     inStack.assign(n, false);
16     timer = 0;
17     numComps = 0;
18     for(int i=0; i<n; i++) if(disc[i] == 0) dfs(i);
19 }
20 void dfs(int node){
21     disc[node] = low[node] = ++timer;
22     st.push_back(node);
23     inStack[node] = true;
24     for(auto adjN : adj[node]){
25         if(disc[adjN] == 0){
26             dfs(adjN);
27             low[node] = min(low[node], low[adjN]);
28         } else if(inStack[adjN]){
29             low[node] = min(low[node], disc[adjN]);
30         }
31     }
32     if(low[node] == disc[node]){
33         while(true){
34             int cur = st.back(); st.pop_back();
35             inStack[cur] = false;
36             comp[cur] = numComps;
37             if(cur == node) break;
38         }
39         numComps++;
40     }
41 }
42 };
43
44 int main(){
45     vector<vector<int>> adj = {{1}, {2,3}, {0}, {4}, {}};
46     TarjanSCC scc(5, adj);
47     cout << "#SCCs = " << scc.numComps << "\n"; // 3
48     return 0;
49 }

```

## 5.3 Matching

### 5.1 Kuhn (bipartite matching)

```

1 // TC:  $O(V * E)$  donde V = nodos lado izquierdo, E = aristas
2 // usa: matching maximo en grafo bipartito; usa DFS con vis para
3 // encontrar augmenting paths
4 // nota: para N, M <= 1e3 anda bien; para mas usar Hopcroft-Karp ( $O(\sqrt{V * E})$ )
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 int n, m;
9 vector<int> adj[1005];

```

```

9
10 bool try_match(int node, vector<int> adj[], vector<int>& matchR, vector<
11     bool>& vis){
12     for(auto adjN : adj[node]){
13         if(vis[adjN]) continue;
14         vis[adjN] = true;
15         if(matchR[adjN] == -1 || try_match(matchR[adjN], adj, matchR, vis)){
16             matchR[adjN] = node;
17             return true;
18         }
19     }
20     return false;
21 }
22
23 int main(){
24     // entrada: n nodos izquierda, m nodos derecha, m aristas (u, v) con u
25     // en [1..n], v en [1..m]
26     cin >> n >> m;
27     for(int i=0 ; i<m ; i++){
28         int u, v; cin >> u >> v;
29         adj[u].push_back(v);
30     }
31     vector<int> matchR(m + 1, -1);
32     int matching = 0;
33     for(int node = 1; node <= n; node++){
34         vector<bool> vis(m + 1, false);
35         if(try_match(node, adj, matchR, vis)) matching++;
36     }
37     cout << matching << "\n";
38     return 0;
39 }

```

### 5.2 Hopcroft-Karp

```

1 // TC:  $O(\sqrt{V} * E)$  donde V = nodos lado izquierdo, E = aristas
2 // usa: matching maximo en grafo bipartito; usa BFS + DFS por capas para
3 // encontrar varios augmenting paths por fase
4 // nota: para N, M <= 1e5; para N, M <= 1e3 alcanza Kuhn ( $O(V * E)$ )
5 struct HopcroftKarp {
6     int n, m; // n->#left_nodes m->#right_nodes
7     vector<vector<int>> adj;
8     vector<int> dist, matchU, matchV;
9
10     HopcroftKarp(int n, int m): n(n), m(m) {
11         adj.assign(n + 1, {});
12         matchU.assign(n + 1, 0);
13         matchV.assign(m + 1, 0);
14         dist.assign(n + 1, 0);
15     }
16
17     void addEdge(int u, int v) {
18         adj[u].push_back(v);
19     }
20
21     bool bfs() {

```

```

21 queue<int> q;
22 for (int u = 1; u <= n; u++) {
23     if (matchU[u] == 0) {
24         dist[u] = 0;
25         q.push(u);
26     } else {
27         dist[u] = -1;
28     }
29 }
30 bool found = false;
31 while (!q.empty()) {
32     int u = q.front(); q.pop();
33     for (int v : adj[u]) {
34         if (matchV[v] != 0 && dist[matchV[v]] == -1) {
35             dist[matchV[v]] = dist[u] + 1;
36             q.push(matchV[v]);
37         }
38         if (matchV[v] == 0) {
39             found = true;
40         }
41     }
42 }
43 return found;
44 }
45
46 bool dfs(int u) {
47     for (int v : adj[u]) {
48         if (matchV[v] == 0 ||
49             (dist[matchV[v]] == dist[u] + 1 && dfs(matchV[v]))) {
50             matchU[u] = v;
51             matchV[v] = u;
52             return true;
53         }
54     }
55     dist[u] = -1;
56     return false;
57 }
58
59 int maxMatching() {
60     int matching = 0;
61     while (bfs()) {
62         for (int u = 1; u <= n; u++) {
63             if (matchU[u] == 0 && dfs(u)) {
64                 matching++;
65             }
66         }
67     }
68     return matching;
69 }
70 };
71
72 int main(){
73     // entrada: n nodos izquierda, m nodos derecha, k aristas (u, v) con u
74     // en [1..n], v en [1..m]
75     int n, m, k;
76     cin >> n >> m >> k;

```

```

76 HopcroftKarp hk(n, m);
77 for(int i=0;i<k;i++){
78     int u, v; cin >> u >> v;
79     hk.addEdge(u, v);
80 }
81 cout << hk.maxMatching() << "\n";
82 return 0;
83 }

```

## 5.4 MST

### 5.1 Kruskal

```

1 // TC: O(M log M)
2 // usa: MST con aristas ordenadas por peso
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct DSUk{
7     vector<int> p, r;
8     DSUk(int n): p(n), r(n, 0){ iota(p.begin(), p.end(), 0); }
9     int find(int v){ return p[v] == v ? v : p[v] = find(p[v]); }
10    bool unite(int a, int b){
11        a = find(a); b = find(b);
12        if(a == b) return false;
13        if(r[a] < r[b]) swap(a, b);
14        p[b] = a;
15        if(r[a] == r[b]) r[a]++;
16        return true;
17    }
18 };
19
20 long long kruskal(int n, vector<array<int,3>>& edges){
21     sort(edges.begin(), edges.end(), [](auto& A, auto& B){
22         return A[2] < B[2];
23     });
24     DSUk dsu(n);
25     long long total = 0;
26     int cnt = 0;
27     for(auto [u, v, w] : edges){
28         if(dsu.unite(u, v)){
29             total += w;
30             if(++cnt == n - 1) break;
31         }
32     }
33     return total;
34 }
35
36 int main(){
37     vector<array<int,3>> edges = {{0,1,4},{1,2,1},{2,3,2},{0,3,5}};
38     cout << "MST weight = " << kruskal(4, edges) << "\n"; // 7
39     return 0;
40 }

```

## 5.2 Prim (priority queue)

```
1 // TC:  $O((N + M) \log N)$ 
2 // usa: MST alternativo, mejor en grafos densos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long INF = 1e18;
7
8 long long prim(int n, const vector<vector<pair<int,int>>>& adj){
9     vector<long long> dist(n, INF);
10    vector<bool> vis(n, false);
11    priority_queue<pair<long long,int>, vector<pair<long long,int>>,
12        greater<pair<long long,int>>> pq;
13    dist[0] = 0;
14    pq.push({0, 0});
15    long long total = 0;
16    while(!pq.empty()){
17        auto [dis, node] = pq.top(); pq.pop();
18        if(vis[node]) continue;
19        vis[node] = true;
20        total += dis;
21        for(auto [adjN, adjW] : adj[node]){
22            if(!vis[adjN] && adjW < dist[adjN]){
23                dist[adjN] = adjW;
24                pq.push({adjW, adjN});
25            }
26        }
27    }
28    return total;
29 }
30
31 int main(){
32     vector<vector<pair<int,int>>> adj(4);
33     adj[0] = {{1,4},{3,5}}; adj[1] = {{0,4},{2,1},{3,2}};
34     adj[2] = {{1,1},{3,2}}; adj[3] = {{0,5},{1,2},{2,2}};
35     cout << "MST weight = " << prim(4, adj) << "\n"; // 7
36 }
```

## 5.5 Basicos

### 5.1 DFS / BFS / componentes

```
1 // TC:  $O(N + M)$ 
2 // usa: recorrido, conteo de componentes
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct Graph{
7     int n;
8     vector<vector<int>> adj;
9     Graph(int n): n(n), adj(n){}
10    void addEdge(int u, int v, bool undirected = true){
```

```
11        adj[u].push_back(v);
12        if(undirected) adj[v].push_back(u);
13    }
14    vector<int> bfs(int start){
15        vector<int> dist(n, -1);
16        queue<int> q;
17        dist[start] = 0; q.push(start);
18        while(!q.empty()){
19            int node = q.front(); q.pop();
20            for(auto adjN : adj[node]) if(dist[adjN] == -1){
21                dist[adjN] = dist[node] + 1;
22                q.push(adjN);
23            }
24        }
25        return dist;
26    }
27    void dfsRec(int node, vector<bool>& vis, vector<int>& order){
28        vis[node] = true;
29        order.push_back(node);
30        for(auto adjN : adj[node]) if(!vis[adjN]) dfsRec(adjN, vis, order);
31    }
32    vector<int> dfs(int start){
33        vector<int> order;
34        vector<bool> vis(n, false);
35        dfsRec(start, vis, order);
36        return order;
37    }
38    int countComponents(){
39        vector<bool> vis(n, false);
40        vector<int> order;
41        int cnt = 0;
42        for(int i=0 ; i<n ; i++){
43            if(!vis[i]){
44                dfsRec(i, vis, order);
45                cnt++;
46            }
47        }
48        return cnt;
49    }
50 };
51
52 int main(){
53     Graph g(5);
54     g.addEdge(0,1); g.addEdge(1,2); g.addEdge(3,4); // 2 components
55     cout << "components: " << g.countComponents() << "\n"; // 2
56     auto dist = g.bfs(0);
57     for(int d : dist) cout << d << " ";
58     cout << "\n";
59     return 0;
60 }
```

### 5.2 Topological sort (Kahn + DFS)

```
1 // TC:  $O(N + M)$ 
2 // usa: orden lineal de un DAG
```

```

3 #include <bits/stdc++.h>
4 using namespace std;
5
6 vector<int> topoKahn(int n, const vector<vector<int>>& adj){
7     vector<int> indegree(n, 0);
8     for(int i=0 ; i<n ; i++) for(auto adjN : adj[i]) indegree[adjN]++;
9     queue<int> q;
10    for(int i=0 ; i<n ; i++) if(indegree[i] == 0) q.push(i);
11    vector<int> order;
12    while(!q.empty()){
13        int node = q.front(); q.pop();
14        order.push_back(node);
15        for(auto adjN : adj[node]){
16            if(--indegree[adjN] == 0) q.push(adjN);
17        }
18    }
19    return order; // size < n if cycle
20 }
21
22 void dfsVisit(int node, const vector<vector<int>>& adj, vector<int>&
23 state, vector<int>& order){
24     state[node] = 1;
25     for(auto adjN : adj[node]){
26         // if(state[adjN] == 1) -> ciclo: back edge a ancestro en la pila de
27         // recursion
28         if(state[adjN] == 0)
29             dfsVisit(adjN, adj, state, order);
30     }
31     state[node] = 2;
32     order.push_back(node);
33 }
34
35 vector<int> topoDFS(int n, const vector<vector<int>>& adj){
36     vector<int> order;
37     vector<int> state(n, 0); // 0=white, 1=gray, 2=black
38     for(int i=0 ; i<n ; i++){
39         if(state[i] == 0)
40             dfsVisit(i, adj, state, order);
41     }
42     reverse(order.begin(), order.end());
43     return order;
44 }
45
46 int main(){
47     vector<vector<int>> adj = {{1,2}, {3}, {3}, {}};
48     auto order = topoKahn(4, adj);
49     for(int node : order) cout << node << " "; // 0 1 2 3
50     cout << "\n";
51     return 0;
52 }

```

### 5.3 DP en DAG (camino mas largo / corto con relajacion)

```

1 // TC: O(N + M)
2 // usa: Procesar en orden topologico relaja cada arista EXACTAMENTE 1 vez
  // (Bellman-Ford lineal).

```

```

3 // Sirve para camino mas largo/corto con pesos arbitrarios (positivos o
4 // negativos) y conteo de caminos.
5 const long long INF = 1e18;
6
7 // Retorna {dist, parent}. dist[i] = -INF si i es inalcanzable desde s.
8 pair<vector<long long>, vector<int>> dagLongestPath(int n, const vector<
9 vector<pair<int, long long>>>& adj, const vector<int>& topo, int s){
10     vector<long long> dist(n, -INF);
11     vector<int> parent(n, -1);
12     dist[s] = 0; // o 1 si se cuentan ciudades/vertices
13
14     for(auto node : topo){
15         if(dist[node] == -INF) continue; // Inalcanzable desde s
16         for(auto [adjN, adjW] : adj[node]){
17             if(dist[node] + adjW > dist[adjN]){
18                 dist[adjN] = dist[node] + adjW;
19                 parent[adjN] = node; // Para reconstruir la ruta
20             }
21         }
22     }
23     return {dist, parent};
24 }
25
26 // Reconstruir camino de s a t (retorna vacio si t es inalcanzable)
27 vector<int> getPath(int s, int t, const vector<int>& parent){
28     if(s != t && parent[t] == -1) return {};
29     vector<int> path;
30     for(int cur = t; cur != -1; cur = parent[cur]) path.push_back(cur);
31     reverse(path.begin(), path.end());
32     if(path.empty() || path[0] != s) return {};
33     return path;
34 }
35
36 // Conteo de caminos s -> t (modulo MOD):
37 // vector<long long> ways(n, 0); ways[s] = 1;
38 // for(auto node : topo) if(ways[node]) for(auto [adjN, adjW] : adj[node
39 // ]){ ways[adjN] = (ways[adjN] + ways[node]) % MOD;

```

### 5.4 Bipartite check + 2-coloring

```

1 // TC: O(N + M)
2 // usa: chequear si un grafo es bipartito y asignar colores
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 pair<bool, vector<int>> isBipartite(int n, const vector<vector<int>>& adj
7 ){
8     vector<int> color(n, -1);
9     for(int i=0 ; i<n ; i++){
10         if(color[i] != -1) continue;
11         queue<int> q;
12         color[i] = 0; q.push(i);
13         while(!q.empty()){
14             int node = q.front(); q.pop();
15             for(auto adjN : adj[node]){

```

```

15         if(color[adjN] == -1){ color[adjN] = color[node] ^ 1; q.push(adjN
16     ); }
17     else if(color[adjN] == color[node]) return {false, {}};
18     }
19 }
20 return {true, color};
21 }
22
23 int main(){
24     vector<vector<int>>> adj = {{1}, {0,2}, {1,3}, {2}}; // path of 4:
25     bipartite
26     auto [ok, color] = isBipartite(4, adj);
27     cout << (ok ? "bipartite" : "not bipartite") << "\n";
28     return 0;
29 }

```

## 5.6 Flujos

### 5.1 Ford-Fulkerson (DFS max flow)

```

1 // TC: O(M * max_flow) con DFS
2 // usa: max flow clasico; busca caminos aumentantes con DFS
3 // diferencia con Edmonds-Karp: FF usa DFS (depende del flujo), EK usa
4 // BFS O(N * M^2)
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 const long long INF = 1e18;
9
10 long long dfs(int node, int t, long long pushed, vector<vector<pair<int,
11     long long>>>& adj, vector<int>& vis){
12     if(node == t || pushed == 0) return pushed;
13     vis[node] = 1;
14     for(auto& [adjN, cap] : adj[node]){
15         if(!vis[adjN] && cap > 0){
16             long long tr = dfs(adjN, t, min(pushed, cap), adj, vis);
17             if(tr > 0){
18                 cap -= tr;
19                 for(auto& [backNode, backCap] : adj[adjN]){
20                     if(backNode == node){
21                         backCap += tr;
22                         break;
23                     }
24                 }
25                 return tr;
26             }
27         }
28     }
29     return 0;
30 }
31
32 long long fordFulkerson(int n, vector<vector<pair<int, long long>>>& adj,
33     int s, int t){

```

```

31     long long flow = 0;
32     while(true){
33         vector<int> vis(n, 0);
34         long long pushed = dfs(s, t, INF, adj, vis);
35         if(pushed == 0) break; // no existe camino aumentante
36         flow += pushed;
37     }
38     return flow;
39 }
40
41 int main(){
42     // Grafo de 4 nodos (mismo ejemplo que Edmonds-Karp)
43     vector<vector<pair<int, long long>>> adj(4);
44     auto add = [&](int u, int v, long long c){
45         adj[u].push_back({v, c});
46         adj[v].push_back({u, 0});
47     };
48     add(0,1,10); add(0,2,5); add(1,2,15); add(1,3,10); add(2,3,10);
49     cout << "max flow = " << fordFulkerson(4, adj, 0, 3) << "\n"; // 15
50     return 0;
51 }

```

### 5.2 Edmonds-Karp (max flow)

```

1 // TC: O(N * M^2)
2 // usa: max flow simple; sirve como baseline o para grafos pequenos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long INF = 1e18;
7
8 long long edmondsKarp(int n, vector<vector<pair<int, long long>>>& adj,
9     int s, int t){
10     long long flow = 0;
11     while(true){
12         vector<int> parent(n, -1);
13         queue<int> q;
14         parent[s] = s;
15         q.push(s);
16         while(!q.empty() && parent[t] == -1){
17             int node = q.front(); q.pop();
18             for(auto [adjN, cap] : adj[node]){
19                 if(cap > 0 && parent[adjN] == -1){
20                     parent[adjN] = node;
21                     q.push(adjN);
22                 }
23             }
24         }
25         if(parent[t] == -1) break;
26         long long add = INF;
27         for(int v=t; v != s; v = parent[v]){
28             for(auto [adjN, cap] : adj[parent[v]]){
29                 if(adjN == v) add = min(add, cap);
30             }
31         }
32         flow += add;
33         for(int v=t; v != s; v = parent[v]){
34             for(auto [adjN, cap] : adj[parent[v]]){
35                 if(adjN == v) cap -= add;
36             }
37         }
38     }
39     return flow;
40 }

```

```

31     for(int v=t; v != s; v = parent[v]){
32         for(auto& [adjN, cap] : adj[parent[v]]){
33             if(adjN == v) cap -= add;
34         }
35         for(auto& [adjN, cap] : adj[v]){
36             if(adjN == parent[v]) cap += add;
37         }
38     }
39     flow += add;
40 }
41 return flow;
42 }
43
44 int main(){
45     // 4-node max flow graph
46     vector<vector<pair<int, long long>>> adj(4);
47     auto add = [&](int u, int v, long long c){
48         adj[u].push_back({v, c});
49         adj[v].push_back({u, 0});
50     };
51     add(0,1,10); add(0,2,5); add(1,2,15); add(1,3,10); add(2,3,10);
52     cout << "max flow = " << edmondsKarp(4, adj, 0, 3) << "\n"; // 15
53     return 0;
54 }

```

### 5.3 Dinic (max flow)

```

1 // TC:  $O(N^2 * M)$  general,  $O(\sqrt{N} * M)$  for bipartite
2 // usa: max flow eficiente, base para muchos problemas
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long INF = 1e18;
7
8 struct Dinic{
9     struct Edge{ int adjN, rev; long long cap; };
10    int n, s, t;
11    vector<vector<Edge>> adj;
12    vector<int> level, it;
13    Dinic(int n, int s, int t): n(n), s(s), t(t){
14        adj.assign(n, {});
15        level.assign(n, -1);
16        it.assign(n, 0);
17    }
18    void addEdge(int u, int v, long long cap){
19        Edge a{v, (int)adj[v].size(), cap};
20        Edge b{u, (int)adj[u].size(), 0};
21        adj[u].push_back(a);
22        adj[v].push_back(b);
23    }
24    bool bfs(){
25        fill(level.begin(), level.end(), -1);
26        queue<int> q;
27        level[s] = 0; q.push(s);
28        while(!q.empty()){

```

```

29            int node = q.front(); q.pop();
30            for(const auto& [adjN, rev, cap] : adj[node]){
31                if(cap > 0 && level[adjN] == -1){
32                    level[adjN] = level[node] + 1;
33                    q.push(adjN);
34                }
35            }
36        }
37        return level[t] != -1;
38    }
39    long long dfs(int node, long long pushed){
40        if(pushed == 0) return 0;
41        if(node == t) return pushed;
42        for(int& i = it[node]; i < (int)adj[node].size(); i++){
43            auto& [adjN, rev, cap] = adj[node][i];
44            if(cap > 0 && level[adjN] == level[node] + 1){
45                long long tr = dfs(adjN, min(pushed, cap));
46                if(tr == 0) continue;
47                cap -= tr;
48                adj[adjN][rev].cap += tr;
49                return tr;
50            }
51        }
52        return 0;
53    }
54    long long maxflow(){
55        long long flow = 0;
56        while(bfs()){
57            fill(it.begin(), it.end(), 0);
58            while(long long pushed = dfs(s, INF)){
59                flow += pushed;
60            }
61        }
62        return flow;
63    }
64 };
65
66 int main(){
67     Dinic dinic(4, 0, 3);
68     dinic.addEdge(0,1,10);
69     dinic.addEdge(0,2,5);
70     dinic.addEdge(1,2,15);
71     dinic.addEdge(1,3,10);
72     dinic.addEdge(2,3,10);
73     cout << "max flow = " << dinic.maxflow() << "\n"; // 15
74     return 0;
75 }

```

### 5.4 Min-Cost Max-Flow

```

1 // TC:  $O(F * N * M)$  with SPFA; swap for Dijkstra + potentials to get  $O(F * (N+M) \log N)$ 
2 // usa: maximo flujo con costo minimo (asume costos no negativos)
3 #include <bits/stdc++.h>
4 using namespace std;

```

```

5  const long long INF = 1e18;
6
7
8  struct MCMF{
9      // to = destino, rev = indice de la arista inversa en g[to]
10     // cap = capacidad restante, cost = costo unitario
11     struct Edge{ int to, rev; long long cap, cost; };
12     int n;
13     // dist = distancia reducida desde s
14     vector<vector<Edge>> g;
15     // pot = potentials para Dijkstra (h(v) en reweighting)
16     vector<long long> dist, pot;
17     // parent_v = nodo anterior en el camino aumentante, parent_e = indice
18     // de la arista usada
19     vector<int> parent_v, parent_e;
20     MCMF(int n): n(n){
21         g.assign(n, {});
22         dist.assign(n, INF);
23         pot.assign(n, 0);
24         parent_v.assign(n, -1);
25         parent_e.assign(n, -1);
26     }
27     void addEdge(int u, int v, long long cap, long long cost){
28         Edge a{v, (int)g[v].size(), cap, cost}; // arista real
29         Edge b{u, (int)g[u].size(), 0, -cost}; // arista residual (cap 0,
30         // costo negativo)
31         g[u].push_back(a);
32         g[v].push_back(b);
33     }
34     // returns (maxflow, mincost)
35     pair<long long, long long> minCostMaxFlow(int s, int t){
36         long long flow = 0, cost = 0;
37         while(true){
38             // SPFA con potentials: busca el camino de costo minimo de s a t en
39             // el grafo residual
40             fill(dist.begin(), dist.end(), INF);
41             dist[s] = 0;
42             vector<bool> inq(n, false); // inq = nodo actualmente en la
43             // cola de SPFA
44             queue<int> q;
45             q.push(s); inq[s] = true;
46             while(!q.empty()){
47                 int v = q.front(); q.pop();
48                 inq[v] = false;
49                 for(int i=0; i<(int)g[v].size(); i++){
50                     auto& e = g[v][i];
51                     // arista relajable con costos reducidos (pot[v] - pot[e.to] >=
52                     // 0)
53                     if(e.cap > 0 && dist[v] + e.cost + pot[v] - pot[e.to] < dist[e.
54                     to]){
55                         dist[e.to] = dist[v] + e.cost + pot[v] - pot[e.to];
56                         parent_v[e.to] = v;
57                         parent_e[e.to] = i;
58                         if(!inq[e.to]){ inq[e.to] = true; q.push(e.to); }
59                     }
60                 }
61             }
62         }
63     }
64 }

```

```

55     }
56     if(dist[t] == INF) break; // no hay mas camino de s a t
57     // actualiza potentials para mantener costos reducidos no negativos
58     for(int v=0; v<n; v++){ if(dist[v] < INF) pot[v] += dist[v];
59     // encuentra el cuello de botella (minima capacidad en el camino)
60     long long add = INF;
61     for(int v=t; v != s; v = parent_v[v]){
62         add = min(add, g[parent_v[v]][parent_e[v]].cap);
63     }
64     // empuja flujo por el camino y suma el costo
65     for(int v=t; v != s; v = parent_v[v]){
66         auto& e = g[parent_v[v]][parent_e[v]];
67         e.cap -= add;
68         g[v][e.rev].cap += add;
69         cost += add * e.cost;
70     }
71     flow += add;
72 }
73 return {flow, cost};
74 }
75 };
76
77 int main(){
78     MCMF mcmf(4);
79     mcmf.addEdge(0,1,10,1);
80     mcmf.addEdge(0,2,5,2);
81     mcmf.addEdge(1,3,10,1);
82     mcmf.addEdge(2,3,10,1);
83     auto [flow, cost] = mcmf.minCostMaxFlow(0, 3);
84     cout << "flow=" << flow << " cost=" << cost << "\n";
85     return 0;
86 }

```

## 5.7 Especiales

### 5.1 2-SAT (implication graph + SCC)

```

1  // TC: O(N + M)
2  // usa: formulas booleanas en CNF con clausulas de 2 literales
3  #include <bits/stdc++.h>
4  using namespace std;
5
6  struct TwoSAT{
7      int n;
8      vector<vector<int>> adj, radj;
9      TwoSAT(int n): n(n){
10         adj.assign(2*n, {});
11         radj.assign(2*n, {}); // transposed graph
12     }
13     // literal x is true if value[x] = 1; literal !x corresponds to x ^ 1
14     void implies(int x, int y){
15         adj[x].push_back(y);
16         radj[y].push_back(x);
17     }

```

```

18 void addOr(int x, bool fx, int y, bool fy){
19     implies(x ^ fx, y ^ fy);
20     implies(y ^ fy, x ^ fx);
21 }
22 void setTrue(int x){
23     implies(x ^ 1, x);
24 }
25 void setFalse(int x){
26     implies(x, x ^ 1);
27 }
28 void dfs1(int node, vector<int>& vis, vector<int>& order){
29     vis[node] = 1;
30     for(auto adjN : adj[node])
31         if(!vis[adjN])
32             dfs1(adjN, vis, order);
33     order.push_back(node);
34 }
35 void dfs2(int node, int cl, vector<int>& comp){
36     comp[node] = cl;
37     for(auto adjN : radj[node])
38         if(comp[adjN] == -1)
39             dfs2(adjN, cl, comp);
40 }
41 vector<bool> solve(){
42     vector<int> comp(2*n, -1), order, vis(2*n, 0);
43     for(int i=0 ; i<2*n ; i++){
44         if(!vis[i])
45             dfs1(i, vis, order);
46         int j = 0;
47         for(int i=2*n-1 ; i>=0 ; i--){
48             if(comp[order[i]] == -1)
49                 dfs2(order[i], j++, comp);
50         }
51         vector<bool> val(n, false);
52         for(int i=0 ; i<n ; i++){
53             if(comp[2*i] == comp[2*i+1])
54                 return {}; // empty = unsat
55             val[i] = comp[2*i] < comp[2*i+1];
56         }
57         return val;
58     }
59 }
60 int main(){
61     TwoSAT ts(2);
62     // (x0 OR x1) AND (!x0 OR !x1) -> x0 XOR x1
63     ts.addOr(0, false, 1, false); // !x0 -> x1
64     ts.addOr(0, true, 1, true); // x0 -> !x1
65     // ts.setTrue(0); // fuerza x0 = true
66     // ts.setFalse(1); // fuerza x1 = false
67     auto val = ts.solve();
68     if(val.empty()) cout << "UNSAT\n";
69     else cout << "x0=" << val[0] << " x1=" << val[1] << "\n";
70     return 0;
71 }

```

## 5.2 Euler tour / Hierholzer (camino y circuito)

**Teoria de Grafos Eulerianos** ( $O(V + E)$ ): Recorre cada **arista** exactamente una vez. Todas las aristas deben pertenecer a una sola componente conexa (se ignoran vertices aislados de grado 0).

- **Circuito Euleriano**: Empieza y termina en el **mismo** vertice ( $A \rightarrow \dots \rightarrow A$ ).
- **Camino Euleriano**: Empieza en  $A$  y termina en  $B$  con  $B \neq A$  ( $A \rightarrow \dots \rightarrow B$ ).
- **No dirigido - Circuito**: Todos los vertices tienen **grado par**.
- **No dirigido - Camino**: Exactamente **dos vertices tienen grado impar** (inicio  $A$  y fin  $B$ ); todos los demas tienen grado par.
- **Dirigido - Circuito**: Para todo vertice  $v$ ,  $\text{in}(v) = \text{out}(v)$ .
- **Dirigido - Camino**: Inicio  $A$  con  $\text{out}(A) - \text{in}(A) = 1$ , fin  $B$  con  $\text{in}(B) - \text{out}(B) = 1$ , y para todos los demas  $\text{in}(v) = \text{out}(v)$ .
- **Algoritmo de Hierholzer**: DFS que consume aristas; en post-order se apila el vertice y se invierte al final ( $O(V + E)$ ).

```

1 // TC: O(N + M) - Algoritmo de Hierholzer
2 // usa: existencia y construccion de camino / circuito euleriano
3 // (visita cada ARISTA exactamente una vez; admite multigrafos)
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 // ----- Condiciones necesarias y suficientes -----
8 // NO DIRIGIDO, conexo:
9 // - circuito euleriano: todos los vertices tienen grado par
10 // - camino euleriano: exactamente 0 o 2 vertices con grado impar
11 // (si son 2, el camino va de uno al otro)
12 // DIRIGIDO, debilmente conexo sobre el subgrafo con aristas:
13 // - circuito euleriano: indeg(v) == outdeg(v) para todo v
14 // - camino euleriano: existe s con outdeg(s) = indeg(s)+1,
15 // existe t con indeg(t) = outdeg(t)+1,
16 // y el resto cumple indeg == outdeg
17
18 // ----- Existencia (no dirigido) -----
19 // returns: 0 = circuito, 1 = camino (start!=end), -1 = no existe
20 int eulerianStatusUndirected(int n, const vector<vector<int>>& adj){
21     int odd = 0, any = -1;
22     for(int v=0; v<n; v++){
23         int d = (int)adj[v].size();
24         if(d & 1) odd++;
25         if(d > 0 && any == -1) any = v;
26     }
27     if(any == -1) return 0; // grafo vacio -> circuito trivial
28     // chequeo rapido de conexion entre vertices con aristas
29     vector<int> vis(n, 0);
30     queue<int> q; q.push(any); vis[any] = 1;
31     while(!q.empty()){
32         int node = q.front(); q.pop();
33         for(auto adjN : adj[node]) if(!vis[adjN]){ vis[adjN] = 1; q.push(adjN); }

```

```

34 }
35 for(int v=0;v<n;v++) if(adj[v].size() && !vis[v]) return -1;
36 if(odd == 0) return 0;
37 if(odd == 2) return 1;
38 return -1;
39 }
40
41 // ----- Existencia (dirigido) -----
42 // devuelve: 0 = circuito, 1 = camino, -1 = no existe; *start = nodo
    inicial
43 int eulerianStatusDirected(int n, const vector<vector<int>>& adj,
44 int& start){
45     int moreOut = -1, moreIn = -1, bad = 0;
46     start = 0;
47     for(int v=0;v<n;v++){
48         if(!adj[v].empty()) start = v;
49         int indeg = 0;
50         for(int u=0;u<n;u++) for(int x : adj[u]) if(x == v) indeg++;
51         int outdeg = (int)adj[v].size();
52         int diff = outdeg - indeg;
53         if(diff == 1){ if(moreOut != -1) bad = 1; moreOut = v; }
54         else if(diff == -1){ if(moreIn != -1) bad = 1; moreIn = v; }
55         else if(diff != 0) bad = 1;
56     }
57     if(bad) return -1;
58     if(moreOut == -1) return 0;           // circuito
59     if(moreIn == -1) return -1;
60     start = moreOut;
61     return 1;                           // camino
62 }
63
64 // ----- Hierholzer (no dirigido, multigrafo) -----
65 // adj guarda (vecino, id) para cada arista; cada arista no dirigida
    aparece
66 // en adj[u] y adj[v]. Tras llamar, adj queda mutado con ids 0..M-1.
67 // Retorna la secuencia de vertices; si es circuito, path.front() == path
    .back().
68 vector<int> eulerianPathUndirected(int n, vector<vector<pair<int,int>>>&
    adj){
69     // reconstruye el grafo asignando ids consistentes
70     vector<vector<pair<int,int>>> g(n);
71     int idx = 0;
72     for(int u=0;u<n;u++){
73         for(auto& [v, idOld] : adj[u]){
74             if(u < v){ // procesa cada arista no dirigida una sola vez
75                 g[u].push_back({v, idx});
76                 g[v].push_back({u, idx});
77                 idx++;
78             }
79         }
80     }
81     vector<int> vis(idx, 0); // vis de aristas (eid), no de vertices
82     adj = move(g);
83
84     // elegir inicio segun existencia (trabajamos con la lista de vecinos)
85     vector<vector<int>> nbrs(n);

```

```

86     for(int u=0;u<n;u++) for(auto& [v, eid] : adj[u]) nbrs[u].push_back(v);
87     int start = 0, status = eulerianStatusUndirected(n, nbrs);
88     if(status == 1){
89         for(int v=0;v<n;v++) if(((int)adj[v].size()) & 1){ start = v; break;
90         }
91     }else if(status == 0){
92         for(int v=0;v<n;v++) if(!adj[v].empty()){ start = v; break; }
93     }else return {};
94
95     stack<int> st;
96     vector<int> path, it(n, 0);
97     st.push(start);
98     while(!st.empty()){
99         int v = st.top();
100         while(it[v] < (int)adj[v].size() && vis[adj[v][it[v]].second]) it[v]
101         ++;
102         if(it[v] < (int)adj[v].size()){
103             auto [u, eid] = adj[v][it[v]];
104             vis[eid] = 1;
105             st.push(u);
106         }else{
107             path.push_back(v);
108             st.pop();
109         }
110     }
111     reverse(path.begin(), path.end());
112     // si el camino no usa todas las aristas, no existe euleriano
113     int total = 0; for(int x : vis) total += x;
114     if(total != (int)vis.size()) return {};
115     return path;
116 }
117
118 // ----- Hierholzer (dirigido) -----
119 vector<int> eulerianPathDirected(int n, vector<vector<pair<int,int>>>&
    adj){
120     // reescribir ids consistentes
121     vector<vector<pair<int,int>>> g(n);
122     int idx = 0;
123     for(int u=0;u<n;u++){
124         for(auto& [v, idOld] : adj[u]){
125             g[u].push_back({v, idx++});
126         }
127     }
128     vector<int> vis(idx, 0); // vis de aristas (eid), no de vertices
129     adj = move(g);
130
131     // existencia (sobre la lista de vecinos)
132     vector<vector<int>> nbrs(n);
133     for(int u=0;u<n;u++) for(auto& [v, eid] : adj[u]) nbrs[u].push_back(v);
134     int start = 0;
135     int status = eulerianStatusDirected(n, nbrs, start);
136     if(status == -1) return {};
137
138     stack<int> st;
139     vector<int> path, it(n, 0);
140     st.push(start);

```

```

139 while(!st.empty()){
140     int v = st.top();
141     while(it[v] < (int)adj[v].size() && vis[adj[v][it[v]].second]) it[v]++;
142     if(it[v] < (int)adj[v].size()){
143         auto [u, eid] = adj[v][it[v]];
144         vis[eid] = 1;
145         st.push(u);
146     }else{
147         path.push_back(v);
148         st.pop();
149     }
150 }
151 reverse(path.begin(), path.end());
152 int total = 0; for(int x : vis) total += x;
153 if(total != (int)vis.size()) return {};
154 return path;
155 }
156
157 int main(){
158     // ---- no dirigido: triangulo 0-1-2-0 tiene circuito euleriano (todos
159     // grado par) ----
160     vector<vector<int>> adjU(3);
161     adjU[0] = {1,2}; adjU[1] = {0,2}; adjU[2] = {0,1};
162     int stU = eulerianStatusUndirected(3, adjU);
163     cout << "no dirigido status: " << stU << " (0=circuito, 1=camino)\n";
164
165     // ---- dirigido: 0->1->2->0 es circuito; agregamos 0->3->2 para tener
166     // camino 0->2 ----
167     vector<vector<int>> adjD(4);
168     adjD[0] = {1,3}; adjD[1] = {2}; adjD[2] = {0}; adjD[3] = {2};
169     int startD = 0;
170     int stD = eulerianStatusDirected(4, adjD, startD);
171     cout << "dirigido status: " << stD << " start=" << startD << "\n";
172
173     // reconstruir un circuito euleriano en el triangulo
174     vector<vector<pair<int,int>>> agU(3);
175     agU[0].push_back({1, 0}); agU[1].push_back({0, 0});
176     agU[1].push_back({2, 1}); agU[2].push_back({1, 1});
177     agU[2].push_back({0, 2}); agU[0].push_back({2, 2});
178     auto pathU = eulerianPathUndirected(3, agU);
179     for(int v : pathU) cout << v << " ";
180     cout << "\n";
181     return 0;
182 }

```

### 5.3 Hamiltonian path / cycle (bitmask DP + backtracking)

**Teoria de Grafos Hamiltonianos (NP-completo):** Recorre cada **vertice** exactamente una vez.

- **Ciclo Hamiltoniano:** Empieza y termina en el **mismo** vertice ( $A \rightarrow \dots \rightarrow A$ ), visitando cada vertice intermedio exactamente una vez.

- **Camino Hamiltoniano:** Empieza en  $A$  y termina en  $B$  con  $B \neq A$  ( $A \rightarrow \dots \rightarrow B$ ), visitando cada vertice del grafo exactamente una vez.
- **Teorema de Dirac** (suficiente): Si  $n \geq 3$  y para todo  $v$ ,  $\deg(v) \geq \lceil n/2 \rceil$ , existe ciclo hamiltoniano.
- **Teorema de Ore** (suficiente): Si para todo par no adyacente  $(u, v)$ ,  $\deg(u) + \deg(v) \geq n$ , existe ciclo hamiltoniano.
- **Grafos Torneo:** En cualquier torneo (grafo dirigido completo), **siempre** existe al menos un camino hamiltoniano.
- **Enfoque CP** ( $n \leq 20$ ): Bitmask DP en  $O(2^n \cdot n^2)$  o  $O(2^n \cdot n)$ . Estado  $dp[mask][v] = 1$  (camino sobre subconjunto  $mask$  terminando en  $v$ ).
- **Cuadriculas / Caballo:** Heurística de Warnsdorff (greedy hacia el vecino con menor grado de salida).

```

1 // TC: O(N^2 * 2^N) (bitmask DP), O(N!) peor caso (backtracking)
2 // usa: existencia y construccion de camino / ciclo hamiltoniano;
3 // (visita cada VERTICE exactamente una vez; NP-completo)
4 // TSP (min costo de ciclo hamiltoniano) si las aristas tienen peso
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 // -- Teorema de Dirac (condicion SUFICIENTE) --
9 // Si G es simple, |V| = n >= 3, y todo vertice tiene grado >= n/2,
10 // entonces G tiene un ciclo hamiltoniano. (No es necesario.)
11 // Ore: si para todo par no adyacente (u,v) se cumple
12 // deg(u) + deg(v) >= n, entonces G es hamiltoniano.
13
14 bool satisfiesDirac(int n, const vector<vector<int>>& adj){
15     if(n < 3) return false;
16     for(int v=0;v<n;v++) if((int)adj[v].size() < (n+1)/2) return false;
17     return true;
18 }
19
20 bool satisfiesOre(int n, const vector<vector<int>>& adj){
21     if(n < 3) return false;
22     // construye conjunto de adyacencia
23     vector<vector<char>> A(n, vector<char>(n, 0));
24     for(int u=0;u<n;u++) for(int v:adj[u]) A[u][v] = 1;
25     for(int u=0;u<n;u++)
26         for(int v=u+1;v<n;v++)
27             if(!A[u][v] && ((int)adj[u].size() + (int)adj[v].size() < n))
28                 return false;
29     return true;
30 }
31
32 // -- Bitmask DP: Hamiltonian path starting at 'src' --
33 // dp[mask][v] = true si hay un camino que visita exactamente 'mask' y
34 // termina en v
35 // memoria: O(N * 2^N) bool; tiempo: O(N^2 * 2^N)
36 bool hasHamiltonianPathFrom(int n, const vector<vector<int>>& adj, int
src){
    vector<vector<char>> dp(1<<n, vector<char>(n, 0));
    dp[1<<src][src] = 1;

```

```

37 for(int mask=1; mask<(1<<n); mask++){
38     for(int node=0 ; node<n ; node++) if(dp[mask][node]){
39         for(auto adjN : adj[node]){
40             if(!(mask & (1<<adjN))){
41                 dp[mask | (1<<adjN)][adjN] = 1;
42             }
43         }
44     }
45 }
46 for(int mask=1; mask<(1<<n); mask++){
47     if(__builtin_popcount(mask) != n) continue;
48     for(int node=0 ; node<n ; node++) if(dp[mask][node]) return true;
49 }
50 return false;
51 }
52
53 // -- Bitmask DP: Hamiltonian cycle --
54 // dp[mask][node] = true si hay un camino de 'src' que visita 'mask' y
55 // termina en node
56 bool hasHamiltonianCycle(int n, const vector<vector<int>>& adj, int src =
57 0){
58     if(n < 3) return false;
59     vector<vector<char>> dp(1<<n, vector<char>(n, 0));
60     dp[1<<src][src] = 1;
61     for(int mask=1; mask<(1<<n); mask++){
62         for(int node=0 ; node<n ; node++) if(dp[mask][node]){
63             for(auto adjN : adj[node]){
64                 if(!(mask & (1<<adjN))){
65                     dp[mask | (1<<adjN)][adjN] = 1;
66                 }
67             }
68         }
69     }
70     int full = (1<<n) - 1;
71     for(int node=0 ; node<n ; node++){
72         if(dp[full][node]){
73             // cierra el ciclo si node es adyacente a src
74             for(auto adjN : adj[node]) if(adjN == src) return true;
75         }
76     }
77     return false;
78 }
79
80 // -- Reconstruccion de un camino hamiltoniano (bitmask DP) --
81 // devuelve la secuencia de vertices o vacio si no existe
82 vector<int> findHamiltonianPath(int n, const vector<vector<int>>& adj,
83 int src){
84     vector<vector<int>> par(1<<n, vector<int>(n, -1));
85     vector<vector<char>> dp(1<<n, vector<char>(n, 0));
86     dp[1<<src][src] = 1;
87     for(int mask=1; mask<(1<<n); mask++){
88         for(int node=0 ; node<n ; node++) if(dp[mask][node]){
89             for(auto adjN : adj[node]){
90                 if(!(mask & (1<<adjN))){
91                     dp[mask | (1<<adjN)][adjN] = 1;
92                     par[mask | (1<<adjN)][adjN] = node;

```

```

90         }
91     }
92 }
93 }
94 int endV = -1, full = -1;
95 for(int mask=1; mask<(1<<n); mask++){
96     if(__builtin_popcount(mask) != n) continue;
97     for(int v=0; v<n; v++) if(dp[mask][v]){ endV = v; full = mask; break;
98     }
99     if(endV != -1) break;
100 }
101 if(endV == -1) return {};
102 vector<int> path;
103 int cur = endV, m = full;
104 while(cur != -1){
105     path.push_back(cur);
106     int p = par[m][cur];
107     m &= ~(1<<cur);
108     cur = p;
109 }
110 reverse(path.begin(), path.end());
111 return path;
112 }
113
114 // -- TSP minimo: ciclo hamiltoniano de costo minimo --
115 // dp[mask][v] = min costo de camino src -> ... -> v visitando 'mask'
116 // respuesta = min_v dp[full][v] + w(v, src) si el cierre es valido
117 const long long LINF = (1LL<<60);
118 long long tspMinCycle(int n, const vector<vector<long long>>& w, int src
119 = 0){
120     if(n == 1) return 0;
121     vector<vector<long long>> dp(1<<n, vector<long long>(n, LINF));
122     dp[1<<src][src] = 0;
123     for(int mask=1; mask<(1<<n); mask++){
124         for(int v=0; v<n; v++){
125             if(dp[mask][v] == LINF) continue;
126             for(int u=0; u<n; u++){
127                 if(!(mask & (1<<u)) && w[v][u] < LINF){
128                     dp[mask | (1<<u)][u] = min(dp[mask | (1<<u)][u],
129                     dp[mask][v] + w[v][u]);
130                 }
131             }
132         }
133     }
134     int full = (1<<n) - 1;
135     long long ans = LINF;
136     for(int v=0; v<n; v++){
137         if(dp[full][v] < LINF && w[v][src] < LINF){
138             ans = min(ans, dp[full][v] + w[v][src]);
139         }
140     }
141     return ans;
142 }
143
144 int main(){
145     // ejemplo: ciclo de 4 vertices (0-1-2-3-0) -> claramente hamiltoniano

```

```

144 int n = 4;
145 vector<vector<int>> adj(n);
146 auto add = [&](int u, int v){
147     adj[u].push_back(v); adj[v].push_back(u);
148 };
149 add(0,1); add(1,2); add(2,3); add(3,0);
150
151 cout << "Dirac: " << satisfiesDirac(n, adj) << " (n/2 = 2, todos grado
    2 -> true)\n";
152 cout << "Ore: " << satisfiesOre(n, adj) << "\n";
153 cout << "cycle? " << hasHamiltonianCycle(n, adj) << "\n";
154 cout << "path? " << hasHamiltonianPathFrom(n, adj, 0) << "\n";
155
156 auto path = findHamiltonianPath(n, adj, 0);
157 for(int v : path) cout << v << " ";
158 cout << "\n";
159
160 // TSP: mismo grafo con pesos -> ciclo minimo
161 vector<vector<long long>> w(n, vector<long long>(n, LINF));
162 for(int i=0;i<n;i++) w[i][i] = 0;
163 w[0][1] = w[1][0] = 10;
164 w[1][2] = w[2][1] = 20;
165 w[2][3] = w[3][2] = 30;
166 w[3][0] = w[0][3] = 40;
167 cout << "TSP min = " << tspMinCycle(n, w) << "\n"; // 10+20+30+40 = 100
168 return 0;
169 }

```

## 6 DP

### 6.1 Knapsack 0/1

```

1 // TC: O(N * W)
2 // usa: dado N items con peso w[i] y valor v[i], maximizar valor con peso
    total <= W
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int knapsack01(const vector<int>& w, const vector<int>& v, int W){
7     int n = w.size();
8     vector<int> dp(W + 1, 0);
9     for(int i=0;i<n;i++){
10         for(int j=W;j>=w[i];j--){
11             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
12         }
13     }
14     return dp[W];
15 }
16
17 int main(){
18     vector<int> w = {2,3,4}, v = {3,4,5};
19     cout << knapsack01(w, v, 5) << "\n"; // 7 (items 1+2)
20     return 0;

```

```

21 }

```

### 6.2 Knapsack unbounded

```

1 // TC: O(N * W)
2 // usa: como 0/1 pero cada item se puede tomar multiples veces
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int knapsackUnbounded(const vector<int>& w, const vector<int>& v, int W){
7     int n = w.size();
8     vector<int> dp(W + 1, 0);
9     for(int i=0;i<n;i++){
10         for(int j=w[i];j<=W;j++){
11             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
12         }
13     }
14     return dp[W];
15 }
16
17 int main(){
18     vector<int> w = {2,3}, v = {3,4};
19     cout << knapsackUnbounded(w, v, 7) << "\n"; // 11 (3+4+4)
20     return 0;
21 }

```

### 6.3 Knapsack meet-in-the-middle

```

1 // TC: O(2^(N/2))
2 // usa: cuando N ~ 40 y W no es tan pequeno
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 long long mitmKnapsack(const vector<int>& w, const vector<int>& v, long
    long W){
7     int n = w.size();
8     int n1 = n / 2, n2 = n - n1;
9     vector<long long> sums;
10    // enumerate all subsets of first half
11    for(int mask=0; mask<(1<<n1); mask++){
12        long long sw = 0, sv = 0;
13        for(int i=0;i<n1;i++) if(mask & (1<<i)){
14            sw += w[i]; sv += v[i];
15        }
16        if(sw <= W) sums.push_back(sv);
17    }
18    sort(sums.begin(), sums.end(), greater<long long>());
19    long long best = 0;
20    for(int mask=0; mask<(1<<n2); mask++){
21        long long sw = 0, sv = 0;
22        for(int i=0;i<n2;i++) if(mask & (1<<i)){
23            sw += w[n1+i]; sv += v[n1+i];

```

```

24     }
25     if(sw > W) continue;
26     long long rem = W - sw;
27     for(long long x : sums){
28         if(x <= rem){ best = max(best, sv + x); break; }
29     }
30 }
31 return best;
32 }
33
34 int main(){
35     vector<int> w = {1,2,3,4,5,6}, v = {1,2,3,4,5,6};
36     cout << mitmKnapsack(w, v, 10) << "\n"; // 10 (take 1+2+3+4 or similar)
37     return 0;
38 }

```

## 6.4 LIS $O(n \log n)$

```

1 // TC:  $O(N \log N)$ 
2 // usa: longitud de la LIS estrictamente creciente
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int lisLength(const vector<int>& a){
7     vector<int> dp;
8     for(int x : a){
9         auto it = lower_bound(dp.begin(), dp.end(), x);
10        if(it == dp.end()) dp.push_back(x);
11        else *it = x;
12    }
13    return (int)dp.size();
14 }
15
16 // TC:  $O(N \log N)$ , memoria  $O(N)$ 
17 // usa: reconstruye una LIS (la que el algoritmo encontro); una valida cualquiera
18 // no decreciente (LNDS): cambiar lower_bound -> upper_bound
19 // reconstruye via 'parent[i]' (forward) + 'dpIdx' (posicion final de la cola)
20 vector<int> lisSequence(const vector<int>& a){
21     int n = (int)a.size();
22     if(n == 0) return {};
23     vector<int> dp; // dp[k] = menor cola posible de una IS de long k+1
24     vector<int> dpIdx; // dpIdx[k] = indice en a de esa cola
25     vector<int> parent(n, -1); // parent[i] = indice previo en la LIS que termina en i
26     for(int i = 0; i < n; i++){
27         int pos = (int)(lower_bound(dp.begin(), dp.end(), a[i]) - dp.begin());
28         if(pos == (int)dp.size()){
29             dp.push_back(a[i]);
30             dpIdx.push_back(i);
31         }else{

```

```

32         dp[pos] = a[i];
33         dpIdx[pos] = i;
34     }
35     if(pos > 0) parent[i] = dpIdx[pos - 1];
36 }
37 // reconstruir desde la cola de la LIS
38 vector<int> res;
39 for(int i = dpIdx.back(); i != -1; i = parent[i])
40     res.push_back(a[i]);
41 reverse(res.begin(), res.end());
42 return res;
43 }
44
45 // TC:  $O(N \log N)$ , memoria  $O(N)$  (solo un vector extra: n enteros)
46 // usa: variante compacta. 'insertedAt[i]' = pos en dp donde cayo a[i].
47 // Reconstruye recorriendo de atras hacia adelante y agarrando
48 // el ultimo i con insertedAt[i] == need (need baja de L-1 a 0).
49 vector<int> lisSequenceCompact(const vector<int>& a){
50     int n = (int)a.size();
51     if(n == 0) return {};
52     vector<int> dp;
53     vector<int> insertedAt(n);
54     for(int i = 0; i < n; i++){
55         int pos = (int)(lower_bound(dp.begin(), dp.end(), a[i]) - dp.begin());
56         if(pos == (int)dp.size()) dp.push_back(a[i]);
57         else dp[pos] = a[i];
58         insertedAt[i] = pos;
59     }
60     vector<int> res;
61     for(int i = n - 1, need = (int)dp.size() - 1; i >= 0 && need >= 0; i--){
62         if(insertedAt[i] == need)
63             res.push_back(a[i]), need--;
64     }
65     reverse(res.begin(), res.end());
66     return res;
67 }
68
69 // Diferencia entre las 2:
70 // lisSequence usa 2 vectores extra (parent, dpIdx); reconstruye forward.
71 // lisSequenceCompact usa 1 vector extra (insertedAt); reconstruye backward.
72 // Compacta = menos memoria y menos codigo; basta para reconstruir una LIS.
73 // Si necesitas contar/todas las LIS, o generalizar el "anterior", usa la primera.
74
75 int main(){
76     vector<int> a = {1, 3, 2, 4, 5, 2};
77     cout << lisLength(a) << "\n"; // 4
78     auto seq = lisSequence(a);
79     for(int x : seq) cout << x << " "; // p.ej. 1 2 4 5 (alguna LIS valida)
80     cout << "\n";
81     auto seq2 = lisSequenceCompact(a);

```

```

81     for(int x : seq2) cout << x << " ";          // p.ej. 1 2 4 5   (alguna LIS
82         valida)
83     cout << "\n";
84     return 0;
}

```

## 6.5 LCS / Edit distance

```

1 // TC: O(N * M)
2 // usa: longest common subsequence y distancia de edicion (Levenshtein)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 int lcs(const string& a, const string& b){
7     int n = a.size(), m = b.size();
8     vector<vector<int>>> dp(n + 1, vector<int>(m + 1, 0));
9     for(int i=1; i<=n; i++){
10         for(int j=1; j<=m; j++){
11             if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
12             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
13         }
14     }
15     return dp[n][m];
16 }
17
18 int editDistance(const string& a, const string& b){
19     int n = a.size(), m = b.size();
20     vector<vector<int>>> dp(n + 1, vector<int>(m + 1, 0));
21     for(int i=0; i<=n; i++) dp[i][0] = i;
22     for(int j=0; j<=m; j++) dp[0][j] = j;
23     for(int i=1; i<=n; i++){
24         for(int j=1; j<=m; j++){
25             if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1];
26             else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
27         }
28     }
29     return dp[n][m];
30 }
31
32 int main(){
33     cout << lcs("abcde", "ace") << "\n";          // 3
34     cout << editDistance("kitten", "sitting") << "\n"; // 3
35     return 0;
36 }

```

## 6.6 Bitmask DP over subsets

```

1 // TC: O(N * 2^N)
2 // usa: DP donde el estado es un subconjunto de N elementos
3 #include <bits/stdc++.h>
4 using namespace std;
5

```

```

6 // iterate subsets in increasing order
7 // for(int mask=0; mask<(1<<N); mask++){ ... }
8
9 // iterate submasks of mask
10 // for(int sub=mask; sub; sub=(sub-1)&mask){ ... }
11
12 // iterate over supersets of mask (within universe U)
13 // for(int sup=mask; sup<U; sup=(sup+1)|mask){ ... }
14
15 int main(){
16     // iterate submasks of a mask
17     int mask = 0b1011;
18     for(int sub = mask; sub; sub = (sub-1) & mask){
19         // process sub
20     }
21     return 0;
22 }

```

## 6.7 SOS DP

```

1 // TC: O(N * 2^N), N = cantidad de bits (usar N <= 20)
2 //
3 // Las 4 variantes combinan 2 ejes:
4 // - Forward vs Backward: forward usa += (zeta / suma)
5 //                          backward usa -= (mobius / inversa)
6 // - Sufijo 1 vs 2:      1 opera sobre subsets (mask ^ (1<<i))
7 //                      2 opera sobre supersets (mask | (1<<i))
8 // Backward n es la inversa de Forward n.
9
10 #include <bits/stdc++.h>
11 using namespace std;
12
13 int bitsNeeded(int sz){
14     int n = 0;
15     while((1 << n) < sz) n++;
16     return n;
17 }
18
19 // dp[mask] = sum_{sub pertenece mask} f[sub]
20 // Caso clasico: para cada mask, sumar f sobre todas sus sub-masks.
21 vector<long long> sosForward1(const vector<long long>& f){
22     int n = bitsNeeded((int)f.size());
23     vector<long long> dp = f;
24     dp.resize(1 << n);
25     for(int i = 0; i < n; i++){
26         for(int mask = 0; mask < (1 << n); mask++){
27             if(mask & (1 << i))
28                 dp[mask] += dp[mask ^ (1 << i)];
29         }
30     }
31
32     // dp[mask] = sum_{sup contiene mask} f[sup]
33     // Para cada mask, sumar f sobre las masks que la contienen.
34     vector<long long> sosForward2(const vector<long long>& f){
35         int n = bitsNeeded((int)f.size());

```

```

36 vector<long long> dp = f;
37 dp.resize(1 << n);
38 for(int i = 0; i < n; i++)
39     for(int mask = 0; mask < (1 << n); mask++)
40         if(!(mask & (1 << i)))
41             dp[mask] += dp[mask | (1 << i)];
42 return dp;
43 }
44
45 // Inversa de sosForward1 (mobius en el lattice de subsets)
46 vector<long long> sosBackward1(const vector<long long>& g){
47     int n = bitsNeeded((int)g.size());
48     vector<long long> dp = g;
49     dp.resize(1 << n);
50     for(int i = 0; i < n; i++)
51         for(int mask = 0; mask < (1 << n); mask++)
52             if(mask & (1 << i))
53                 dp[mask] -= dp[mask ^ (1 << i)];
54     return dp;
55 }
56
57 // Inversa de sosForward2 (mobius en el lattice de supersets)
58 vector<long long> sosBackward2(const vector<long long>& g){
59     int n = bitsNeeded((int)g.size());
60     vector<long long> dp = g;
61     dp.resize(1 << n);
62     for(int i = 0; i < n; i++)
63         for(int mask = 0; mask < (1 << n); mask++)
64             if(!(mask & (1 << i)))
65                 dp[mask] -= dp[mask | (1 << i)];
66     return dp;
67 }
68
69 int main(){
70     vector<long long> f = {1, 2, 3, 4}; // n = 2
71
72     auto f1 = sosForward1(f); // {1, 3, 4, 10}
73     auto f2 = sosForward2(f); // {10, 6, 7, 4}
74     auto b1 = sosBackward1(f1); // {1, 2, 3, 4} (recupera f)
75     auto b2 = sosBackward2(f2); // {1, 2, 3, 4} (recupera f)
76     return 0;
77 }

```

## Guía rápida.

- **sosForward1**: Zeta sobre subconjuntos. Acumula hacia arriba:  $dp[mask] = \sum_{S \subseteq mask} f[S]$ .
- **sosForward2**: Zeta sobre superconjuntos. Acumula hacia abajo:  $dp[mask] = \sum_{mask \subseteq S} f[S]$ .
- **sosBackward1** / **sosBackward2**: Inversión de Möbius (deshacen el forward respectivo cambiando += por -=).

## Cómo usarlo y aplicaciones comunes:

1. **Consultas de bits sobre un arreglo (ej. CSES Bit Problem)**: Sea  $f[val]$  la frecuencia de cada número en el arreglo de  $N$  elementos:

- $x \mid y = x \iff y \subseteq x$  (contar  $y$  que sean submáscaras de  $x$ ): Se resuelve directamente con  $sosForward1(f)[x]$ .
- $x \& y = x \iff x \subseteq y$  (contar  $y$  que sean supermáscaras de  $x$ ): Se resuelve directamente con  $sosForward2(f)[x]$ .
- $x \& y \neq 0$  (contar  $y$  que comparten al menos un bit con  $x$ ): Se calcula por complemento:  $N - sosForward1(f)[((1 \ll n) - 1) \oplus x]$ .

2. **Convoluciones Bitwise (OR y AND) tipo FFT en  $O(N \cdot 2^N)$** :

- **Convolución OR** ( $C[k] = \sum_{i \mid j=k} A[i] \cdot B[j]$ ):
  1.  $A' = sosForward1(A)$ ,  $B' = sosForward1(B)$
  2. Multiplicar punto a punto:  $C'[k] = A'[k] \cdot B'[k]$
  3. Recuperar con  $C = sosBackward1(C')$
- **Convolución AND** ( $C[k] = \sum_{i \& j=k} A[i] \cdot B[j]$ ):
  1.  $A' = sosForward2(A)$ ,  $B' = sosForward2(B)$
  2. Multiplicar punto a punto:  $C'[k] = A'[k] \cdot B'[k]$
  3. Recuperar con  $C = sosBackward2(C')$

3. **Principio de Inclusión-Exclusión automático**: Al contar combinaciones donde es fácil calcular “al menos  $mask$ ” ( $mask \subseteq S$ ), aplicar **sosBackward2** recupera cuántos cumplen “exactamente  $mask$ ” en  $O(N \cdot 2^N)$ , realizando automáticamente toda la inclusión-exclusión con signos alternados  $(-1)^{|S \setminus mask|}$ .

4. **Problema clásico: CF 449D (Jzzhu and Numbers) — AND de subsecuencias = 0**:

- **Enunciado**: Dado un arreglo de  $N$  enteros ( $N, a_i \leq 10^6 < 2^{20}$ ), contar cuántas subsecuencias no vacías tienen un AND bitwise igual a 0:  $a_{i_1} \& a_{i_2} \& \dots \& a_{i_k} = 0 \pmod{10^9 + 7}$ .
- **Idea**: Para que el AND de una subsecuencia tenga todos los bits de una máscara  $S$  encendidos ( $S \subseteq \text{AND}$ ), **cada** elemento elegido debe cumplir  $S \subseteq a_j$ .
- **Paso 1 (SOS DP)**: Calculamos  $cnt[S] = \sum_{S \subseteq x} \text{freq}[x]$  con **sosForward2**.
- **Paso 2 (Subsecuencias)**: Hay  $2^{cnt[S]} - 1$  subsecuencias cuyo AND contiene a  $S$ .
- **Paso 3 (Inclusión-Exclusión)**: Para que el AND tenga **exactamente 0** bits encendidos:

$$ans = \sum_{S=0}^{2^{20}-1} (-1)^{|S|} \cdot 2^{cnt[S]} \pmod{10^9 + 7}$$

donde  $|S| = \text{popcount}(S)$  (los  $-1$  de  $2^{cnt[S]} - 1$  se cancelan pues  $\sum (-1)^{|S|} = 0$ ). Complejidad:  $O(N + 20 \cdot 2^{20})$ .

## 6.8 Digit DP template

```

1 // TC: O(D * 10 * state_size) where D is number of digits
2 // usa: contar numeros en [0, N] que cumplen una propiedad sobre sus
  digitos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // skeleton only; adapt state, transition, and base case to your problem
7 string digitStr;
8 long long memo[20][2][200]; // dimensions depend on problem (pos, tight,
  state)
9
10 long long updateState(long long state, int d){
11     // problem-specific; return new state after appending digit d
12     return state;
13 }
14
15 long long digitDFS(int pos, bool tight, long long state){
16     if(pos == (int)digitStr.size()) return 1; // adapt per problem (e.g.,
  state matches)
17     if(!tight && memo[pos][0][state] != -1) return memo[pos][0][state];
18     int limit = tight ? digitStr[pos] - '0' : 9;
19     long long res = 0;
20     for(int d=0; d<=limit; d++){
21         long long ns = updateState(state, d);
22         res += digitDFS(pos + 1, tight && (d == limit), ns);
23     }
24     if(!tight) memo[pos][0][state] = res;
25     return res;
26 }
27
28 long long digitDP(long long N){
29     digitStr = to_string(N);
30     memset(memo, -1, sizeof(memo));
31     return digitDFS(0, true, 0);
32 }
33
34 int main(){
35     // count numbers in [0, 999] whose digit sum equals 10
36     digitStr = to_string(999);
37     memset(memo, -1, sizeof(memo));
38     cout << digitDFS(0, true, 0) << "\n"; // (skeleton; fill updateState)
39     return 0;
40 }

```

**Versión iterativa (forward).** Tabla `dp[pos][smaller][state]` llenada de `pos=0` a `pos=D`. `smaller=0`  $\Rightarrow$  `tight` (prox dígito  $\leq N[pos]$ ); `smaller=1`  $\Rightarrow$  libre (0..9).

```

1 // Cuenta cuantos x en [0, N) (= [0, N-1]) cumplen una propiedad sobre
  sus digitos.
2 // Para rango [a, b] INCLUSIVO: digitDP(b+1) - digitDP(a).
3 // TC: O(D * 10 * (STATE_MAX + 1)). Memoria: O(D * 2 * (STATE_MAX + 1))
  .
4
5 // Ejemplo de abajo: contar x sin dos digitos adyacentes iguales.

```

```

6 // state = "ultimo dígito colocado" (o NOT_STARTED al principio).
7 long long digitDP(long long N){
8     // 1) digitos de N, mas significativo primero.
9     vector<int> dig;
10    if(N == 0){
11        dig = {0};
12    }else{
13        long long t = N;
14        while(t > 0){ dig.push_back((int)(t % 10)); t /= 10; }
15        reverse(dig.begin(), dig.end());
16    }
17    int D = (int)dig.size();
18
19    // 2) cota + sentinel. Para "no adyacentes iguales" alcanza 10.
20    // adaptar, ahorita esta del 0 al 9 y el 10 de flag
21    const int STATE_MAX = 10;
22    const int NOT_STARTED = STATE_MAX;
23
24    // 3) dp[pos][smaller][state], state en [0, STATE_MAX].
25    vector<vector<vector<long long>>> dp(D + 1,
26        vector<vector<long long>>(2, vector<long long>(STATE_MAX + 1, 0)));
27    dp[0][0][NOT_STARTED] = 1;
28
29    // 4) DP.
30    for(int pos = 0; pos < D; pos++){
31        for(int smaller = 0; smaller <= 1; smaller++){
32            for(int digit = 0; digit <= STATE_MAX; digit++){
33                long long ways = dp[pos][smaller][digit];
34                // poda
35                if(ways == 0) continue;
36                for(int nxt = 0; nxt <= (smaller ? 9 : dig[pos]); nxt++){
37                    // en este caso no puede haber 2 digitos consecutivos iguales
38                    // se puede modificar para se salten transiciones no validas
39                    if(nxt == digit) continue;
40
41                    int new_digit = nxt;
42                    if(digit == NOT_STARTED && nxt == 0){
43                        new_digit = NOT_STARTED;
44                    }
45                    dp[pos + 1][smaller || (nxt < dig[pos])][new_digit] += ways;
46                }
47            }
48        }
49    }
50
51    // 5) Finales validos. dp[D][1][*] = "< N"; dp[D][0][*] = N exacto (no
  se suma).
52    // Para "no adyacentes": cualquier final vale (incluye NOT_STARTED
  -> el numero 0).
53    long long ans = 0;
54    for(int digit = 0; digit <= STATE_MAX; digit++){
55        ans += dp[D][1][digit]; // considera menores a N, [0,N)
56    }
57    return ans;
58 }
59
60 int main(){

```

```

60 // Ejemplo: contar x en [0, N) sin dos digitos adyacentes iguales.
61 cout << digitDP(11) << "\n"; // -> 11 (0..10 todos validos)
62 cout << digitDP(12) << "\n"; // -> 11 (sin el 11)
63 cout << digitDP(100) << "\n"; // -> 91
64 return 0;
65 }

```

## 6.9 Tree DP (reroot)

```

1 // TC: O(N)
2 // usa: para cada nodo, calcular alguna propiedad asumiendo que ese nodo
   es la raiz
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct TreeDP{
7     int n;
8     vector<vector<int>>> adj;
9     vector<long long> dp, up, ans;
10    vector<int> sz;
11    TreeDP(int n): n(n){
12        adj.assign(n, {});
13        dp.assign(n, 0);
14        up.assign(n, 0);
15        ans.assign(n, 0);
16        sz.assign(n, 0);
17    }
18    void addEdge(int u, int v){
19        adj[u].push_back(v);
20        adj[v].push_back(u);
21    }
22    // 1st pass: compute dp[v] as the answer for subtree of v (rooted at 0)
23    void dfs1(int v, int p){
24        sz[v] = 1;
25        for(int u : adj[v]){
26            if(u == p) continue;
27            dfs1(u, v);
28            sz[v] += sz[u];
29            dp[v] += dp[u] + sz[u]; // example - sum of distances from u
30        }
31    }
32    // 2nd pass: reroot - propagate contributions
33    void dfs2(int v, int p){
34        ans[v] = dp[v] + up[v];
35        for(int u : adj[v]){
36            if(u == p) continue;
37            // when rerooting at u, u's contribution comes out, v's grows
38            up[u] = up[v] + dp[v] - (dp[u] + sz[u]) + (n - sz[u]);
39            dfs2(u, v);
40        }
41    }
42    vector<long long> solve(int root = 0){
43        dfs1(root, -1);
44        dfs2(root, -1);
45        return ans;

```

```

46    }
47 };
48
49 int main(){
50     TreeDP tdp(4);
51     tdp.addEdge(0,1); tdp.addEdge(1,2); tdp.addEdge(1,3);
52     auto ans = tdp.solve(0);
53     for(long long x : ans) cout << x << " "; // sum of distances from each
        node
54     cout << "\n";
55     return 0;
56 }

```

## 7 Geometria

### 7.1 Puntos y segmentos

#### 7.1 Conceptos y formulas de geometria 2D

1. **Vectores, Dot Product y Cross Product.** Un punto o vector en 2D es  $P = (x, y)$ . La resta  $\vec{u} = B - A = (x_B - x_A, y_B - y_A)$  representa el vector que va de A a B.

- **Magnitud:**  $|\vec{u}| = \sqrt{x^2 + y^2}$ , con  $|\vec{u}|^2 = x^2 + y^2$ . En CP, siempre que sea posible compara distancias al cuadrado para trabajar con enteros y evitar perdida de precision por punto flotante.

- **Producto Punto (Dot Product):**

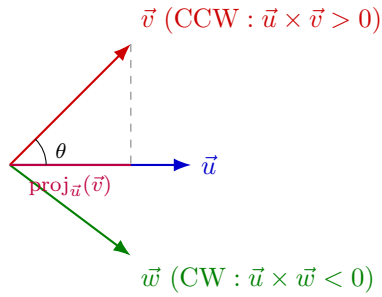
$$\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2 = |\vec{u}| |\vec{v}| \cos \theta$$

- $\vec{u} \cdot \vec{v} > 0 \iff \theta < 90^\circ$  (angulo agudo).
- $\vec{u} \cdot \vec{v} = 0 \iff \theta = 90^\circ$  (perpendiculares / ortogonales).
- $\vec{u} \cdot \vec{v} < 0 \iff \theta > 90^\circ$  (angulo obtuso).
- Proyeccion escalar de  $\vec{u}$  sobre  $\vec{v}$ :  $\frac{\vec{u} \cdot \vec{v}}{|\vec{v}|}$ . Proyeccion vectorial:  $\text{proj}_{\vec{v}}(\vec{u}) = \frac{\vec{u} \cdot \vec{v}}{|\vec{v}|^2} \vec{v}$ .

- **Producto Cruz 2D (Cross Product / Pseudocruz):**

$$\vec{u} \times \vec{v} = x_1 y_2 - y_1 x_2 = |\vec{u}| |\vec{v}| \sin \theta$$

- $|\vec{u} \times \vec{v}| = \text{Area del paralelogramo}$  formado por  $\vec{u}$  y  $\vec{v}$ .
- Area con signo del triangulo  $\triangle ABC$ :  $\frac{1}{2}((B - A) \times (C - A))$ .
- $\vec{u} \times \vec{v} > 0 \iff \vec{v}$  esta a la **izquierda** de  $\vec{u}$  (giro antihorario / CCW).
- $\vec{u} \times \vec{v} < 0 \iff \vec{v}$  esta a la **derecha** de  $\vec{u}$  (giro horario / CW).
- $\vec{u} \times \vec{v} = 0 \iff \vec{u}$  y  $\vec{v}$  son **colineales / paralelos**.



**2. Criterios de Paralelismo y Perpendicularidad.** Dados dos vectores  $\vec{u} = (x_1, y_1)$  y  $\vec{v} = (x_2, y_2)$ :

- **Paralelos** ( $\vec{u} \parallel \vec{v}$ ):

$$\vec{u} \times \vec{v} = x_1 y_2 - y_1 x_2 = 0$$

Misma direccion y sentido si  $\vec{u} \cdot \vec{v} > 0$ ; sentidos opuestos si  $\vec{u} \cdot \vec{v} < 0$ .

- **Perpendiculares** ( $\vec{u} \perp \vec{v}$ ):

$$\vec{u} \cdot \vec{v} = x_1 x_2 + y_1 y_2 = 0$$

- **Vector perpendicular rotado:** Rotar  $\vec{u} = (x, y)$  en  $90^\circ$  antihorario da  $\vec{u}^\perp = (-y, x)$ . Rotar  $-90^\circ$  (horario) da  $(y, -x)$ .

**3. Orientacion de 3 puntos (Prueba CCW).** Dados los puntos  $A, B, C$ , evaluar:

$$\text{orient}(A, B, C) = (B - A) \times (C - A) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A)$$

- $> 0$ :  $C$  queda a la izquierda de la recta dirigida  $A \rightarrow B$  (vuelta a la izquierda / CCW).
- $< 0$ :  $C$  queda a la derecha de la recta dirigida  $A \rightarrow B$  (vuelta a la derecha / CW).
- $= 0$ :  $A, B, C$  son colineales.

**4. Lineas y Distancias.**

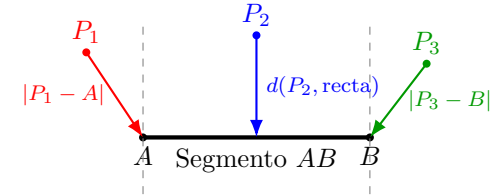
- **Ecuacion general de la recta:**  $Ax + By + C = 0$ . Dados dos puntos  $P$  y  $Q$ :  $A = y_Q - y_P$ ,  $B = x_P - x_Q$ ,  $C = -(Ax_P + By_P)$ .

- **Distancia punto a recta:** Distancia del punto  $P$  a la recta que pasa por  $A$  y  $B$ :

$$d(P, \text{recta}(AB)) = \frac{|(B - A) \times (P - A)|}{|B - A|} = \frac{|Ax_P + By_P + C|}{\sqrt{A^2 + B^2}}$$

- **Distancia punto a segmento  $AB$ :**

- Si  $(B - A) \cdot (P - A) \leq 0$ , el punto mas cercano del segmento es  $A \implies d = |P - A|$ .
- Si  $(A - B) \cdot (P - B) \leq 0$ , el punto mas cercano del segmento es  $B \implies d = |P - B|$ .
- En otro caso, la proyeccion cae dentro del segmento  $\implies d = d(P, \text{recta}(AB))$ .



- **Interseccion de dos rectas:**  $A_1x + B_1y + C_1 = 0$  y  $A_2x + B_2y + C_2 = 0$ . Determinante  $D = A_1B_2 - A_2B_1$ .

- Si  $D = 0$ : las rectas son paralelas ( $C_1A_2 \neq C_2A_1$ ) o coincidentes ( $C_1A_2 = C_2A_1$ ).
- Si  $D \neq 0$ : interseccion unica en:

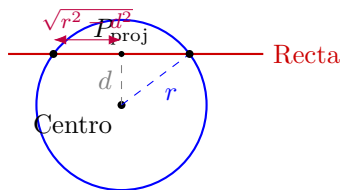
$$x = \frac{B_1C_2 - B_2C_1}{D}, \quad y = \frac{C_1A_2 - C_2A_1}{D}$$

**5. Circunferencias (Circulos).**

- **Ecuacion:**  $(x - h)^2 + (y - k)^2 = r^2$ , con centro  $(h, k)$  y radio  $r$ .
- **Interseccion recta-circunferencia:** Calcular  $d = d(\text{centro}, \text{recta})$ .
  - $d > r$ : 0 intersecciones (exterior).
  - $d = r$ : 1 interseccion (tangente en la proyeccion  $P_{\text{proj}}$  del centro sobre la recta).
  - $d < r$ : 2 intersecciones. Los puntos estan a distancia  $h = \sqrt{r^2 - d^2}$  a ambos lados de  $P_{\text{proj}}$  a lo largo del vector unitario director de la recta.
- **Interseccion entre dos circunferencias:** Centros  $C_1, C_2$  con radios  $r_1, r_2$  y distancia  $d = |C_1 - C_2|$ :
  - $d > r_1 + r_2$  o  $d < |r_1 - r_2|$ : no se intersectan (disjuntos externos o uno contenido en otro).
  - $d = 0$  y  $r_1 = r_2$ : infinitos puntos (circunferencias identicas).
  - $d = r_1 + r_2$  o  $d = |r_1 - r_2|$ : 1 punto (tangencia exterior o interior).
  - En otro caso: 2 puntos de corte simetricos respecto a la linea de centros.

- **Circuncentro de un triángulo**  $\triangle ABC$ : Punto de intersección de las tres mediatrices. Radio de la circunferencia circunscrita:

$$R = \frac{a \cdot b \cdot c}{4 \cdot \text{Area}} = \frac{|B-A| |C-B| |A-C|}{2 |(B-A) \times (C-A)|}$$



## 6. Tips críticos de precisión y tipos en CP.

- **Evitar flotantes en orientación:** Al hacer cross product de coordenadas  $\leq 10^9$ , el resultado puede llegar a  $2 \cdot 10^{18}$ , cabiendo en long long. Si se multiplica con otras distancias, usar \_\_int128\_t.
- **Comparación con EPS:** Para flotantes (double), usar constante  $\epsilon = 10^{-9}$ :
  - Igualdad:  $\text{abs}(a - b) < \text{EPS}$
  - Menor estricto:  $a < b - \text{EPS}$
  - Menor o igual:  $a < b + \text{EPS}$
- **Angulo de un vector:** Usar  $\text{atan2}(y, x)$ , que devuelve el ángulo exacto en  $[-\pi, \pi]$  considerando signos de ambos cuadrantes sin división por cero.

## 7.2 Point + cross + dot

```

1 struct Point{
2     int x,y;
3     void read(){cin >> x >> y;}
4     Point operator -(const Point &b) const {
5         return Point{x-b.x , y-b.y};
6     }
7
8     // Respecto al origen
9     int operator *(const Point &b) const {
10        return x*b.y - y*b.x;
11    }
12
13    // Respecto a this
14    int cross(const Point &b, const Point &c) const {
15        return (b-*this) * (c-*this);
16    }
17    int dot(const Point &b, const Point &c) const {
18        return (b.x-x)*(c.x-x) + (b.y-y)*(c.y-y);
19    }

```

```

20 bool operator <(const Point &b) const {
21     return make_pair(x,y)<make_pair(b.x,b.y);
22 }
23 };

```

## 7.3 Intersección de segmentos

```

1 // TC: O(1)
2 // usa: chequea si los segmentos [a,b] y [c,d] se intersectan (incluye
3 //      colineales solapados)
4 // nota: usa pruebas de orientación (cross products) + bounding boxes; no
5 //      retorna punto de cruce
6 // linear segments intersection
7 bool intersect(Point a, Point b, Point c, Point d){
8     // check parallel or collinear
9     if( (b-a) * (d-c) == 0 ){
10        // check if parallel
11        if(a.cross(b,c) != 0){
12            return false;
13        }
14        // check if collinear, bounding box
15        for(int i=0 ; i<2 ; i++){
16            if(max(a.x,b.x)<min(c.x,d.x) || max(a.y,b.y)<min(c.y,d.y)){
17                return false;
18            }
19            swap(a,c);
20            swap(b,d);
21        }
22        return true;
23    }
24    for(int i=0 ; i<2 ; i++){
25        int x=a.cross(b,c) , y=a.cross(b,d);
26        if( (x<0 && y<0) || (x>0 && y>0)){
27            return false;
28        }
29        swap(a,c);
30        swap(b,d);
31    }
32    return true;
33 }

```

## 7.4 Contains colinear

```

1 // TC: O(1)
2 // usa: chequea si el punto c esta en el segmento [a,b] (colinear y dentro
3 //      del bounding box)
4 bool contains(Point a, Point b, Point c){
5     if( a.cross(b,c) == 0){
6         return (min(a.x,b.x)<=c.x) && (c.x<=max(a.x,b.x))
7             && (min(a.y,b.y)<=c.y) && (c.y<=max(a.y,b.y));
8     }
9     return false;
10 }

```

## 7.5 Distance point-segment / point-line

```
1 // TC: O(1)
2 // usa: distancia minima de un punto a un segmento o a la recta que lo
   contiene
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 // returns squared distance from p to line (a, b)
7 long long dist2PointLine(const Point& p, const Point& a, const Point& b){
8     Point ab = b - a;
9     long long area2 = abs((long long)ab * (p - a));
10    long long len2 = 1LL * ab.x * ab.x + 1LL * ab.y * ab.y;
11    if(len2 == 0) return 1LL * (p.x - a.x) * (p.x - a.x) + 1LL * (p.y - a.y)
        * (p.y - a.y);
12    return (area2 * area2) / len2;
13 }
14
15 long long dist2PointSegment(const Point& p, const Point& a, const Point&
    b){
16     Point ap = p - a;
17     Point ab = b - a;
18     long long dot1 = 1LL * ab.x * ap.x + 1LL * ab.y * ap.y;
19     if(dot1 <= 0) return 1LL * ap.x * ap.x + 1LL * ap.y * ap.y;
20     Point bp = p - b;
21     long long dot2 = -1LL * ab.x * bp.x - 1LL * ab.y * bp.y;
22     if(dot2 <= 0) return 1LL * bp.x * bp.x + 1LL * bp.y * bp.y;
23     long long len2 = 1LL * ab.x * ab.x + 1LL * ab.y * ab.y;
24     long long area2 = abs((long long)ab * ap);
25     return (area2 * area2) / len2;
26 }
27
28 int main(){
29     Point a{0,0}, b{4,0}, p{2,3};
30     cout << "dist to segment: " << dist2PointSegment(p,a,b) << "\n"; // 9
31     cout << "dist to line: " << dist2PointLine(p,a,b) << "\n"; // 9
32     return 0;
33 }
```

## 7.6 Projection of point onto line

```
1 // TC: O(1)
2 // usa: la proyeccion ortogonal de p sobre la recta que pasa por a y b (
   devuelve double)
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 pair<double,double> projectPointLine(const Point& p, const Point& a,
    const Point& b){
7     double ax = a.x, ay = a.y, bx = b.x, by = b.y, px = p.x, py = p.y;
8     double dx = bx - ax, dy = by - ay;
9     double len2 = dx*dx + dy*dy;
10    if(len2 == 0) return {ax, ay};
11    double t = (dx * (px - ax) + dy * (py - ay)) / len2;
```

```
12    return {ax + t * dx, ay + t * dy};
13 }
14
15 int main(){
16     Point a{0,0}, b{4,0}, p{2,3};
17     auto [px, py] = projectPointLine(p, a, b);
18     cout << "projection = (" << px << ", " << py << ")\n"; // (2, 0)
19     return 0;
20 }
```

## 7.2 Poligonos

### 7.1 Convex Hull (monotono)

```
1 // TC: O(N log N) por sort; el barrido es amortizado O(N) (cada pop_back
   se hace a lo sumo N veces por iter)
2 // usa: devuelve los puntos del casco convexo en orden CCW (sin
   colineales si cambias <=0 a <0)
3 vector<Point> convexHull(vector<Point> points) {
4     sort(points.begin(), points.end());
5
6     vector<Point> hull;
7     for(int iter=0; iter<2; iter++) {
8         int S = hull.size();
9         for(const Point &c : points) {
10             while((int)hull.size() >= S+2) {
11                 Point a = hull.end()[-2], b = hull.end()[-1];
12                 // <=0 incluye colineales
13                 if(a.cross(b, c) <= 0) break;
14                 hull.pop_back();
15             }
16             hull.push_back(c);
17         }
18         hull.pop_back(); // evita duplicar el ultimo punto
19         reverse(points.begin(), points.end());
20     }
21
22     return hull;
23 }
```

### 7.2 Polygon Area

```
1 int polygonArea(const vector<Point> &poly) {
2     int area = 0;
3     int n = poly.size();
4     for(int i = 0; i < n; i++) {
5         area += poly[i] * poly[(i+1) % n]; // Producto cruz
6     }
7     return abs(area); // este es el doble del area
8 }
```

### 7.3 Point in polygon (ray casting)

```
1 // TC: O(N)
2 // usa: comprobar si un punto esta dentro de un poligono simple
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 bool pointInPolygon(const Point& p, const vector<Point>& poly){
7     int n = (int)poly.size();
8     bool inside = false;
9     for(int i=0,j=n-1 ; i<n ; j=i++){
10         if((poly[i].y > p.y) != (poly[j].y > p.y)){
11             long long num = 1LL * (p.y - poly[j].y) * (poly[i].x - poly[j].x);
12             long long den = poly[i].y - poly[j].y;
13             long long x_intersect = poly[j].x + num / den;
14             if(p.x < x_intersect) inside = !inside;
15         }
16     }
17     return inside;
18 }
19
20 int main(){
21     vector<Point> square = {{0,0},{4,0},{4,4},{0,4}};
22     cout << pointInPolygon({2,2}, square) << "\n"; // 1 (inside)
23     cout << pointInPolygon({5,5}, square) << "\n"; // 0 (outside)
24     return 0;
25 }
```

### 7.4 Minkowski sum (convex polygons)

```
1 // TC: O(N + M)
2 // usa: suma de Minkowski; util para chequear colision entre dos
3 // poligonos convexos
4 #include <bits/stdc++.h>
5 using namespace std;
6
7 vector<Point> minkowskiSum(const vector<Point>& a, const vector<Point>& b)
8 {
9     // assumes both are in counter-clockwise order, no repeated last point
10     int n = (int)a.size(), m = (int)b.size();
11     auto edgesOf = [](const vector<Point>& p){
12         vector<Point> e;
13         int k = (int)p.size();
14         for(int i=0;i<k;i++) e.push_back(p[(i+1)%k] - p[i]);
15         return e;
16     };
17     vector<Point> ea = edgesOf(a), eb = edgesOf(b);
18     int i = 0, j = 0;
19     vector<Point> result;
20     Point cur{a[0].x + b[0].x, a[0].y + b[0].y};
21     result.push_back(cur);
22     while(i < n && j < m){
23         Point pa = ea[i], pb = eb[j];
24         long long cr = (long long)pa * pb;
```

```
23         if(cr >= 0){ cur.x += pa.x; cur.y += pa.y; i++; }
24         if(cr <= 0){ cur.x += pb.x; cur.y += pb.y; j++; }
25         result.push_back(cur);
26     }
27     while(i < n){ cur.x += ea[i].x; cur.y += ea[i].y; result.push_back(cur); i++; }
28     while(j < m){ cur.x += eb[j].x; cur.y += eb[j].y; result.push_back(cur); j++; }
29     return result;
30 }
31
32 int main(){
33     vector<Point> A = {{0,0},{2,0},{2,2},{0,2}}; // unit-ish square
34     vector<Point> B = {{1,1},{3,1},{3,3},{1,3}};
35     auto C = minkowskiSum(A, B);
36     cout << "Minkowski sum has " << C.size() << " vertices\n";
37     return 0;
38 }
```

### 7.3 Herramientas

#### 7.1 Li Chao Tree (long long)

```
1 // TC: O(log X) per query/insert
2 // usa: mantener un lower envelope de lineas y consultar en cualquier x
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 const long long LINF = (1LL<<60);
7
8 struct Line{
9     long long m, b; // y = m*x + b
10     Line(long long m=0, long long b=LINF): m(m), b(b){}
11     long long get(long long x) const { return m*x + b; }
12 };
13
14 struct LiChao{
15     struct Node{
16         Line line;
17         Node *l, *r;
18         Node(const Line& line): line(line), l(nullptr), r(nullptr){}
19     };
20     long long X_LO, X_HI;
21     Node* root;
22     LiChao(long long X_LO, long long X_HI): X_LO(X_LO), X_HI(X_HI){
23         root = nullptr;
24     }
25     void addLine(Line newLine){ addLine(root, X_LO, X_HI, newLine); }
26     void addLine(Node*& node, long long lo, long long hi, Line newLine){
27         if(!node){
28             node = new Node(newLine);
29             return;
30         }
31         long long mid = (lo + hi) >> 1;
```

```

32 bool left = newLine.get(lo) < node->line.get(lo);
33 bool middle = newLine.get(mid) < node->line.get(mid);
34 if(middle){
35     swap(node->line, newLine);
36 }
37 if(hi - lo == 0) return;
38 if(left != middle){
39     addLine(node->l, lo, mid, newLine);
40 }else{
41     addLine(node->r, mid + 1, hi, newLine);
42 }
43 }
44 long long query(long long x){ return query(root, X_LO, X_HI, x); }
45 long long query(Node* node, long long lo, long long hi, long long x){
46     if(!node) return LINF;
47     long long res = node->line.get(x);
48     if(lo == hi) return res;
49     long long mid = (lo + hi) >> 1;
50     if(x <= mid) return min(res, query(node->l, lo, mid, x));
51     else return min(res, query(node->r, mid + 1, hi, x));
52 }
53 };
54
55 int main(){
56     LiChao lc(-(long long)1e9, (long long)1e9);
57     lc.addLine(Line(1, 0)); // y = x
58     lc.addLine(Line(-1, 5)); // y = -x + 5
59     cout << "min at x=2: " << lc.query(2) << "\n"; // -2+5 = 3
60     cout << "min at x=10: " << lc.query(10) << "\n"; // 10 vs -5 -> -5
61     return 0;
62 }

```

## 7.2 Sweep line template

```

1 // TC: depends on problem; usually  $O((N + E) \log N)$ 
2 // usa: problemas donde se barre una linea/punto y se mantiene una EDD
   sobre eventos
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 struct SweepEvent{
7     long long x;
8     int type; // 0 = add, 1 = query, 2 = remove (problem-specific)
9     int id;
10    bool operator<(const SweepEvent& o) const { return x < o.x; }
11 };
12
13 // usage:
14 //     sort events by x
15 //     maintain a data structure (set, multiset, segment tree) keyed by y
16 //     process events in order, calling add/query/remove handlers
17
18 int main(){
19     // skeleton: fill events with x, type, id and process in order
20     vector<SweepEvent> evts = {{1, 0, 0}, {3, 1, 1}, {5, 2, 0}};

```

```

21     sort(evts.begin(), evts.end());
22     for(auto& e : evts) cout << "(" << e.x << ", " << e.type << ", " << e.id
23         << ") ";
24     cout << "\n";
25     return 0;
26 }

```

## 7.3 Closest pair of points (sweepline)

```

1 // TC:  $O(N \log N)$ 
2 // usa: encontrar el par de puntos mas cercanos en un plano
3 // idea: ordenar por x, mantener una "caja" (set ordenado por y) con
   puntos
4 //     cuya x esta a  $\leq d$  del actual; evict y query acotados por d
5 // nota: hypot(dx,dy) =  $\sqrt{dx^2+dy^2}$  sin overflow intermedio (mas
   seguro que  $\sqrt{dx*dx+dy*dy}$ )
6 #include <bits/stdc++.h>
7 using namespace std;
8
9 double closestPair(vector<Point>& pts){
10     sort(pts.begin(), pts.end(), [](const Point& a, const Point& b){
11         return a.x < b.x || (a.x == b.x && a.y < b.y);
12     });
13     double d = 1e18;
14     set<pair<long long, long long>> box;
15     int j = 0;
16     for(int i = 0; i < (int)pts.size(); i++){
17         long long xi = pts[i].x, yi = pts[i].y;
18         while(j < i && (double)(xi - pts[j].x) > d){
19             box.erase({pts[j].y, pts[j].x});
20             j++;
21         }
22         long long ylow = (long long)floor((double)yi - d);
23         long long yhigh = (long long)ceil((double)yi + d);
24         for(auto it = box.lower_bound({ylow, LLONG_MIN});
25             it != box.end() && it->first <= yhigh; ++it){
26             double dist = hypot((double)(xi - it->second), (double)(yi - it->
27                 first));
28             if(dist < d) d = dist;
29         }
30         box.insert({yi, xi});
31     }
32     return d;
33 }
34
35 int main(){
36     vector<Point> pts = {{0,0}, {5,5}, {1,1}, {3,3}, {0,2}};
37     cout << "closest distance = " << closestPair(pts) << "\n"; // ~1.41
38     return 0;
39 }

```

## 7.4 Rotating calipers (polygon diameter)

```

1 // TC: O(N) (assumes convex polygon sorted counter-clockwise)
2 // usa: encontrar el par de vertices mas distantes (diametro) en un
   poligono convexo
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 long long rotatingCalipers(const vector<Point>& p){
7     int n = (int)p.size();
8     if(n == 1) return 0;
9     if(n == 2){
10         long long dx = p[0].x - p[1].x, dy = p[0].y - p[1].y;
11         return dx*dx + dy*dy;
12     }
13     long long best = 0;
14     int k = 1;
15     for(int i=0;i<n;i++){
16         int ni = (i + 1) % n;
17         while(true){
18             int nk = (k + 1) % n;
19             long long cur = abs((long long)(p[ni].x - p[i]) * (p[nk].y - p[k]));
20             long long nxt = abs((long long)(p[ni].x - p[i]) * (p[(nk+1)%n].y - p[(k
21 +1)%n].y)));
22             if(nxt > cur) k = nk;
23             else break;
24         }
25         long long d1 = 1LL * (p[i].x - p[k].x) * (p[i].x - p[k].x) + 1LL * (p
26 [i].y - p[k].y) * (p[i].y - p[k].y);
27         long long d2 = 1LL * (p[ni].x - p[k].x) * (p[ni].x - p[k].x) + 1LL *
28 (p[ni].y - p[k].y) * (p[ni].y - p[k].y);
29         best = max(best, max(d1, d2));
30     }
31     return best;
32 }
33
34 int main(){
35     vector<Point> tri = {{0,0}, {10,0}, {5,8}}; // triangle CCW
36     cout << "diameter^2 = " << rotatingCalipers(tri) << "\n";
37     return 0;
38 }

```